VIDFUSION: AN AI-POWERED MULTI-VIDEO SUMMARIZATION AND FUSION PLATFORM



Bachelor of Science (Software Engineering)

Session (2022-2026)

Submitted By:

Muhammad Moazzam (2022F-MULBSSWE-042)

Sultan Kamran (2022F-MULBSSWE-025)

SUPERVISOR

Dr. Muhammad Saleem

Assistant Professor

Department of Software Engineering

SCHOOL OF SOFTWARE ENGINEERING

MINHAJ UNIVERSITY, LAHORE

VIDFUSION: AN AI-POWERED MULTI-VIDEO SUMMARIZATION AND FUSION PLATFORM

Project

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science (Software Engineering)

at the

Minhaj University, Lahore

by

Muhammad Moazzam Reg. No. 2022F-MULBSSWE-042

Sultan Kamran Reg. No. 2022F-MULBSSWE-025

Dr. Muhammad Saleem	Dr. Shahan Yamin Siddiqui

Assistant Professor School of Software Engineering	Head of Department School of Software Engineering

DECLARATION

It is declared that this is an original piece of our own work, except where otherwise acknowledged in text and references. This work has not been submitted in any form for another degree or diploma at any university or other institution for tertiary education and shall not be submitted by us in future for obtaining any degree from this or any other University or Institution.

Muhammad Moazzam Signature: _____

Reg. No. 2022F-MULBSSWE-042

Sultan Kamran Signature: _____

Reg. No. 2022F-MULBSSWE-025



DEDICATION

To

Our Parents

For their endless prayers, patience and sacrifices, for the quiet certainty that we could do this even on the days we doubted it ourselves, and for everything they gave up so that we would never have to choose between an education and anything else.

Our Teachers

For the guidance, patience and knowledge that shaped the way we think about engineering, and for asking the questions in class that we only understood the value of once we were debugging this project late into the night.

Our Friends and Classmates

For testing early, broken versions of this platform without complaint, and for the honest feedback that made the finished system better than either of us would have built working alone.

CERTIFICATE OF APPROVAL

It is certified that the project titled "VIDFUSION: AN AI-POWERED MULTI-VIDEO SUMMARIZATION AND FUSION PLATFORM" carried out by Muhammad Moazzam, 2022F-MULBSSWE-042 and Sultan Kamran, 2022F-MULBSSWE-025, under the supervision of Dr. Muhammad Saleem, Minhaj University, Lahore, is fully adequate, in scope and quality, as a final year project for the degree of Bachelor of Science (Software Engineering). The undersigned recommend its acceptance.

Supervisor: _____

Dr. Muhammad Saleem, Assistant Professor, School of Software Engineering

Internal Examiner 1: _____

School of Software Engineering

Internal Examiner 2: _____

School of Software Engineering

FYP Coordinator: _____

Ms. Nayab Gull, Lecturer, School of Software Engineering

Head of Department: _____

Dr. Shahan Yamin Siddiqui, School of Software Engineering

Dean: _____

Prof. Dr. Abdul Aziz, Dean Faculty of CS & IT

ACKNOWLEDGEMENT

All praise is due to Allah Almighty for granting us the strength, patience and opportunity to complete this project. We would like to express our sincere gratitude to our supervisor, Dr. Muhammad Saleem, for the guidance, feedback and encouragement throughout the development of this project. His advice was instrumental whenever the project's technical direction needed to be reconsidered.

We are deeply grateful to our parents and families for their unconditional support, patience and prayers throughout our academic journey. Their encouragement gave us the confidence to take on a project of this scale and see it through to completion.

We would also like to thank our teachers at the Department of Software Engineering, Minhaj University Lahore, for the knowledge and skills they imparted over the course of our degree, much of which was applied directly in building this system. Finally, we thank our friends and classmates who tested early versions of the platform and offered honest feedback that helped improve it.

ABSTRACT

VidFusion is a full-stack web platform that turns one or more long YouTube videos into a short, faithful summary: a condensed highlight-reel video cut with the original audio, and a structured written summary with chapters, key takeaways, and notable quotes. A user searches for or pastes one to ten YouTube videos, picks a target duration from six duration profiles (2 to 30 minutes), and the system downloads and transcribes the source material, semantically segments it into complete-thought units, extracts the concepts the content is actually about, selects a budget-constrained subset of segments that maximises concept coverage and timeline coverage while penalising redundancy, abstractively summarises the result, and renders the selected segments into a single video with ffmpeg.

The system is built as a FastAPI backend with MongoDB for persistence and a React 18/Tailwind CSS frontend. Every model in the pipeline runs locally and is open-source rather than a paid third-party API: Whisper for transcription, Sentence-BERT (BAAI/bge-base-en-v1.5) for semantic embeddings, a distilled BART model (sshleifer/distilbart-cnn-12-6) for abstractive compression, and an optionally-enabled local Ollama model (llama3.2:3b) for concept extraction, with a deterministic TF-IDF keyphrase fallback used by default because the LLM path was measured to be too slow on CPU hardware for an interactive demo. Long-running merge jobs are tracked through a durable, MongoDB-backed job and task-queue system rather than an in-process thread: once the core pipeline publishes an immutable "evidence manifest" describing which segments were selected, two independent lease-based background workers — one for the highlight video, one for the written summary — claim and process that manifest, so a crashed worker or a server restart recovers the job exactly once instead of losing it or silently duplicating it.

The project also includes a source-rights and publication-gating layer that records, per source video, the basis under which it may be reused (user-owned, Creative Commons, explicit permission, public domain, or unknown), and gates whether a job's output is eligible for publication accordingly; a semantic search and conversational question-answering feature backed by a FAISS vector index over every transcript the platform has processed; and a shadow-mode acquisition-observability system that records, as an append-only, hash-chained event log, every download/metadata/caption read performed against a source video, without yet enforcing anything, as a foundation for future compliance auditing.

The selection pipeline was evaluated against a hand-authored gold standard of 103 important concepts across seven real YouTube videos spanning five genres (tutorials, a podcast, an interview, and technical lectures), using concept recall, concept precision, redundancy, and temporal-coverage metrics defined specifically for this problem rather than borrowed wholesale from text-summarisation literature such as ROUGE or BLEU, since the system's central claim is about which parts of the source were chosen, not only how the chosen text reads. A component-ablation study on held-out videos the scoring weights were never tuned against confirmed that the concept-extraction and coverage terms both contribute measurably to selection quality over a relevance-only baseline. The backend carries 548 automated tests across 57 test files covering the semantic segmenter, concept extractor, coverage selector, summarisation policy, fusion engine, the durable task queue and worker recovery logic, the rights and acquisition-observability services, and API-level security and language-capability gating. This document describes the requirements, design, implementation, evaluation, and testing of the system, and reflects honestly on what was measured, what had to be disabled by default for hardware reasons, and what remains open for future work.

LIST OF FIGURES

Figure 1: Use Case Diagram - VidFusion

Figure 2: System Architecture Diagram

Figure 2b: System Architecture Diagram (reference rendering)

Figure 3: Entity Relationship Diagram (ERD)

Figure 4: Data Flow Diagram - Level 0

Figure 5: Data Flow Diagram - Level 1

Figure 6: Class Diagram

Figure 7: Activity Diagram - Create Merge Job

Figure 8: Sequence Diagram - Create Merge Job

Figure 9: Collaboration Diagram - Create Merge Job

Figure 10: State Transition Diagram - Merge Job Lifecycle

Figure 11: Component Diagram

Figure 12: Deployment Diagram

Figure 13: Work Breakdown Structure

Figure 14: Gantt Chart

LIST OF TABLES

Table 1: FR-01 Register Account

Table 2: FR-02 Login

Table 3: FR-03 Password Recovery

Table 4: FR-04 Create Merge Job

Table 5: FR-05 Preflight Check Sources

Table 6: FR-06 Monitor Job Progress

Table 7: FR-07 Retrieve Job Result

Table 8: FR-08 Retry a Failed Output Component

Table 9: FR-09 Search Transcripts

Table 10: FR-10 Chat with Video Content

Table 11: FR-11 Record Source Rights

Table 12: FR-12 View Job Rights Manifest

Table 13: FR-13 View History

Table 14: FR-14 Manage Users (Admin)

Table 15: UC-01 Register Account

Table 16: UC-02 Login

Table 17: UC-03 Create Merge Job

Table 18: UC-04 Monitor Job Progress

Table 19: UC-05 Retrieve Job Result

Table 20: UC-06 Retry a Failed Output Component

Table 21: UC-07 Search Transcripts

Table 22: UC-08 Chat with Video Content

Table 23: UC-09 Record Source Rights

Table 24: UC-10 View Job Rights Manifest

Table 25: UC-11 View History

Table 26: UC-12 Manage Users (Admin)

Table 27: TC-01 Successful login with correct email and password

Table 28: TC-02 Failed login with wrong password

Table 29: TC-03 Create a merge job with a feasible duration

Table 30: TC-04 Reject a merge job with an infeasible duration

Table 31: TC-05 Retry only the failed text-output component

Table 32: TC-06 Worker crash recovery via lease expiry

Table 33: TC-07 Duration-accuracy boundary at the Express profile's minimum

Table 34: TC-08 Record a rights attestation and confirm it affects the publication gate

Table 35: TC-09 Non-owner cannot read another user's job rights manifest

TABLE OF CONTENTS

Declaration.....4

Dedication.....6

Certificate of Approval.....7

Acknowledgement.....8

Abstract.....9

List of Figures.....11

List of Tables.....12

CHAPTER 1: INTRODUCTION TO THE PROBLEM.....17

CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION.....24

CHAPTER 3: USE CASE ANALYSIS.....37

Use Case Diagram Description.....38

CHAPTER 4: DESIGN.....48

Architecture Diagram.....49

Entity Relationship Diagram (ERD).....51

DFD Level 0 (Context Diagram).....52

DFD Level 1.....53

Class Diagram.....54

Activity Diagram.....55

Sequence Diagram.....58

Collaboration Diagram.....58

State Transition Diagram.....59

Component Diagram..... 60

Deployment Diagram..... 61

CHAPTER 5: TESTING..... 63

CHAPTER 6: TOOLS AND TECHNIQUES..... 76

Languages..... 77

Frameworks and Libraries..... 77

Database..... 77

Media Acquisition and Processing..... 78

AI / ML Models..... 78

Authentication and Security..... 80

DevOps..... 80

Testing..... 80

Version Control..... 80

CHAPTER 7: SUMMARY AND CONCLUSION..... 81

CHAPTER 8: USER MANUAL..... 84

CHAPTER 9: LESSONS LEARNT AND FUTURE WORK..... 112

REFERENCES..... 117

APPENDIX A: ENVIRONMENT SETUP AND INSTALLATION GUIDE..... 120

A.1 Overview..... 121

A.2 Prerequisites..... 121

A.3 Option A — Local development (Python + Node directly)..... 121

4. npm install..... 121

5. python prewarm_demo.py..... 122

6. docker compose up -d mongo.....122

7. PYTHONPATH=. pytest tests/ backend/tests/ -q.....122

A.4 Option B — Docker Compose (whole stack, production topology).....122

docker compose up -d --build.....122

A.5 Common operational commands.....123

A.6 Known setup caveats.....123

A.7 Key backend API routes referenced in this document.....123

CHAPTER 1:
INTRODUCTION TO THE PROBLEM

1.1 Introduction

Long-form video — podcasts, lectures, interviews, hour-long tutorials — carries a lot of a viewer's time cost against a comparatively small amount of information density. A one-hour conversation might contain fifteen minutes of material a given viewer actually needs, surrounded by introductions, tangents, and repeated explanations of the same point in different words. Two broad approaches exist to deal with this today: ask someone (or something) to write a text summary, or ask a platform's own auto-chapter feature to mark rough topic boundaries. Neither actually produces a shorter video a person can watch, and neither tells you why a given sentence was chosen over another.

VidFusion treats "produce a shorter version of this video" as the primary product, with a written summary as a complementary output rather than the only one. A user supplies one or more YouTube videos and a target duration, and the system returns both a condensed video — built from real cut segments of the original footage with the original audio intact, not a synthetic voiceover over stock visuals — and a structured text summary with chapters, takeaways, and quotes, so the same underlying selection decision can be consumed either way depending on what the user actually wants.

1.2 Purpose

The purpose of this project is to build and rigorously evaluate a video-summarisation system whose core claim — "these are the parts of the source that matter" — is measurable rather than asserted. Most existing summarisation tools report, at best, that their output text reads fluently; they rarely demonstrate that the underlying selection decision covered the material a human would judge important, or that it did not quietly over-represent the first few minutes of the source, which is the part every similarity-based selector is naturally biased toward (an early segment that introduces a topic looks similar to almost any summary of that topic). This project set out to build the selection pipeline on principles that make that bias visible and correctable, and to prove it with a held-out evaluation rather than only a qualitative "it looks reasonable" judgement.

1.3 Objective

The concrete objectives of the project were:

- To build an end-to-end pipeline that converts one or more YouTube videos into both a condensed highlight video and a structured written summary, within a user-selected target duration.
- To replace naive "sort candidates by similarity to one generated summary and take the top N" selection with a principled, budget-constrained optimisation that explicitly rewards concept coverage, timeline coverage, and segment coherence, and explicitly penalises redundant picks.
- To extract what a video is actually about — a ranked list of concepts with supporting evidence and timestamps — rather than relying on a single abstractive summary as the only signal for what is "important."
- To make every numeric claim about a finished job checkable: duration accuracy, timeline coverage, redundancy, and concept recall/precision computed once per job and returned by the API rather than asserted in documentation.
- To evaluate the system against a gold standard that the system itself had no part in producing, including a genuinely held-out partition of videos never used while tuning the selection weights, so the reported numbers are not simply re-discovering the training signal.
- To make the background work (downloading, transcribing, selecting, rendering) durable: a server crash or restart partway through a job should recover and finish it, not lose it or silently double-process it.
- To support multiple source videos in one job — fusing related content, detecting near-duplicate material across sources, and producing one coherent output rather than requiring one job per video.
- To build a source-rights and publication-gating layer so that what the platform can produce is explicitly tied to a recorded basis for reusing the source material, rather than treating every public YouTube URL as unconditionally safe to re-publish from.

1.4 Existing Solution

Several categories of existing tools address parts of this problem:

Platform-native auto-chapters and auto-generated descriptions (as offered by YouTube itself) mark rough topic boundaries using view-behaviour and metadata signals, but they do not produce a shorter video, do not fuse multiple sources, and are not something a third party can evaluate or tune — the chaptering logic is entirely opaque and platform-controlled.

Browser-extension and SaaS "AI YouTube summariser" tools (e.g. Eightify, Glasp, NoteGPT, and similar products) take a single video's transcript, send it to a hosted large language model, and return bullet-point text notes. This is useful for a quick read but does not produce a shortened video, is limited to one source at a time, and — because the underlying selection is "ask an LLM to summarise this transcript" — offers no mechanism to check whether the output actually covers the source evenly rather than over-weighting whatever appeared early or was phrased memorably.

Generic "paste a transcript into ChatGPT" workflows work for short material but require the user to do their own chunking for anything beyond a model's context window, require a separate tool entirely to produce a shortened video rather than just text, and provide no persistence, job tracking, or per-user history.

Manual video editing remains the only approach that reliably produces a genuinely well-chosen highlight reel, but it is slow, requires editing skill, and does not scale to a library of videos a person wants summarised regularly.

A recurring limitation across the automated tools above is that none of them separate "what is this video about" from "write me a summary of it." Scoring candidate material by similarity to a single generated summary — which is what most of these tools do under the hood, whether or not they expose that detail — makes the summary itself a bottleneck: a topic the summary only mentions once can never win a selection slot no matter how much of the source actually discusses it, and a summary-writing model's own tendency to foreground the opening of a source becomes the selection system's tendency too.

1.5 Proposed Solution

VidFusion differs from the tools above in ways grounded in what is actually implemented in the codebase, not in aspiration:

First, concept extraction is a first-class, separate stage from summarisation, not a byproduct of it. The system chunks a long transcript, asks an LLM (or, by default on this project's hardware, a deterministic TF-IDF keyphrase extractor) what each chunk is about, merges near-duplicate concepts across chunks using sentence-embedding similarity, and ranks the result globally by importance, how many chunks support it, and how spread out in time it is. Candidate segments then compete on how well they explain these concepts, not on how similar they are to one fixed summary sentence.

Second, selection is a budgeted, submodular optimisation with an explicit, inspectable scoring formula — concept coverage, semantic relevance discounted by redundancy, temporal coverage of under-represented parts of the timeline, and segment coherence, each with a configurable weight — rather than a sorted-list top-N cut. Coverage of the timeline is rewarded as a concave function (the first second spent in an unrepresented region of the video is worth far more than the sixtieth second in a region already well covered), which is what directly corrects the beginning-heavy selection bias measured during development (documented in Chapter 9) without resorting to a blunt "one clip per decile" quota that would force in genuinely low-value material just to tick a box.

Third, every claim the pipeline makes about a finished job is a number computed from that job's own output, not an assumption: duration accuracy, how many deciles of the source timeline were touched, redundancy measured relative to the video's own baseline similarity (not a fixed, encoder-dependent threshold), and concept recall/precision against the gold standard used in evaluation. This is what let the beginning-heavy selection bias and a silent BART-bypass bug (Chapter 9) be found and fixed from measurement rather than suspicion.

Fourth, reliability and provenance are treated as first-class engineering concerns rather than an afterthought bolted onto a working prototype. Background work is tracked through a durable, lease-based job and task-queue system so a crash mid-render is recoverable rather than catastrophic; and a source-rights model records, per video a user submits, an explicit basis for reusing it, gating what the platform will let a user publish accordingly. Neither of these was handed down as a specification — both emerged from building the system honestly and then asking what would actually go wrong with a multi-user, always-on deployment of it.

1.6 Project Plan

The project was carried out over the seventh and eighth semesters, from January 2026 through September 2026, following the phased plan summarised below. Work proceeded roughly in the order requirement analysis, system design, backend implementation, frontend implementation, testing and evaluation, deployment, and documentation, though later phases overlapped with earlier ones as features were implemented and measured incrementally rather than in a strict waterfall sequence.

1.6.1 Work Breakdown Structure

The work was broken down into six top-level packages, each grouping the related functional requirements described in Chapter 2: account and authentication, video acquisition and transcription, the semantic selection and summarisation pipeline, the rights/publication-gating and acquisition-observability layer, testing and evaluation, and deployment and documentation.

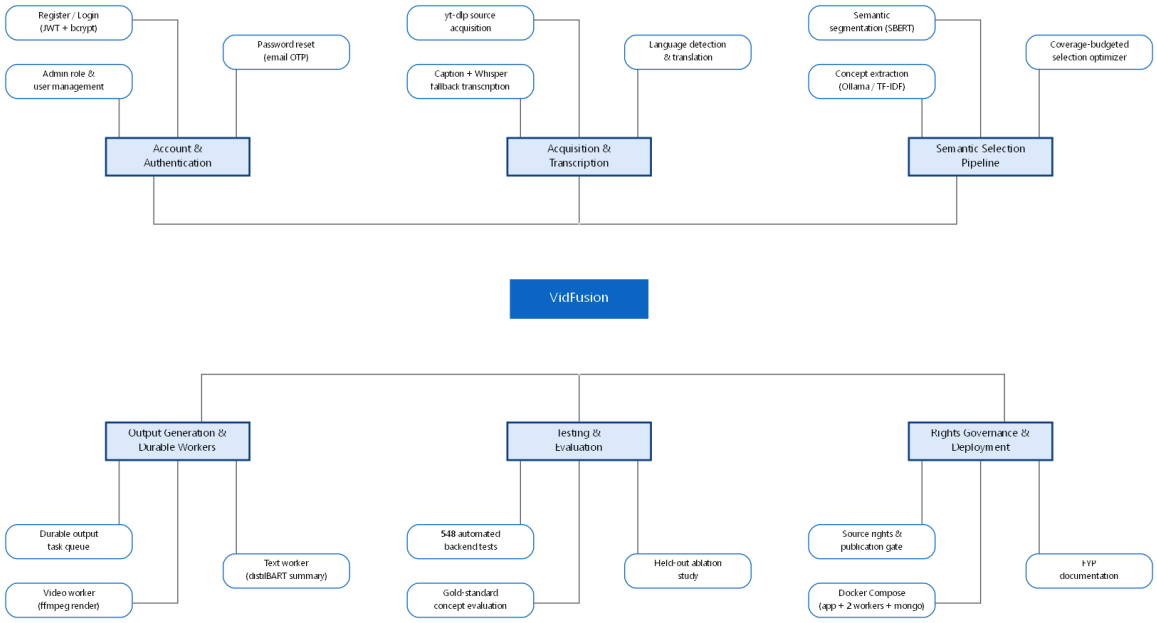


Figure 13: Work Breakdown Structure

1.6.2 Gantt Chart

The timeline below maps each work package onto the months in which it was carried out. Later phases begin before earlier ones fully close because the backend pipeline, the frontend, and testing alongside it proceeded feature-by-feature rather than as strictly separate stages.

Task	Jan 2026	Feb 2026	Mar 2026	Apr 2026	May 2026	Jun 2026	Jul 2026	Aug 2026	Sep 2026
FYP proposal & topic approval									
Requirement analysis & SRS									
System design (ERD, class, architecture diagrams)									
Backend implementation (auth, pipeline, durable workers)									
Semantic selection pipeline (segmentation, concepts, selector)									
Testing & gold-standard evaluation harness									
Rights governance & Docker Compose deployment									
Documentation & final submission									

Figure 14: Gantt Chart

CHAPTER 2:
SOFTWARE REQUIREMENT SPECIFICATION

2.1 Introduction

2.1.1 Purpose

This chapter specifies the functional and non-functional requirements of VidFusion, following the standard functional/non-functional requirement structure described in the software engineering literature [1]. It is written from the actual implemented system (backend routers, services, models, and workers) rather than as an aspirational document, so every requirement below corresponds to code that exists in the repository at the time of writing.

2.1.2 Scope

The system in scope covers: user registration, authentication, and an administrative role; multi-video merge-job creation with a user-selected duration profile; a semantic selection pipeline producing both a condensed highlight video and a structured written summary; durable background processing of that pipeline through a lease-based task queue with independent video and text output workers; live job-progress reporting over Server-Sent Events; semantic search and conversational question-answering over every transcript the platform has processed, backed by a FAISS vector index; a source-rights and publication-gating layer; and a per-user history and admin panel. Out of scope: payment/billing, native mobile applications, and speaker diarisation (see Chapter 9 for future work).

2.1.3 Definitions, Acronyms, and Abbreviations

- API: Application Programming Interface
- JWT: JSON Web Token, used here for stateless session authentication
- SSE: Server-Sent Events, a one-way HTTP streaming mechanism used for live job-progress updates
- FAISS: Facebook AI Similarity Search, a library for efficient nearest-neighbour search over dense vector embeddings
- SBERT: Sentence-BERT, a family of transformer models that produce sentence-level embeddings suitable for cosine-similarity comparison
- LLM: Large Language Model (here, a locally-hosted Ollama model used for optional concept extraction and chat answer synthesis)
- Concept: a named idea the source material is "about", extracted with an importance score, supporting evidence, and the timestamps it occurs at
- Segment: a semantically-coherent, timestamp-bounded unit of a transcript — the unit the selection optimizer picks and ffmpeg cuts

- Evidence Manifest: an immutable record, published once per job, of exactly which segments and sources were selected, handed off to the independent output workers so they never re-derive selection logic
- Rights Basis: the recorded justification (owned, Creative Commons, explicit permission, public domain, or unknown) for a user's platform-level right to reuse a given source video
- Job/Task Queue: a persisted unit of background work (highlight-video rendering, text-summary generation) processed by a worker outside the HTTP request cycle

2.2 Overall Description

2.2.1 Product Perspective

VidFusion is a standalone, self-contained web application composed of a React single-page frontend and a FastAPI REST backend, backed by MongoDB.

System interfaces: The backend exposes a versioned REST API under /api/v1 (auth, merge, transcript, search, chat, youtube, rights). Live merge-job progress is delivered to the frontend over Server-Sent Events at /api/v1/merge/{job_id}/stream rather than polling for the primary progress path, though the job status endpoint itself (GET /api/v1/merge/{job_id}) can also be polled.

User interfaces: A React 18 frontend styled with Tailwind CSS, with dedicated pages for search by title and by keyword, a transcript viewer, a merge-preview/selection page, a summarised player for jumping to highlighted moments, a merged-podcast/highlight player, a user account panel (profile, security, history), and an admin panel (user list, platform statistics, account management).

Hardware interfaces: No GPU is required; every model in the pipeline (Whisper, the SBERT embedding model, distilBART, and the optional Ollama model) is sized to run acceptably on CPU, which was itself a deliberate constraint documented in Chapter 6. A server capable of running Python 3.12, Node.js, and ffmpeg is sufficient.

Software interfaces: MongoDB (via Motor, the async driver, and Pymongo) for all persistent data — users, history, merge jobs, transcripts, durable output tasks, source-rights records, and acquisition-observability events; ffmpeg for video trimming, concatenation, and transition rendering; yt-dlp for source video/audio acquisition; the youtube-transcript-api and, as a fallback, OpenAI Whisper for transcript acquisition; HuggingFace transformers and

sentence-transformers for the summarisation and embedding models; FAISS for the semantic search vector index; and an optionally-present local Ollama server for LLM-backed concept extraction and chat answer synthesis, with every one of these falling back to a deterministic non-LLM path if Ollama is not reachable.

Communications interfaces: HTTPS REST over JSON for all API calls; JWT bearer tokens in the Authorization header for authenticated requests; a separately-scoped, short-lived media token for protected video/audio/subtitle download endpoints; Server-Sent Events for live merge-job progress.

2.2.2 Product Functions

The following functional requirements are drawn directly from the implemented routers and services.

FR-01

Name	Register Account
Description	Create a new user account with an email, password, and optional display name.
Input	email, password, name (optional)
Output	user profile and role
Basic Work Flow	Client submits the signup form -> auth router's /signup endpoint -> AuthService hashes the password with bcrypt, auto-generates a username from the email if one was not supplied, and creates the user document with role "user" (public registration can never create an admin account).

FR-02

Name	Login
Description	Authenticate an existing user with a username or email and password, and issue a session.
Input	login (username or email), password
Output	access token and user profile
Basic Work Flow	Client submits credentials to /login -> AuthService looks up the account by username or email and verifies the password against the stored bcrypt hash (or,

	for an account created before bcrypt was adopted, against a legacy SHA-256 hash, upgrading it to bcrypt transparently on a successful login) -> a JWT access token carrying the user's id and role is issued.

FR-03

Name	Password Recovery
Description	Reset a forgotten password via an emailed one-time verification code.
Input	email (to request a code); email and code (to verify); reset_token and new password (to reset)
Output	a sent confirmation; a verified reset token; a success confirmation
Basic Work Flow	/send-verification-code emails a time-limited code -> /verify-code checks it and issues a reset token -> /reset-password accepts the token and the new password, re-hashing it with bcrypt.

FR-04

Name	Create Merge Job
Description	Submit one to ten YouTube videos, a target summary duration, and output preferences (highlight video, audio, or both), and have the system produce a condensed summary.
Input	video_ids (1-10), target_duration_minutes (5-30, or down to 2 via the Express profile), generate_video, generate_audio, style, and an optional scope/query for a keyword-focused rather than overall summary
Output	a job_id and initial status, or a 422 error if the requested duration is not feasible against the combined source length
Basic Work Flow	POST /api/v1/merge validates video count and deduplicates ids -> resolves each source's duration and rejects the request up front if any source's duration cannot be verified, or if the requested duration is not strictly shorter than the combined source length -> creates a MergeJob document with status "pending" and hands it to the background pipeline.
Alternate Flow	A1. If fewer than one or more than ten valid video ids are supplied after deduplication, the request is rejected with HTTP 400 before a job is created.

	A2. If the requested duration is not achievable, the response carries a structured SOURCE_DURATION_UNAVAILABLE or duration-infeasibility error rather than a generic failure.

FR-05

Name	Preflight Check Sources
Description	Check, before committing to a full job, whether a set of candidate videos are reachable and have a usable transcript language.
Input	video_ids, an optional per-source timeout
Output	a per-video readiness/eligibility report
Basic Work Flow	POST or GET /api/v1/merge/preflight resolves each candidate's transcript availability and detected language without starting a full merge job, so the frontend can warn the user before they spend a job attempt on an unusable source.

FR-06

Name	Monitor Job Progress
Description	Track a merge job's status, stage, and progress percentage in real time.
Input	job_id
Output	current status (one of twelve JobStatus values, e.g. transcribing, analyzing, fusing, summarizing, enriching, generating_video, video_ready, completed, partial_success, error), progress_percent, and a human-readable stage_message
Basic Work Flow	Frontend opens GET /api/v1/merge/{job_id}/stream (Server-Sent Events) or polls GET /api/v1/merge/{job_id} -> the backend pipeline updates the job document's status/progress/stage fields as it moves through transcription, semantic analysis, fusion, summarisation, enrichment, and rendering.

FR-07

Name	Retrieve Job Result

Description	Fetch the completed output of a merge job — the written summary, the selected concepts, selection-quality metrics, and links to the rendered video/audio/subtitles.
Input	job_id
Output	summary_text, rich_output (chapters, key takeaways, quotes, TL;DR), concepts[], selection_metrics, and download links
Basic Work Flow	GET /api/v1/merge/{job_id}/result returns the full job document's output fields once the relevant component (video and/or text) has reached a ready state; GET /{job_id}/video, /audio, and /subtitles stream the corresponding rendered file; GET /{job_id}/export produces a text or PDF export of the summary.

FR-08

Name	Retry a Failed Output Component
Description	Re-attempt only the video or only the text half of a job that partially failed, without re-running the whole pipeline.
Input	job_id, output_type ("video" or "text")
Output	the component's state reset to queued and re-dispatched to its worker
Basic Work Flow	POST /api/v1/merge/{job_id}/retry-{output_type} re-queues a durable output task for the named component against the job's existing, already-published evidence manifest, so a text-generation failure does not force the (potentially much slower) video render to be repeated, and vice versa.

FR-09

Name	Search Transcripts
Description	Run a natural-language semantic search across every video the platform has processed and indexed.
Input	a free-text query, optional top_k
Output	ranked transcript segments with video id/title, score, and timestamp

Basic Work Flow	GET /api/v1/search embeds the query with the same sentence-embedding model used during indexing and performs a FAISS nearest-neighbour search over the persisted vector index, returning each match's source video and timestamp so the result can be used as a deep link.

FR-10

Name	Chat with Video Content
Description	Ask a natural-language question and receive an answer grounded in the platform's indexed transcripts, with source attribution.
Input	question, optional video_ids to scope the search, top_k
Output	an answer and a list of the source segments it was grounded in
Basic Work Flow	POST /api/v1/chat retrieves the top-k relevant segments from FAISS (optionally scoped to specific videos) -> if Ollama is reachable, the retrieved context and question are sent to it for answer synthesis; otherwise the top retrieved segments are stitched together directly as a local, non-generative fallback answer -> the response always includes the literal source segments so a user can verify the answer against the original material.

FR-11

Name	Record Source Rights
Description	Record, per source video a user has submitted, the basis under which they are entitled to reuse it.
Input	source_id, claim_type (ownership, fair_use, or public_domain) for a user attestation; or grantor/evidence details for a recorded third-party permission
Output	a persisted SourceRightsRecord
Basic Work Flow	POST /api/v1/rights/attest or /permission records the user's declared basis -> the record is later consulted by evaluate_publication_gate when a job involving that source is evaluated for publication eligibility.

FR-12

Name	View Job Rights Manifest
Description	Produce a complete rights-and-attribution report for a completed job: which sources contributed which physical clip intervals, their recorded rights basis, and the resulting publication-gate status.
Input	job_id
Output	a combined manifest (source reuse report, rights records, publication gate status, composite attribution text)
Basic Work Flow	GET /api/v1/rights/job/{job_id}/manifest verifies the requester owns the job (or is an admin) without revealing whether a job exists to anyone else -> builds an objective reuse report from the job's actual selected segments and known source durations -> returns the job's frozen rights_snapshot (captured once, at render time, as a write-once historical record) rather than recomputing rights state live, so the manifest reflects what the rights situation actually was when the content was produced.

FR-13

Name	View History
Description	List the current user's past searches, transcript views, and merge jobs.
Input	username
Output	a list of history entries with type, timestamp, and type-specific fields (query, videoId, jobId, videoCount)
Basic Work Flow	GET /api/user-history/{username} returns the user's history entries newest-first; entries are written by the frontend as the corresponding action (a search, a transcript view, a completed merge) occurs.

FR-14

Name	Manage Users (Admin)
Description	List all users, view platform statistics, and activate/deactivate or edit a user

	account.
Input	(list/stats: none); (toggle/update: username and the fields to change)
Output	a user list with role but no password hashes; aggregate platform statistics; a success/failure result for a mutation
Basic Work Flow	An admin-only route group, each guarded by a require_role("admin") dependency that re-derives the caller's role from the token rather than trusting a client-supplied flag, exposes GET /users, GET /admin/stats, PATCH /admin/toggle-user/{username}, PUT /admin/user/{username}, and POST /admin/create-user; an admin cannot deactivate their own account through this route, to avoid an admin locking themselves out.

2.2.3 User Characteristics

The primary user is assumed to be a student, researcher, content consumer, or professional who wants to extract the substance of long-form video without watching it in full, comfortable with a typical web application but not expected to understand transcript alignment, embedding models, or submodular optimisation. A secondary administrative role exists for platform operation (user management, aggregate statistics) and is not exposed to ordinary users.

2.2.4 Constraints

- The requested summary duration must be strictly shorter than the combined duration of the selected source videos; this is enforced server-side at job-creation time, before a job document is even created.
- ffmpeg must be present and reachable on the host, since the highlight-reel renderer shells out to the real binary rather than a bundled library.
- The LLM-backed concept-extraction path (ENABLE_CONCEPT_LLM) is disabled by default because it was measured, on this project's CPU-only development hardware, to take roughly 180 seconds per transcript chunk — a 25-minute video would need on the order of 20 minutes just for this stage before timing out and falling back regardless. The deterministic TF-IDF fallback is always available and is what actually runs by default.
- The in-process rate limiter in the FastAPI middleware is per-process and in-memory, an accepted constraint for the current single-instance deployment target.

- A transcript whose detected language is not supported by the configured pipeline terminates the job during preflight with an explicit summary_unavailable_reason, rather than silently producing a degraded or garbled summary.

2.2.5 Assumptions and Dependencies

- A MongoDB instance is reachable and its URI is configured; MONGODB_URI defaults to a local instance for development.
- ffmpeg, yt-dlp, and a Python 3.12 environment with the project's machine-learning dependencies (torch, transformers, sentence-transformers, faiss-cpu, openai-whisper) are installed.
- Ollama, if the operator wants LLM-backed concept extraction or chat answer synthesis, is a genuinely optional dependency — the system is designed and tested to run with it entirely absent, falling back to TF-IDF keyphrases and raw FAISS-segment answers respectively.
- A HuggingFace access token (HF_TOKEN) is optional but recommended, since anonymous model downloads share a per-IP rate quota that an authenticated pull does not.

2.2.6 Apportioning of Requirements

FR-01 through FR-03 and FR-13/FR-14 are core account and platform-management requirements delivered in the current release. FR-04 through FR-12 constitute the video-summarisation, search, and rights-governance feature set, also fully delivered in the current release. Deferred to future work (Chapter 9): speaker diarisation as a segmentation signal, re-enabling visual (CLIP-based) scoring by default, and horizontally scaling the durable output workers across multiple machines.

	2.3 Specific Requirements

2.3.1 Functional Requirements

The functional requirements are as specified in section 2.2.2 (FR-01 through FR-14) and are elaborated further as use cases in Chapter 3.

2.3.2 Non-Functional Requirements

The non-functional requirements below are organised around the quality characteristics defined in ISO/IEC 25010 [2] (functional suitability, usability, reliability, performance efficiency, portability, and maintainability), grouped under the headings most relevant to what was actually built.

Usability: Duration selection is exposed as six named profiles (Express, Quick Highlights, Key Concepts, Balanced Summary, Detailed Summary, Comprehensive Summary at 2, 5, 10, 15, 20, and 30 minutes respectively) rather than a raw numeric field, so a user chooses an intent ("quick highlights") rather than guessing a number of minutes; long-running jobs surface live stage text over Server-Sent Events (e.g. "Composing segment 3 of 12"-style progress) rather than an opaque spinner.

Reliability: This is the area the design invests in most, because every stage depends on either an external video platform or a locally-hosted model that can be slow, absent, or fail outright. The durable output-task queue (output_task_queue.py) uses atomic, lease-based claiming — a worker claims a task with a renewable lease, and a startup recovery routine (recover_expired_output_tasks, recover_prepared_output_dispatches) re-queues any task whose lease expired without completing, so a crashed worker or a server restart mid-render is recovered exactly once rather than losing the job or risking two workers processing it simultaneously. Every selection-pipeline stage (semantic segmentation, concept extraction, redundancy scoring) has an explicit, documented fallback (Chapter 6) so the absence of Ollama, a failure of SBERT to load, or an untimestamped transcript each degrade the output's quality rather than failing the job outright. Bounded retry (max_retries = 3 on a job) prevents a permanently broken job from being retried forever.

Performance: Per-process rate limiting bounds request volume per client IP, with status-polling and progress-stream traffic explicitly exempted so a frontend polling a slow job cannot itself trip the limiter. A storage-hygiene routine runs at boot and hourly thereafter, enforcing size and age caps on the local highlight-video and source-video caches (10 GB / 14 days and 10 GB / 30 days respectively) so disk usage is self-bounding rather than growing without limit across many jobs. Embedding and summarisation models are pre-loaded once at process startup rather than on first request, eliminating cold-start latency on the first real job.

Design Constraints: All provider-specific and tunable configuration (model names, selection-scoring weights, timeouts, cache limits) lives in one validated Pydantic settings

module (backend/core/config.py) read from a single .env file, rather than scattered process.env-style reads, so a misconfiguration is visible in one place; the module additionally refuses to start in a production environment with a weak or default signing key, rather than booting insecurely.

Portability: The backend, a dedicated text-output worker, and a dedicated video-output worker are each built from the same Docker image and declared as separate services in docker-compose.yml, alongside a MongoDB container, with named volumes for the local media cache and the FAISS index so a container rebuild does not discard in-progress work or force a full search-index rebuild.

Maintainability: Business logic is organised into single-purpose service modules (semantic_segmenter, concept_extractor, coverage_selector, summarization_policy, fusion_engine, rights_service, acquisition_service) with pure, side-effect-free core functions specifically so they can be unit tested without a running MongoDB instance, a GPU, or network access — which is also why the project's 548-test suite runs deterministically in CI-style conditions.

License Agreement: The project uses standard open-source dependencies (FastAPI, React, Whisper, sentence-transformers, transformers, FAISS, ffmpeg) under their respective permissive or research-friendly open licenses. No proprietary code was incorporated. The platform acquires source video via YouTube under the terms applicable to end-user access of that platform, and the rights/publication-gating feature (FR-11/FR-12) exists specifically to make the basis for any further reuse an explicit, recorded decision rather than an unexamined assumption.

CHAPTER 3:
USE CASE ANALYSIS

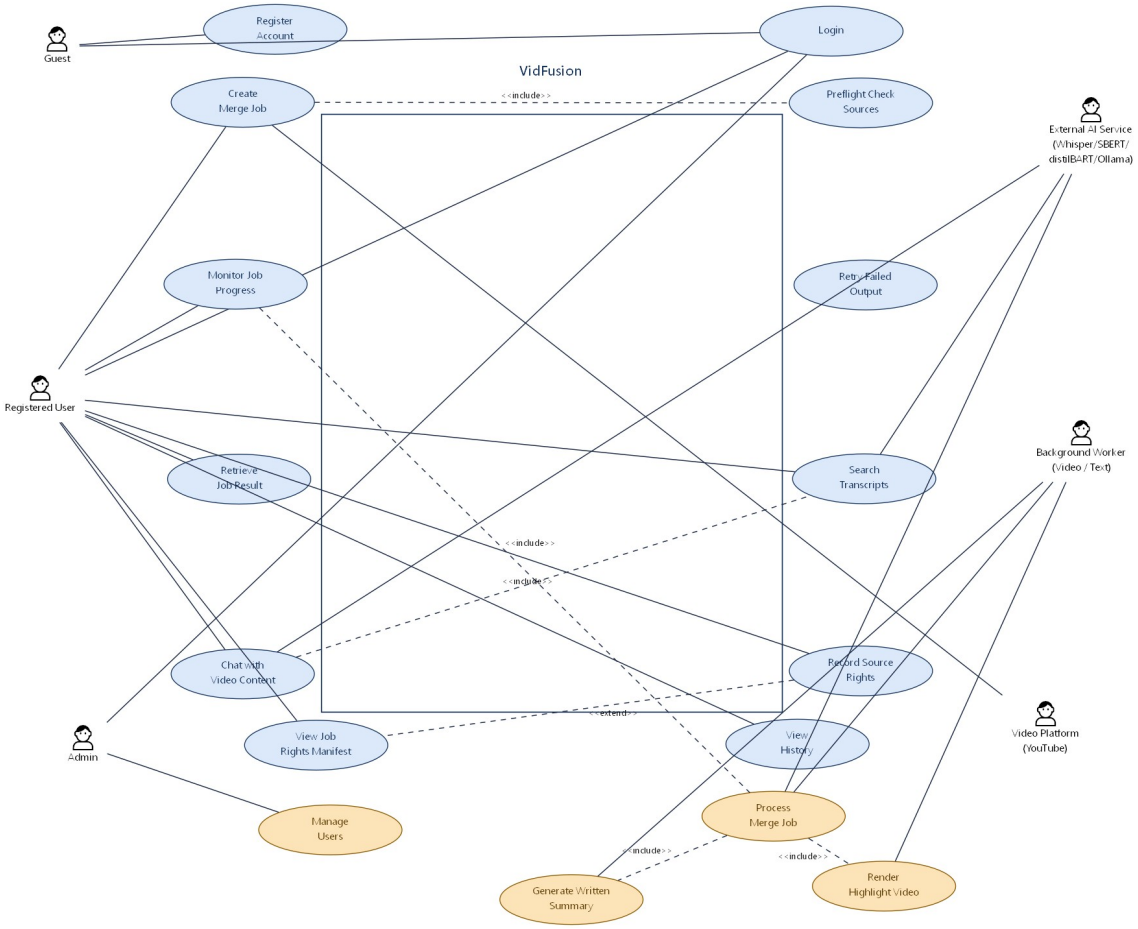


Figure 1: Use Case Diagram - VidFusion

Use Case Diagram Description

The aggregated use case diagram for VidFusion shows five actors: Guest, Registered User, Admin, Background Worker, and External AI Service (the locally-hosted Whisper/SBERT/distilBART/Ollama models, drawn as a single secondary actor since at this level which specific model answers a given call is an implementation detail). The Guest actor can only reach Register Account and Login — everything else requires an authenticated session. The Registered User actor reaches the bulk of the use cases: Login, Create Merge Job, Monitor Job Progress, Retrieve Job Result, Search Transcripts, Chat with Video Content, Record Source Rights, and View History. Several of these are drawn with an include relationship into the concrete step that does the work: Create Merge Job includes Preflight Check Sources, since a well-behaved client checks source eligibility before committing to a full job; Monitor Job Progress includes Process Merge Job, the use case owned by the Background Worker actor that actually performs transcription, selection, summarisation, and rendering asynchronously rather than inside the HTTP request; and Chat with Video Content

includes Search Transcripts, since the chat endpoint's first step is exactly the same FAISS retrieval the dedicated search page uses. Process Merge Job in turn includes two further use cases owned by the two independent output workers: Render Highlight Video and Generate Written Summary, reflecting that once the core pipeline publishes its evidence manifest, these two outputs are produced by separate durable workers rather than as one monolithic step. The External AI Service actor is reached by every use case that calls out to a transcription, embedding, summarisation, or language model (Process Merge Job, Chat with Video Content). The Admin actor reaches a separate lane of use cases — Manage Users and View Platform Statistics — that a Registered User cannot reach. Record Source Rights and View Job Rights Manifest are drawn as their own lane reachable from a completed job, since they can be exercised independently of actively creating a new merge job.

USE CASE 1: Register Account

ID	UC-01
Name	Register Account
Description	A new user creates an account with an email, password, and optional display name.
Primary Actor	Guest
Secondary Actor	None
Pre-Condition	No existing account holds the given email.
Post-Condition	A user document exists with role "user" and a bcrypt-hashed password.
Basic Flow	1. Guest submits email, password, and optionally a display name on the Sign Up page. 2. System checks for an existing account with that email. 3. System auto-generates a username from the email if one was not supplied, hashes the password with bcrypt, and creates the account. 4. System returns the created user's profile.
Alternate Flow	A1. If the email is already registered, the request is rejected rather than silently overwriting the existing account.

	A2. Public registration can never set role to "admin"; the request schema deliberately has no role field, so an administrative account can only be created through the protected admin-only create-user route.

USE CASE 2: Login

ID	UC-02
Name	Login
Description	A registered user authenticates with a username or email and password and receives a session token.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	The user holds an account.
Post-Condition	The user holds a valid JWT access token carrying their id and role.
Basic Flow	1. User submits a login identifier (username or email) and password. 2. System looks up the account and compares the password against the stored bcrypt hash. 3. System issues a JWT access token. 4. Client attaches the token as a Bearer header on subsequent requests.
Alternate Flow	A1. If the stored password hash is in the legacy SHA-256 format (predating the project's adoption of bcrypt), the system verifies against that format instead and transparently re-hashes the password with bcrypt on this successful login, so the account is upgraded without requiring the user to reset anything. A2. An incorrect password or unknown login both return an equivalent authentication failure, avoiding an obvious signal that would let an attacker distinguish a wrong password from a nonexistent account.

USE CASE 3: Create Merge Job

ID	UC-03

Name	Create Merge Job
Description	A user submits one to ten YouTube videos and a target duration, and the system begins producing a condensed video and/or written summary.
Primary Actor	Registered User
Secondary Actor	Background Worker; External AI Service
Pre-Condition	User is authenticated.
Post-Condition	A MergeJob document exists in a non-terminal status and background processing has begun, or the request was rejected with a specific, structured error before any job was created.
Basic Flow	1. User selects one or more videos (via search or direct link) and a duration profile in the Merge Preview page. 2. System deduplicates and validates the video ids, resolves each source's actual duration, and rejects the request if the target duration is not strictly less than the combined source duration. 3. System creates the MergeJob document and dispatches it to background processing. 4. System returns a job_id and an estimated completion time.
Alternate Flow	A1. If any source's duration cannot be resolved (e.g. the video is private, deleted, or region-blocked), the system returns a structured SOURCE_DURATION_UNAVAILABLE error naming the specific unresolved video ids, rather than a generic failure. A2. If the requested duration is not feasible given the combined source length, the system returns a duration-infeasibility error describing what durations would actually be achievable.

USE CASE 4: Monitor Job Progress

ID	UC-04
Name	Monitor Job Progress
Description	A user watches the live status of a submitted merge job as it moves through

	transcription, selection, summarisation, and rendering.
Primary Actor	Registered User
Secondary Actor	Background Worker
Pre-Condition	A job has been created.
Post-Condition	The user observes the job's current status, stage, and progress percentage; no state is changed by observing.
Basic Flow	1. User opens the job-progress view, which connects to the job's Server-Sent Events stream. 2. System pushes status/progress/stage updates as the pipeline advances (transcribing, analyzing, fusing, summarizing, enriching, generating_video). 3. Once the highlight video specifically is ready, the job may enter a non-terminal "video_ready" checkpoint, letting the user view the video immediately while the written summary continues generating in the background. 4. The job reaches a terminal status: completed, partial_success (one output component failed but the other succeeded), or error.
Alternate Flow	A1. If the SSE connection drops, the client falls back to polling the job's status endpoint directly, which reflects the same underlying state.

USE CASE 5: Retrieve Job Result

ID	UC-05
Name	Retrieve Job Result
Description	A user views or downloads the completed output of a merge job.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	The job has produced at least one ready output component.
Post-Condition	No state is changed; content is returned or streamed to the user.
Basic Flow	1. User opens the result view for a completed (or partially completed) job.

	2. System returns the written summary (chapters, key takeaways, quotes, TL;DR), the ranked concepts with their supporting evidence and timestamps, and the selection-quality metrics computed for this specific job. 3. User downloads the rendered highlight video, an audio-only export, SRT subtitles, or a text/PDF export of the written summary.
Alternate Flow	A1. If only one output component (video or text) is ready, the result view shows that component and indicates the other's current status rather than withholding the whole result.

USE CASE 6: Retry a Failed Output Component

ID	UC-06
Name	Retry a Failed Output Component
Description	A user re-attempts only the video or only the text half of a job whose output partially failed.
Primary Actor	Registered User
Secondary Actor	Background Worker
Pre-Condition	The job's video_component or text_component status is "failed".
Post-Condition	A fresh durable task for that component is queued against the job's existing evidence manifest.
Basic Flow	1. User sees a failed component on the result view and selects Retry. 2. System re-queues a durable output task for the named component, reusing the already-selected segments and concepts rather than re-running segmentation and selection from scratch. 3. The corresponding worker claims and processes the task as in UC-04.

USE CASE 7: Search Transcripts

ID	UC-07
Name	Search Transcripts

Description	A user runs a natural-language search across every video the platform has previously processed.
Primary Actor	Registered User
Secondary Actor	External AI Service (embedding model, FAISS index)
Pre-Condition	User is authenticated; at least one video has been indexed by a prior merge job.
Post-Condition	No state change; ranked results are returned.
Basic Flow	1. User enters a free-text query. 2. System embeds the query with the same sentence-embedding model used during indexing and searches the FAISS vector index. 3. System returns ranked transcript segments, each attributed to its source video and timestamp.
Alternate Flow	A1. If the FAISS index is empty (nothing indexed yet), the system returns an explicit "still being indexed" message rather than an empty result that could be mistaken for "no matches".

USE CASE 8: Chat with Video Content

ID	UC-08
Name	Chat with Video Content
Description	A user asks a natural-language question and receives an answer grounded in the platform's indexed transcripts.
Primary Actor	Registered User
Secondary Actor	External AI Service (FAISS, optionally Ollama)
Pre-Condition	User is authenticated; relevant content has been indexed.
Post-Condition	No state change; an answer with source attribution is returned.
Basic Flow	1. User asks a question, optionally scoped to specific videos. 2. System retrieves the top-k relevant segments from the FAISS index (UC-07's retrieval step, reused).

	3. If a local Ollama model is reachable, system sends the retrieved context and the question to it for answer synthesis; otherwise the top retrieved segments are stitched together directly as the answer. 4. System returns the answer together with the literal source segments it was grounded in, so the user can verify it against the original material.
Alternate Flow	A1. If no relevant segments are retrieved at all, the system states plainly that it could not find relevant information, rather than letting a language model answer ungrounded.

USE CASE 9: Record Source Rights

ID	UC-09
Name	Record Source Rights
Description	A user records the basis under which they are entitled to reuse a given source video.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User is authenticated.
Post-Condition	A SourceRightsRecord exists for the (user, source) pair.
Basic Flow	1. User declares a claim type (ownership, fair use, or public domain) for a source video, or records third-party permission evidence (grantor, evidence URL, scope). 2. System persists the record, keyed to the user and the canonical source id.
Alternate Flow	A1. A user attestation is recorded as an unverified declaration distinct from a platform-verified basis; the two are not conflated when the publication gate is later evaluated.

USE CASE 10: View Job Rights Manifest

ID	UC-10
Name	View Job Rights Manifest

Description	A user views a complete rights-and-attribution report for a completed job.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User owns the job (or is an Admin).
Post-Condition	No state change; the manifest is returned.
Basic Flow	1. User requests the rights manifest for a job they own. 2. System verifies ownership without revealing whether a job id exists to anyone who does not own it. 3. System returns the job's frozen rights_snapshot — captured once, at render time, as a write-once historical record — together with an objective report of which physical clip intervals came from which source, and the resulting publication-gate status.
Alternate Flow	A1. If the job predates the rights_snapshot feature, a reconstruction is built from current rights records instead, but is explicitly labelled as a reconstruction rather than historical evidence, and can never itself authorise publication.

USE CASE 11: View History

ID	UC-11
Name	View History
Description	A user views their past searches, transcript views, and merge jobs.
Primary Actor	Registered User
Secondary Actor	None
Pre-Condition	User is authenticated.
Post-Condition	No state change.
Basic Flow	1. User opens the History panel.

	2. System returns the user's history entries, newest first, each carrying its type and type-specific fields (a search query, a viewed video id, or a completed merge's job id and video count).

USE CASE 12: Manage Users (Admin)

ID	UC-12
Name	Manage Users
Description	An administrator lists all users, views aggregate platform statistics, and activates, deactivates, or edits a user account.
Primary Actor	Admin
Secondary Actor	None
Pre-Condition	The caller's role, verified from the token on every request rather than trusted from client state, is "admin".
Post-Condition	The requested listing is returned, or the targeted account is updated.
Basic Flow	1. Admin opens the admin user-list or statistics panel. 2. System returns every user's profile and role (never a password hash) or the requested aggregate statistics. 3. Admin toggles a user's active status or edits their profile fields.
Alternate Flow	A1. An admin cannot deactivate their own account through this use case, preventing an admin from locking themselves out of the platform.

CHAPTER 4:
DESIGN

The diagrams referenced in this chapter are inserted as figures alongside the explanatory text below. Each subsection describes what the corresponding diagram shows and why it is structured the way it is, grounded in the actual modules and classes present in the codebase.

Architecture Diagram

The architecture diagram shows VidFusion as a layered system: a React/Tailwind single-page frontend, a FastAPI REST API, a durable MongoDB-backed task queue, two independent background workers, and a distinct layer of locally-hosted AI models the pipeline depends on. The frontend never talks to MongoDB, ffmpeg, or the AI models directly — every request goes through the FastAPI layer, which is the only component holding database credentials and model configuration. Inside the backend, the diagram separates Routers (thin HTTP-to-service wiring, one per feature area: auth, merge, transcript, search, chat, rights, youtube), the Merge Pipeline (the core orchestration that runs transcription through to evidence-manifest publication), and the Service Layer (semantic segmentation, concept extraction, coverage selection, summarisation policy, fusion, rights, and acquisition-observability logic). A distinct pair of boxes represents the Video Worker and Text Worker, drawn separately from the API process because — when SEPARATE_OUTPUT_WORKERS_ENABLED is set, which is how the project's own docker-compose.yml runs it — they are genuinely separate OS processes (and, in the Docker topology, separate containers) that poll the durable output-task queue independently, rather than threads inside the API process. The diagram is structured this way specifically because the lease-based claiming logic in output_task_queue.py is written to be correct under multiple concurrent worker instances, which the current one-worker-per-type deployment does not strictly require but is deliberately built to support.

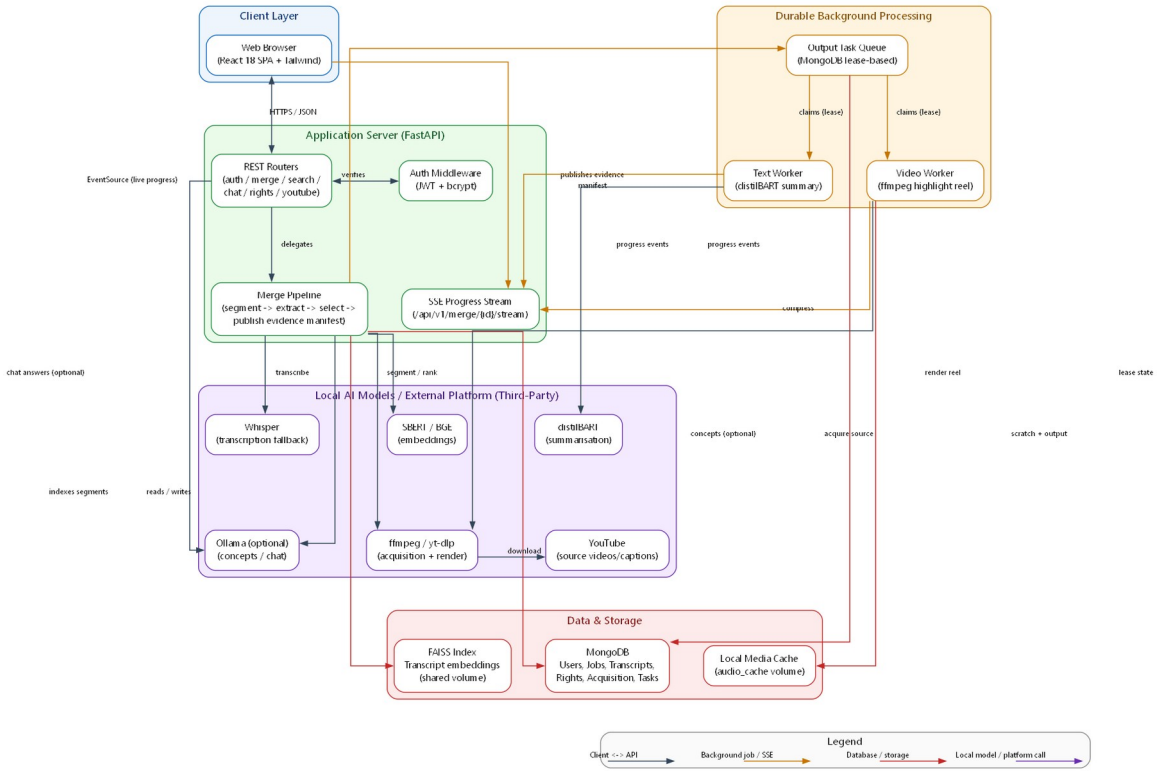


Figure 2: System Architecture Diagram

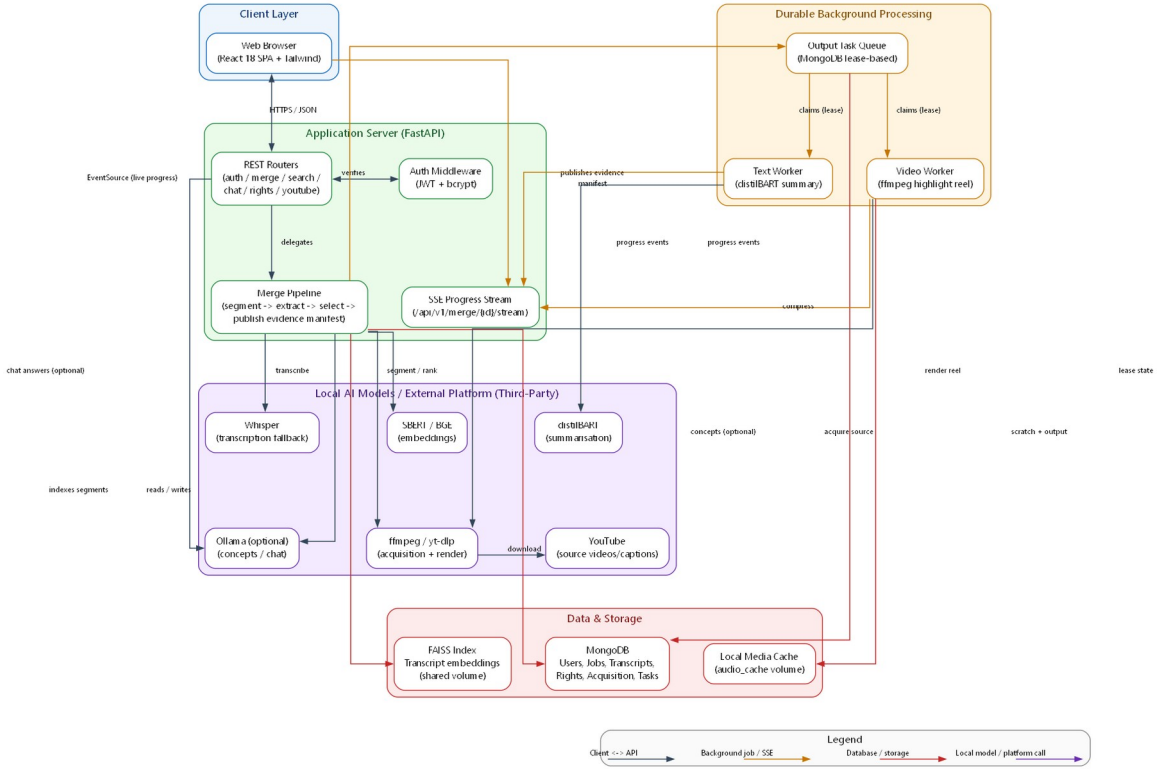


Figure 2b: System Architecture Diagram (reference rendering)

Entity Relationship Diagram (ERD)

The ERD is built directly from the collections the backend actually persists to: users, history, jobs, transcripts, source_rights, acquisition (the decisions/events/manifests of the shadow-observability system), and output_tasks. A user document is the root entity every other collection references by user_id. The jobs collection is the largest and most central: each MergeJob document embeds its own results (segments, concepts, rich_output, fusion_metadata, selection_metrics) directly on the document rather than splitting them into child collections, because they are always read and written as a unit with their parent job and never queried independently of it — the same "document-shaped, not relational" reasoning that motivated choosing MongoDB in the first place (Chapter 6). The output_tasks collection has a one-to-many relationship from jobs (one job can have up to two durable tasks, one per output type) and is the one collection with its own independent lifecycle fields (lease, worker ownership, attempts) distinct from the parent job's own status, reflecting that a task's claim/lease state is worker-queue bookkeeping, not job-level business state. source_rights has a many-to-one relationship into users (many rights records per user, one per source video they have asserted a basis for) and is referenced from a job's frozen rights_snapshot by value, not by live reference, since a snapshot must not change retroactively if the user's recorded rights basis changes later.

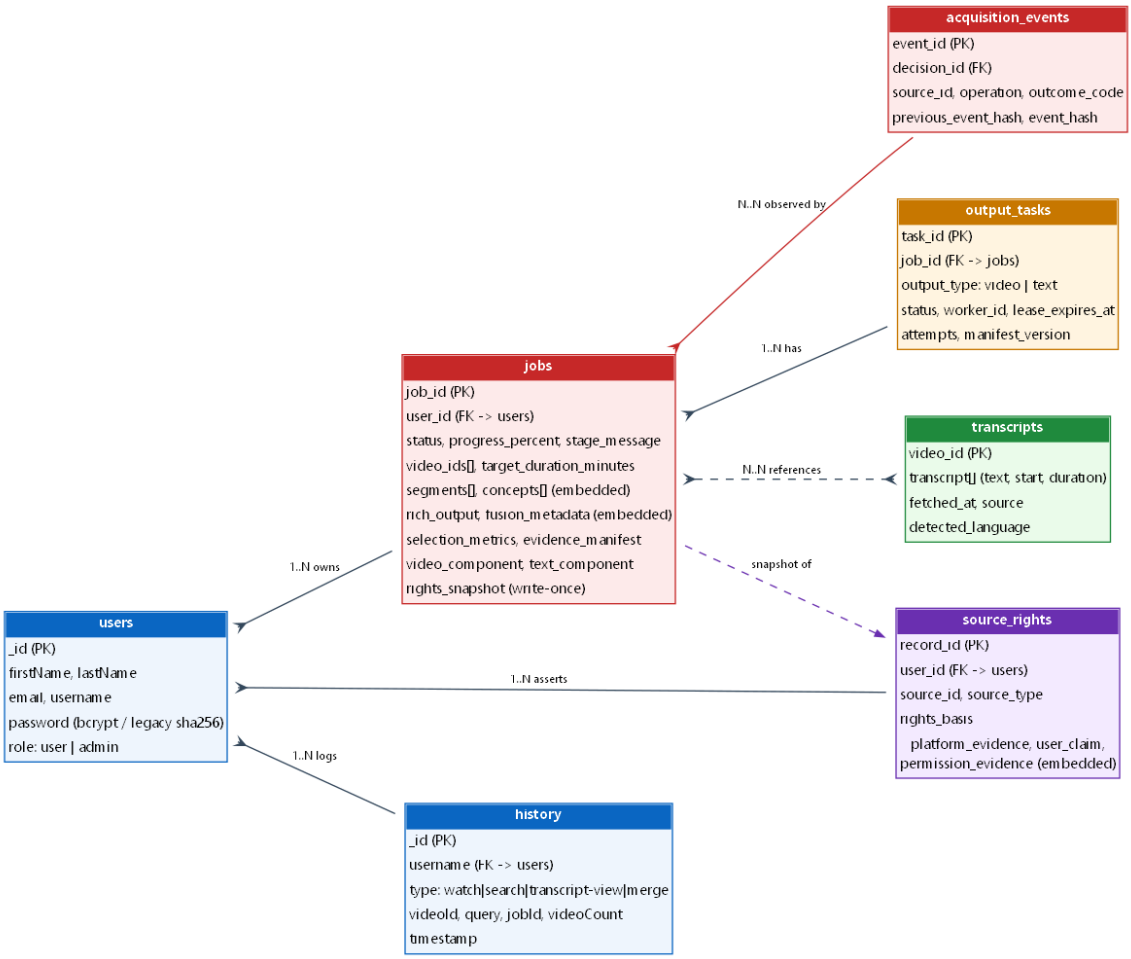


Figure 3: Entity Relationship Diagram (ERD)

DFD Level 0 (Context Diagram)

The Level 0 diagram treats the whole VidFusion backend as a single process and shows its data flows to and from four external entities: the User (via the browser), the Database (MongoDB), the Video Platform (YouTube, reached for source video/caption acquisition), and the AI Model Layer (collectively, since at this level which specific model — Whisper, SBERT, distilBART, or Ollama — handles a given call is not yet relevant). The User submits video selections and a target duration and receives job-progress updates and a finished summary; the system reads and writes user accounts, job state, and transcripts to the Database; it fetches source video, audio, and captions from the Video Platform; and it sends transcript text to, and receives transcriptions, embeddings, summaries, and extracted concepts from, the AI Model Layer.

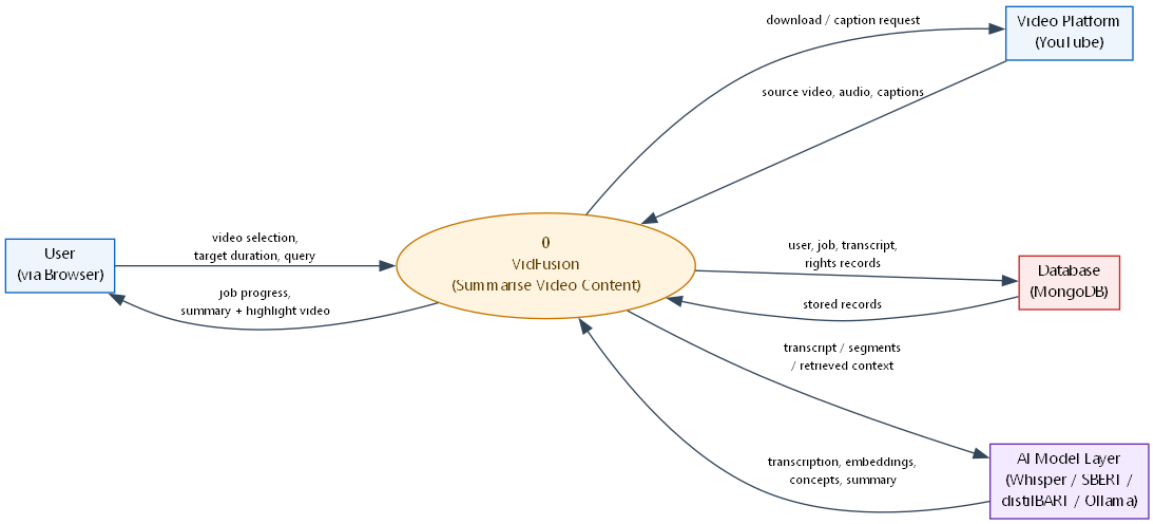


Figure 4: Data Flow Diagram - Level 0

DFD Level 1

The Level 1 diagram decomposes the single process from Level 0 into its major functional processes: Authentication, Merge Job Creation, Semantic Selection (segmentation, concept extraction, coverage-budgeted selection), Output Generation (video rendering, text summarisation), Search/Chat, and Rights Governance, each shown with its own data flows into and out of the relevant data stores. Output Generation is drawn as consuming a single, fixed input — the evidence manifest produced by Semantic Selection — rather than reading the raw transcript or re-running selection itself, which mirrors the actual code path: once a manifest is published, the video and text workers are deliberately prevented from seeing anything upstream of it, so neither output's generation logic can silently diverge from what was actually selected.

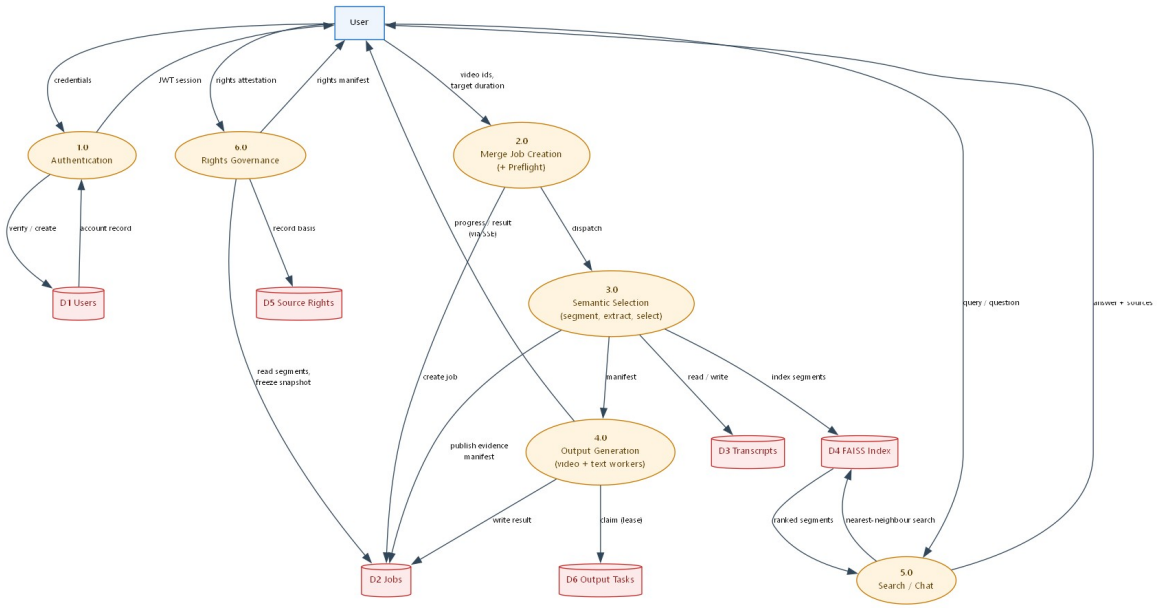


Figure 5: Data Flow Diagram - Level 1

Class Diagram

The class diagram combines the persistent data model with the Service-layer classes that create or mutate it. MergeJob sits at the centre, referenced by output_tasks (a 1-to-N composition, since a task's lifecycle is entirely owned by its parent job) and User (a many-to-one "belongs to" association). MergeJob contains, as embedded value objects rather than separate referenced classes (UML composition, the filled-diamond end), its own ConceptInfo list, VideoSegment list, RichOutput, and FusionMetadata, mirroring that these are read and written as part of the job document itself. SourceRightsRecord is drawn referencing User by id rather than being embedded in it, since a user accumulates many rights records over time and they are queried independently (by source id) rather than always loaded with the user. Around this data model, the Service classes are shown with dashed dependency arrows: SemanticSegmenter and ConceptExtractor both produce input consumed by CoverageSelector; CoverageSelector's output is consumed independently by both VideoService (rendering) and SummarizationPolicy/FusionEngine (text), reflecting the manifest hand-off described above; RightsService is shown depending on MergeJob only to read its selected segments and source durations when building a reuse report, never to mutate job state itself.

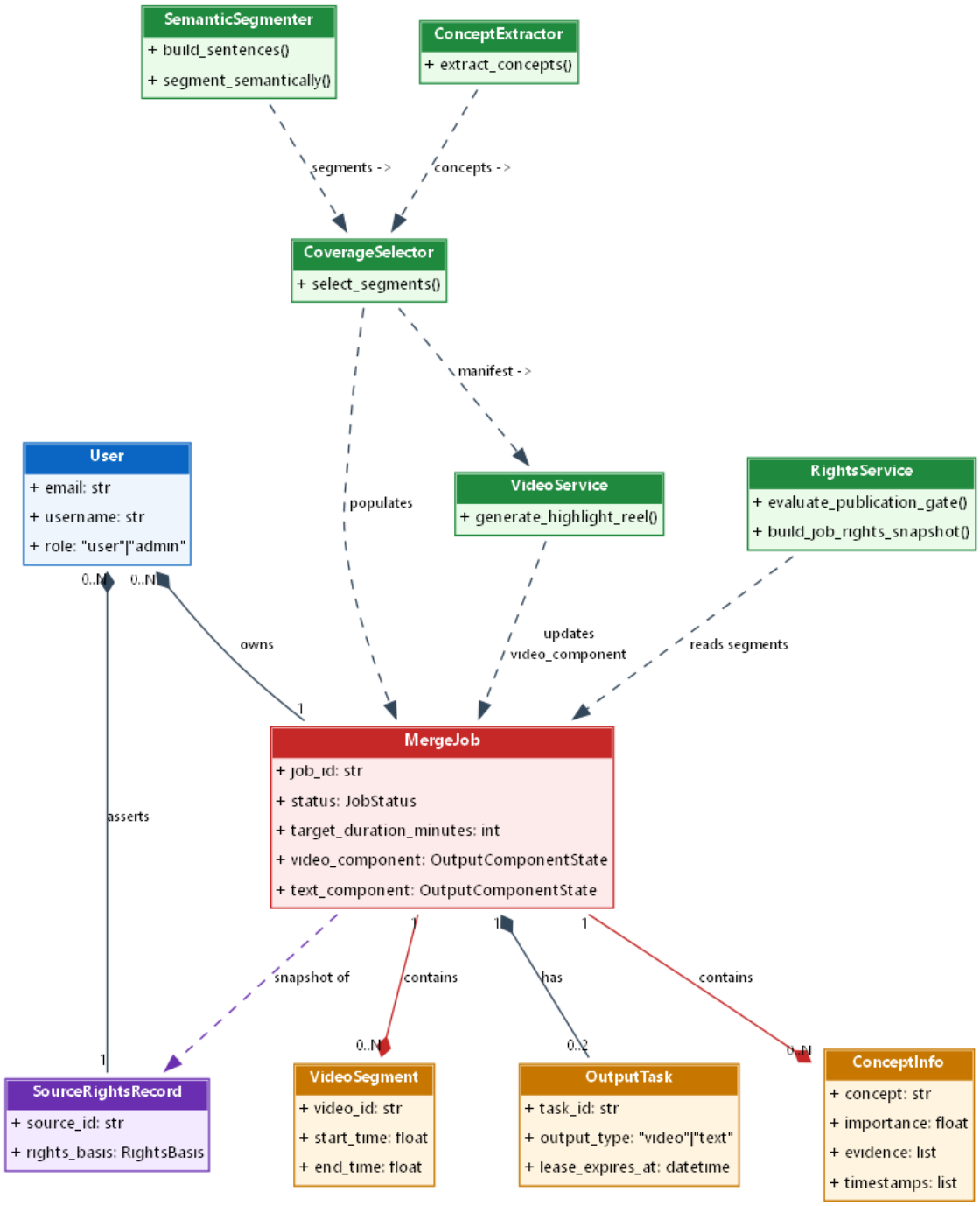


Figure 6: Class Diagram

Activity Diagram

The activity diagram traces the Create Merge Job use case as a swimlane flow between the User, the API, and the two Background Workers. The User's lane is short — submit sources and a duration, later view the result — while the pipeline's lane carries the bulk of the activity: transcribe, segment semantically, extract concepts, select segments under budget, then a fork into two parallel branches once the evidence manifest is published — one branch

(the Video Worker) rendering the highlight reel with ffmpeg, the other (the Text Worker) running the summarisation policy and producing the structured written output — which join only in the sense that both write back to the same job document, each updating only its own component's status field. A decision diamond appears after the concept-extraction step, representing the Ollama-available/unavailable branch that determines whether concepts come from the LLM path or the TF-IDF fallback, and another after segmentation, representing the SBERT-available/unavailable branch that determines whether semantic boundaries or duration-only boundaries are used.

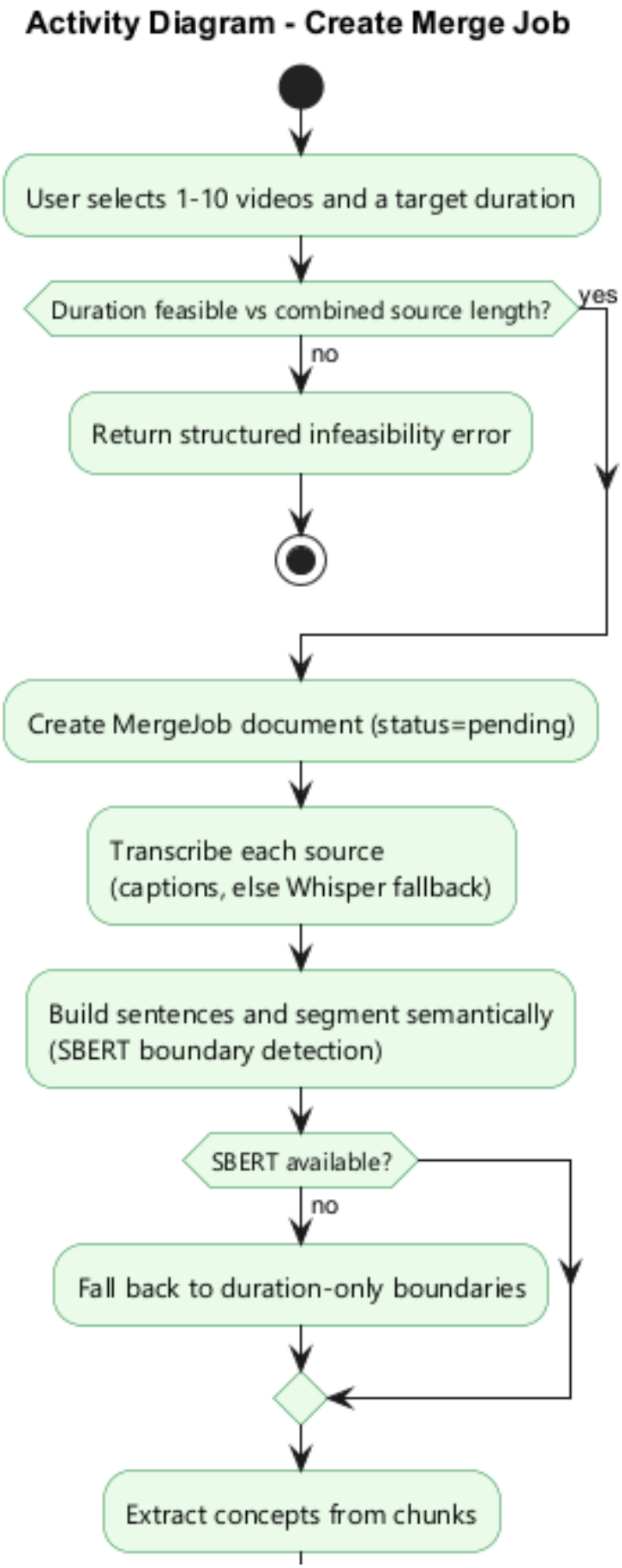

Activity Diagram - Create Merge Job
User selects 1-10 videos and a target duration
Duration feasible vs combined source length?
yes
no
Return structured infeasibility error
Create MergeJob document (status=pending)
Transcribe each source
(captions, else Whisper fallback)
Build sentences and segment semantically
(SBERT boundary detection)
SBERT available?
no
Fall back to duration-only boundaries
Extract concepts from chunks

Figure 7: Activity Diagram - Create Merge Job

Sequence Diagram

The sequence diagram details the Create Merge Job use case through to the video_ready checkpoint, chosen because it best demonstrates the system's two-worker, manifest-handoff design. It shows the frontend calling POST /api/v1/merge, the router validating duration feasibility and creating the MergeJob document, and returning a job_id immediately rather than blocking on the pipeline. A second, independent timeline then shows the core pipeline running segmentation, concept extraction, and selection, and publishing the evidence manifest; from that point, two further independent lifelines show the Video Worker and Text Worker each separately claiming a durable task referencing that manifest, each updating only its own component's state, and the Video Worker's completion being what flips the job into the non-terminal video_ready status while the Text Worker is still running.

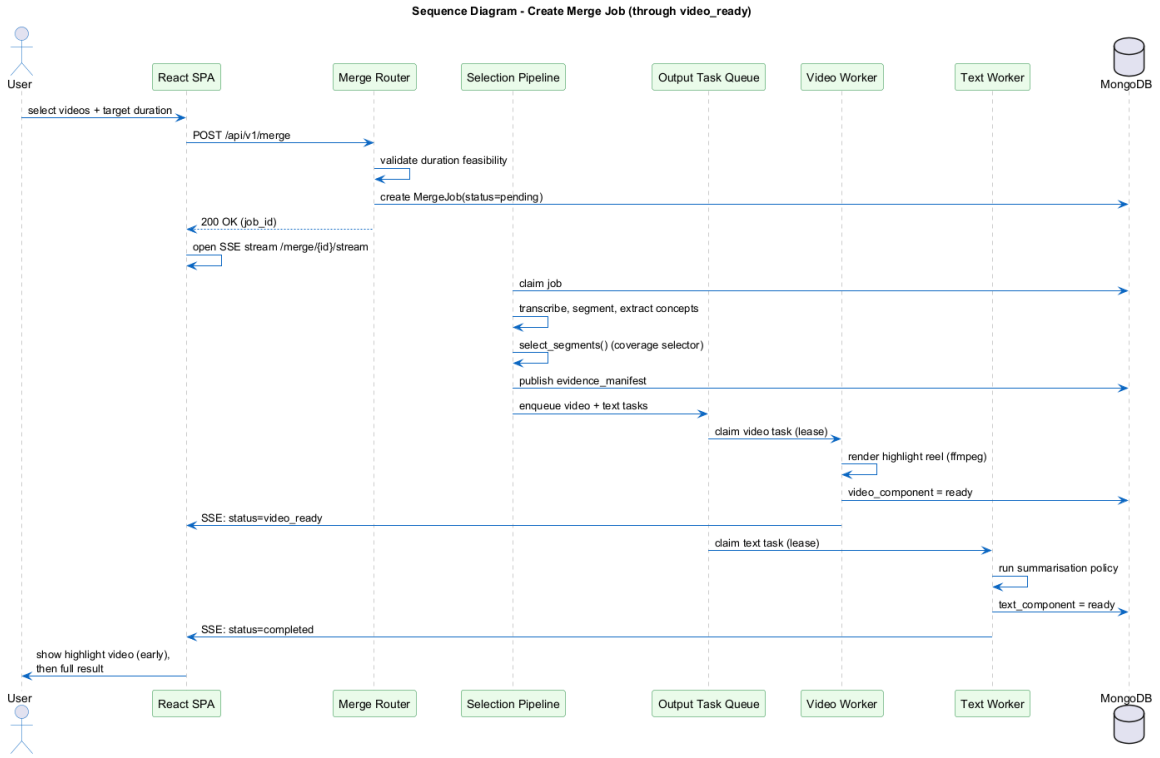


Figure 8: Sequence Diagram - Create Merge Job

Collaboration Diagram

The collaboration diagram represents the same Create Merge Job use case as the sequence diagram but organised around the objects and their message links. The request chain runs across the top — User to the React SPA, to the Merge router, to the job-creation logic —

exactly as in the sequence diagram, but the collaboration view's real value is in what radiates outward from CoverageSelector: it receives segments from SemanticSegmenter and concepts from ConceptExtractor (messages 4 and 5), and its own output is a single evidence manifest consumed independently by VideoWorker and TextWorker (messages 6 and 7), making explicit that CoverageSelector is the single point where segmentation and concept signals are combined, and that neither output worker is ever reachable directly from the router or from each other.

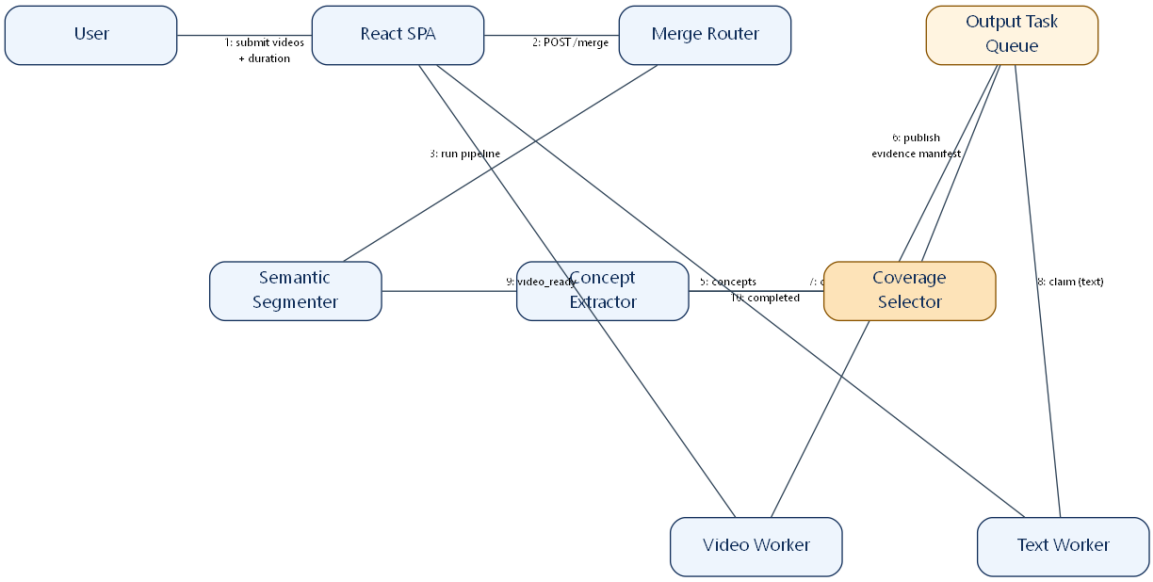


Figure 9: Collaboration Diagram - Create Merge Job

State Transition Diagram

The state transition diagram models the lifecycle of a MergeJob's status field, which is the clearest explicit state machine in the system: pending -> transcribing -> analyzing -> fusing -> summarizing -> enriching -> generating_video, with a non-terminal video_ready checkpoint reachable once the highlight reel specifically is finished but before the written summary has, from which the job still proceeds on to one of the terminal states completed, partial_success (one output component failed, the other succeeded), or error. The diagram also shows cancelled as a terminal state reachable from any non-terminal status. This diagram is included because the reliability story in Chapter 2's non-functional requirements is a claim about this state machine behaving correctly under worker crashes and restarts — which is exactly what the lease/heartbeat fields on the underlying output task (claimed_at, lease_expires_at, worker ownership) exist to guarantee, recovered on worker startup by recover_expired_output_tasks.

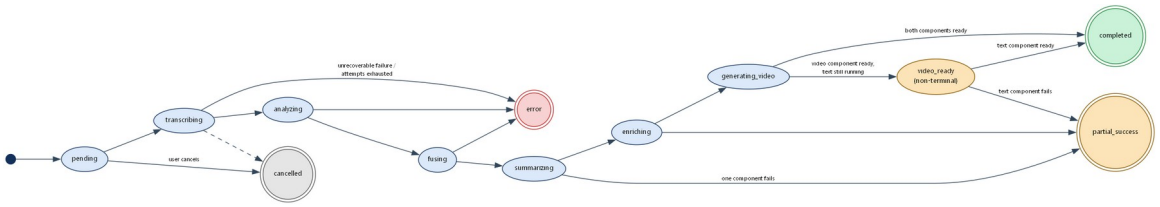


Figure 10: State Transition Diagram - Merge Job Lifecycle

Component Diagram

The component diagram groups the backend into Frontend, Routers, Pipeline/Services, and Infrastructure layers, connected through explicit provided and required interfaces rather than plain arrows. The Frontend depends on a single RouterAPI socket over HTTP/REST; each router then exposes its own provided interface (MergeAPI, AuthAPI, SearchAPI, ChatAPI, RightsAPI) and requires the matching service interface. MergeAPI requires both a SelectionPipelineAPI (segmentation through evidence-manifest publication) and an OutputTaskQueueAPI (for status/retry operations), while ChatAPI requires both a VectorSearchAPI (FAISS) and, optionally, an LLMProviderAPI (Ollama) — drawn with a dashed optional-dependency arrow specifically because the chat use case is designed to function with that dependency entirely absent. On the Infrastructure side, VideoWorker depends on MediaProcessingAPI (ffmpeg) and PersistenceAPI (MongoDB); TextWorker depends on SummarizationModelAPI (the local HuggingFace models) and PersistenceAPI; both depend on OutputTaskQueueAPI to claim and complete their tasks.

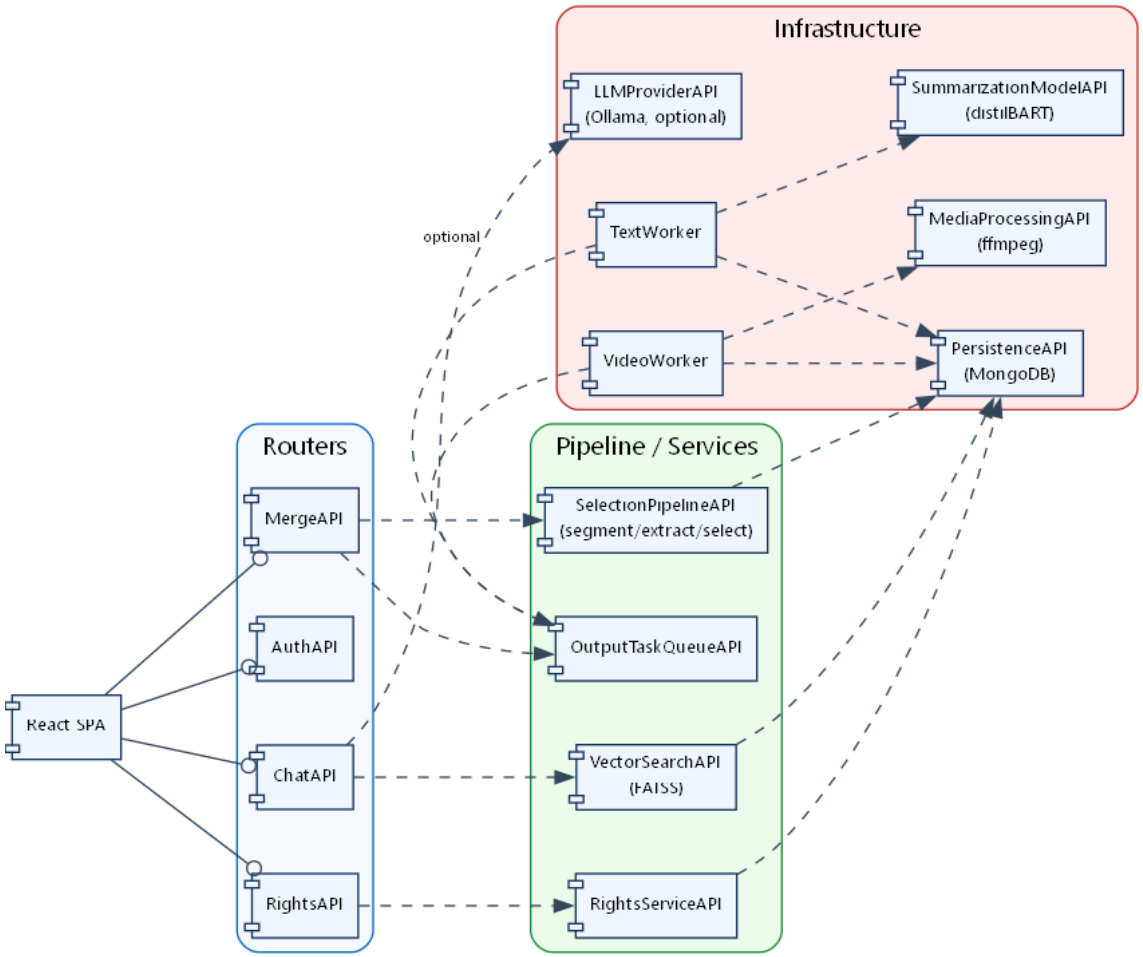


Figure 11: Component Diagram

Deployment Diagram

The deployment diagram shows the project's actual docker-compose.yml topology: an app container (the FastAPI API, exposed on port 8000), a text-worker container and a video-worker container (each built from the same image but started with a different module entry point), and a mongo container (MongoDB 7), all sharing two named Docker volumes — audio_cache (the local scratch and highlight-video cache every container needs read/write access to) and faiss_index (the shared vector search index) — so that whichever container produces or needs a given file sees the same filesystem state regardless of which container actually writes it. The app container is the only one exposed outside the compose network; the two workers and MongoDB are reached only from inside it. This topology is the project's reproducible, self-contained deployment target, independent of any specific cloud platform.

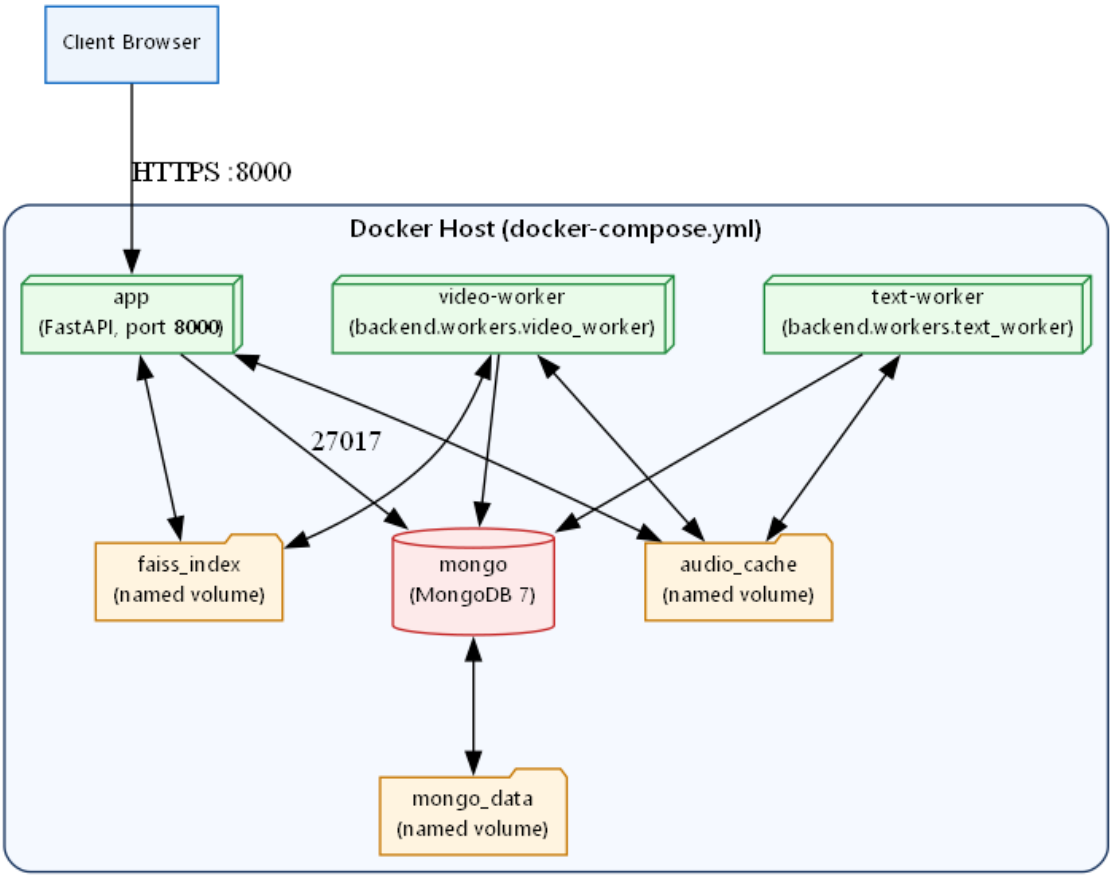


Figure 12: Deployment Diagram

CHAPTER 5:
TESTING

5.1 Test Case Specifications

The nine test cases below are worked examples of the black-box test design techniques covered in Section 5.2 (equivalence partitioning, boundary value analysis, decision tables, and state transition testing), documented in the university's own test-case format (ID, priority, steps, expected result, status) so the design technique behind each one is traceable on paper. They are a representative sample chosen to cover the highest-risk flows (authentication, merge job creation, duration feasibility, job recovery, and rights gating), not an enumeration of every test performed. The actual scale of automated testing is the 548-test suite described in Section 5.3.1.

TC-01

Priority	High
Description	Successful login with correct email and password
Reference	UC-02 Login
Users	Registered User
Pre-requisites	An account exists
Steps	1) POST /api/login with the account's login identifier and password. 2) Inspect the response.
Input	{ "login": "user@example.com", "password": "CorrectPass123" }
Expected result	HTTP 200; response contains a valid JWT access token and the user's profile
Status	Pass

TC-02

Priority	High
Description	Failed login with wrong password
Reference	UC-02 Login
Users	Registered User
Pre-requisites	An account exists

Steps	1) POST /api/login with the correct login identifier and an incorrect password. 2) Inspect the response.
Input	{ "login": "user@example.com", "password": "WrongPassword" }
Expected result	HTTP 401/400; no token issued; an unknown-login attempt returns an equivalent failure so the two cannot be distinguished
Status	Pass

TC-03

Priority	High
Description	Create a merge job with a feasible duration
Reference	UC-03 Create Merge Job
Users	Registered User
Pre-requisites	User authenticated; source video durations resolvable
Steps	1) POST the merge request with 1-10 valid video ids and a target_duration_minutes strictly less than the combined source duration. 2) Poll or stream the job until a terminal or video_ready status. 3) Inspect the result.
Input	{ "video_ids": ["<25-minute source>"], "target_duration_minutes": 10, "generate_video": true }
Expected result	Job progresses pending -> transcribing -> ... -> completed (or video_ready then completed); result.selection_metrics.duration_accuracy is close to 1.0
Status	Pass

TC-04

Priority	High
Description	Reject a merge job whose requested duration is not feasible
Reference	UC-03 Create Merge Job
Users	Registered User

Pre-requisites	User authenticated
Steps	1) POST the merge request with a target_duration_minutes greater than or equal to the combined source duration.
Input	{ "video_ids": ["<8-minute source>"], "target_duration_minutes": 10 }
Expected result	HTTP 422 before any job document is created; the error names the achievable duration range rather than a generic failure
Status	Pass

TC-05

Priority	Medium
Description	Retry only the failed text-output component of a job
Reference	UC-06 Retry a Failed Output Component
Users	Registered User
Pre-requisites	A job exists whose text_component.status is "failed" and whose video_component is already "ready"
Steps	1) POST /api/v1/merge/{job_id}/retry-text. 2) Poll the job. 3) Confirm the video output was not re-rendered.
Input	POST /api/v1/merge/<job_id>/retry-text
Expected result	text_component resets to queued and eventually reaches ready; video_component's existing ready state and video_path are untouched
Status	Pass

TC-06

Priority	High
Description	Worker crash recovery via lease expiry
Reference	Reliability NFR (Chapter 2)

Users	Background Worker (system-level)
Pre-requisites	An output task has been claimed by a worker whose lease has since expired without completion (simulating a crash)
Steps	1) Simulate a worker claiming a task and then stopping without releasing it. 2) Advance time past the lease window. 3) Start a (or another) worker and run its recovery pass.
Input	An output_tasks document with status "claimed" and lease_expires_at in the past
Expected result	Recovery re-queues the task exactly once; a second concurrent recovery attempt does not re-queue it a second time
Status	Pass

TC-07

Priority	Medium
Description	Duration-accuracy boundary at the Express profile's minimum
Reference	FR-04 Create Merge Job
Users	Registered User
Pre-requisites	A source long enough to make a 2-minute summary feasible
Steps	1) POST a merge request with target_duration_minutes = 2 (the Express profile's floor). 2) Inspect the result's selection_metrics.
Input	{ "target_duration_minutes": 2 }
Expected result	duration_style resolves to "headline"; the rendered reel's actual duration is within the configured tolerance of the 120-second budget
Status	Pass

TC-08

Priority	Medium
Description	Record a user rights attestation and confirm it affects the publication gate
Reference	UC-09 Record Source Rights
Users	Registered User
Pre-requisites	User authenticated; a source video with no existing rights record
Steps	1) POST /api/v1/rights/attest with claim_type "ownership". 2) Create a merge job using that source. 3) Request the job's rights manifest.
Input	{ "source_id": "<video_id>", "claim_type": "ownership" }
Expected result	The rights record is persisted; the job's rights_snapshot, frozen at render time, reflects the attested basis and a corresponding publication_gate_status
Status	Pass

TC-09

Priority	High
Description	Non-owner cannot read another user's job rights manifest
Reference	UC-10 View Job Rights Manifest
Users	Registered User
Pre-requisites	Two distinct user accounts; a job owned by user A
Steps	1) As user B, GET /api/v1/rights/job/{A's job_id}/manifest.
Input	GET /api/v1/rights/job/<job_id>/manifest (Bearer token for user B)
Expected result	HTTP 404, identical to the response for a nonexistent job id, so user B cannot distinguish "not yours" from "does not exist"
Status	Pass

5.2 Black Box Test Cases

5.2.1 Equivalence Partitions

Target duration (minutes), mapped to one of six duration profiles: - Partition A (Express): 2 minutes -> headline style, 300-word summary target, 120-second reel budget. - Partition B (Quick Highlights): 5 minutes -> brief style, 750-word target, 300-second budget. - Partition C (Key Concepts): 10 minutes -> standard style, 1500-word target, 600-second budget. - Partition D (Balanced/Detailed): 15-20 minutes -> comprehensive-leaning styles, 2200-2800-word targets. - Partition E (Comprehensive): 30 minutes (the maximum) -> 4200-word target, 1800-second budget. - Partition F (invalid, infeasible): requested minutes such that requested_seconds >= combined source seconds -> rejected at job-creation time with a structured error, never silently clamped.

Video count per job, valid range 1-10: - Partition A (invalid, empty): zero valid ids after deduplication -> HTTP 400, no job created. - Partition B (valid, single source): 1 video -> source_scope resolves to "single". - Partition C (valid, multi-source): 2-10 videos -> source_scope resolves to "multi"; fusion/deduplication across sources is exercised. - Partition D (invalid, over-limit): more than 10 distinct ids -> HTTP 400, no job created.

5.2.2 Boundary Value Analysis (Segment Duration)

- segment length = SEGMENT_MIN_SECONDS (20s) exactly: the minimum legal segment; a boundary below this is not honoured and the segment is folded into its neighbour instead of being kept as an under-length clip.
- segment length = SEGMENT_MAX_SECONDS (90s) exactly: a boundary is forced here even if no natural topic shift was detected, placed at the weakest similarity gap within the legal window rather than at an arbitrary word count.
- segment length just under 20s: folded into an adjacent segment rather than discarded, so content is never silently lost at a sub-minimum boundary.
- budget fill = SELECT_MIN_BUDGET_FILL (0.90) exactly: the optimizer is permitted to stop selecting further segments only once this fraction of the requested duration has been reached, confirmed by a regression test pinned specifically because an earlier version of the optimizer could return a reel at only a small fraction of its budget when the source's remaining material stopped adding marginal value.

5.2.3 Decision Table Testing

The following decision table covers the conditions governing whether a Create Merge Job request proceeds: video ids valid and within 1-10 (Y/N), every source's duration resolvable (Y/N), requested duration feasible against the combined source length (Y/N).

Rule 1: ids=Y, durations=Y, feasible=Y -> Action: job created, HTTP 200/202 with a job_id. Rule 2: ids=N, durations=*, feasible=* -> Action: HTTP 400, no job created (checked first, before any duration probing). Rule 3: ids=Y, durations=N, feasible=* -> Action: HTTP 422 SOURCE_DURATION_UNAVAILABLE naming the unresolved ids, no job created. Rule 4: ids=Y, durations=Y, feasible=N -> Action: HTTP 422 duration-infeasibility error, no job created.

5.2.4 State Transition Testing (Job Lifecycle)

States: pending, transcribing, analyzing, fusing, summarizing, enriching, generating_video, video_ready, completed, partial_success, error, cancelled. - pending -> transcribing: triggered when the background task picks up a newly created job; tested by asserting the status field changes within the pipeline's first observable step. - generating_video -> video_ready: triggered specifically when the video component finishes while the text component has not yet; tested by asserting this intermediate status is reachable and is treated as non-terminal by the API (polling continues rather than stopping). - * -> partial_success: triggered when exactly one of the two output components ends in a failed state while the other reaches ready; tested by forcing one worker's task to fail and asserting the aggregate job status reflects the mixed outcome rather than reporting a flat failure. - claimed -> queued (lease-expiry recovery): triggered on worker startup when an output task's lease_expires_at is in the past; tested by asserting recovery only re-queues a task whose lease has actually expired, never one still within its lease window — this is what stops two worker instances from double-processing the same render.

5.2.5 Use Case Testing

Each of the twelve use cases in Chapter 3 was exercised end-to-end against a running instance: registering and logging in; creating a single-source and a multi-source merge job through to a completed result; retrying a deliberately-failed text component without disturbing an already-ready video component; searching previously indexed transcripts and confirming returned timestamps resolve to the correct point in the source video; asking a grounded question through the chat endpoint both with and without Ollama running,

confirming the fallback path still returns a source-attributed answer; recording a rights attestation and confirming it is reflected in a subsequently generated job's frozen rights snapshot; and exercising the admin user-management routes with both an admin and a non-admin token to confirm the role check is enforced server-side.

5.3 White Box Test Cases

5.3.1 Automated Test Suite

The backend's automated test suite currently totals 548 passing test functions across 57 test files (50 files under tests/, 7 under backend/tests/), run with pytest. The suite's distribution reflects where the project's own risk assessment placed its effort: the largest files are test_multi_video_production.py (67 tests, covering end-to-end multi-source job production), test_concept_extractor.py (35 tests), test_summarization_service.py (33 tests), test_fusion_sentence_extraction.py (26 tests), and test_summarization_policy.py and test_coverage_selector.py (24 tests each) — together accounting for the large majority of the suite and concentrated almost entirely on the semantic selection pipeline described in Chapter 4, which is both the system's central technical contribution and its highest-complexity surface. Further files cover the durable output-task queue and worker recovery logic (test_output_task_queue.py, test_output_workers.py, test_video_first_pipeline.py), the rights and acquisition-observability services (backend/tests/test_rights_service.py, test_rights_hardening.py, test_rights_api.py, test_acquisition_observability.py), language-capability gating and non-English transcript handling (test_language_capability_gate.py, test_language_state_machine.py, test_whisper_multilingual_recovery.py, test_translation_service.py), and API-level security properties (tests/test_phase1_security.py, test_phase1_source_isolation.py, test_player_auth_contract.py). A separate frontend test suite (Jest/React Testing Library) covers the duration-selector and multi-select UI components and the rights-badge/rights-panel components.

Honestly stated, the semantic selection pipeline and the rights/durability subsystems were prioritised for automated coverage precisely because they are the highest-complexity, highest-consequence surfaces; broader end-to-end UI coverage and a small number of older, pre-FastAPI-migration script files at the repository root (one-off benchmarking and audit scripts, not part of the maintained pytest suite) were not brought under the same discipline and are called out here rather than silently omitted.

5.3.2 Evaluation Methodology — Selection Quality

Unlike a conventional text-summarisation project, the primary evaluation question for this system is not "does the output text read well" but "did the system select the right material from the source." This cannot be answered with standard text-generation metrics such as ROUGE or BLEU, which compare generated text against a reference text and say nothing about which parts of a much longer source a system chose to keep; the project therefore built a dedicated evaluation harness (evaluation/) against a hand-authored gold standard rather than reusing a metric designed for a different problem.

The gold standard (evaluation/gold.json) consists of 103 hand-identified important concepts across seven real YouTube videos spanning five genres (two programming tutorials, a long-form health podcast, an interview, two technical lectures, and one general-audience educational video), each concept tagged as "core" (the video is materially incomplete without it) or "secondary" (worth having), with the literal time windows in the source where it is substantively discussed. This gold set was authored by reading the transcripts directly and is deliberately independent of the system's own TF-IDF/SBERT machinery, specifically so the evaluation cannot simply be re-discovering what the system itself would have picked.

Against this gold standard, the harness computes: concept recall (the tier-weighted share of gold concepts a selection touches, requiring at least 8 seconds of overlap with a gold window to count as a hit); concept precision (the share of selected running time that falls inside some gold window); redundancy (the share of gold-hit time spent re-covering a concept already covered, measured relative to the video's own baseline pairwise-similarity rather than a fixed, encoder-dependent threshold); temporal coverage (how many tenths of the source timeline the selection touches at all); a complete-thought ratio (the share of selected segments that both start and end on a real sentence boundary); and duration accuracy (how closely the rendered output's length matches what was requested, given what was actually reachable within the source).

A subsequent, more rigorous integrity-audited evaluation (documented in evaluation/m2_report.md) explicitly partitioned videos into development data (used while tuning the selection weights) and multiple fully-held-out partitions the weights were never exposed to — including short-form, long-form (49-79 minute university lecture) and query-focused holdouts — and ran a component-ablation study on each, comparing the full pipeline against the same pipeline with the concept signal, the coverage term, the redundancy penalty,

the coherence term, or the narrative-ordering step individually disabled. The results were reported honestly rather than selectively: on the held-out long-form partition, removing the coverage term measurably worsened both timeline coverage (7.3 vs 8.3 of 10 deciles touched) and duration accuracy (0.970 vs 0.997), supporting that component's inclusion; but the same audit also reported, without softening the finding, that the redundancy penalty produced no measurable reduction in mean pairwise segment similarity on the held-out data (0.624 with it enabled versus 0.615 with it disabled — in the wrong direction to claim benefit), and that the narrative-ordering component's benefit to perceived summary quality remains "pending human annotation" rather than claimed as proven. Reporting a component that did not demonstrate its intended effect, rather than quietly dropping it from the write-up, was a deliberate choice to keep the evaluation's conclusions trustworthy rather than merely favourable.

5.4 Performance Testing

Performance testing was informal rather than tool-driven: end-to-end job latency was measured directly (Section 5.3.2's fair-latency comparison records real wall-clock timings on held-out videos, e.g. roughly 89 seconds for BGE embedding preparation and roughly 8 seconds for concept extraction on a representative source, against a roughly 63-second shared-preparation cost every method pays regardless of selection strategy), confirming that the semantic selection stages add single-digit-seconds to low-tens-of-seconds of overhead on top of the shared transcript-preparation cost, not a multiplicative blow-up. ffmpeg rendering time scales with the number and length of selected segments rather than with the source video's full length, which was confirmed by comparing render times for the same target duration against sources of substantially different lengths. No dedicated load-testing tool (e.g. k6, Locust) was used within the project timeline; concurrent-job behaviour was exercised only through the automated worker-recovery tests described in 5.3.1, not through a sustained multi-job load test.

5.5 Stress Testing

Stress testing focused on the durable task queue's crash-safety rather than on raw request volume: the automated suite deliberately simulates a worker claiming a task and then disappearing mid-lease, and confirms the recovery sweep re-queues the task exactly once rather than leaving it stuck or double-processing it once a second worker instance also attempts recovery. Stress testing of concurrent ffmpeg renders under heavy simultaneous load

was not performed; the project's development and evaluation environment ran one job at a time.

5.6 System Testing

System testing consisted of running the full stack (the FastAPI app, the MongoDB container, and — where SEPARATE_OUTPUT_WORKERS_ENABLED was set — the two independent worker containers) via docker-compose.yml and exercising a complete user journey through the browser: signing up, creating a multi-source merge job, observing live progress over the SSE stream, reaching the video_ready checkpoint before the written summary had finished, retrieving the completed result, and downloading the rendered video and the exported summary, to confirm the containerised, multi-worker topology behaves identically to the inline single-process development mode from the user's perspective.

5.7 Regression Testing

5.7.1 Selecting Regression Tests

Every bug documented in Chapter 9 with a testable, deterministic root cause was converted into a permanent regression test rather than fixed and left unverified. Confirmed examples: the beginning-heavy selection bias (now pinned by assertions on decile coverage in the coverage-selector tests), the silent BART-bypass defect where a narrative under the word-count threshold skipped compression entirely regardless of its actual coherence (now covered by test_summarization_policy.py's branch-selection tests), and the budget-fill early-stop guard (Section 5.2.2). The principle applied: if a bug could recur silently — its symptom not obviously visible without deliberately checking a metric — it earns an automated regression test.

5.7.2 Regression Testing Steps

1. After any change to a pipeline service module, run the full suite (PYTHONPATH=. pytest tests/ backend/tests/ -q) and confirm all 548 tests still pass before considering the change complete.

2. For changes touching the coverage selector or concept extractor specifically, confirm test_coverage_selector.py and test_concept_extractor.py still pass, since these pin the exact scoring behaviour the evaluation methodology in 5.3.2 depends on.

3. For changes touching the durable task queue or worker recovery, confirm test_output_task_queue.py and test_output_workers.py still pass, since these pin the lease/claim contract both workers depend on.

4. Re-run the evaluation harness (evaluation/evaluate.py, evaluation/analyse.py) after any change to the selection scoring weights, and compare against the previously recorded results.json rather than assuming an improvement, since a weight change that looks reasonable in isolation can regress the held-out metrics.

CHAPTER 6:
TOOLS AND TECHNIQUES

Languages

Python 3.12: The entire backend, all AI/ML services, and the semantic selection pipeline are written in Python, chosen because every machine-learning library the project depends on (transformers, sentence-transformers, faiss-cpu, openai-whisper, scikit-learn) is Python-first, and writing the orchestration layer in the same language avoids a cross-language boundary between the API and the models it calls.

JavaScript/JSX (React): The frontend is written in JSX, chosen for its component model's fit with the genuinely stateful, live-updating parts of the UI — the job-progress view consuming a Server-Sent Events stream, and the merge-preview page's multi-video selection state.

Frameworks and Libraries

FastAPI: Chosen as the backend web framework [4] for native async support (used throughout for MongoDB access via Motor and for the SSE progress stream), automatic OpenAPI documentation generation (exposed at /docs), and first-class Pydantic integration, which is what lets every request and response model in the project (MergeJobCreate, MergeJob, the rights models) be validated declaratively rather than by hand.

React 18 with Tailwind CSS: React [5] was chosen for the frontend for its component reusability across the search, transcript-viewer, and merge-preview pages. Tailwind CSS was layered on top for utility-first styling rather than a full component-library dependency, keeping the frontend's own bundle size and styling surface smaller than adopting a complete design-system library would have.

Database

MongoDB with Motor/Pymongo: A document database [6] was chosen because the core content record — a MergeJob, with its embedded segments, concepts, rich_output, and fusion_metadata — is naturally document-shaped and varies in structure between a single-source and a multi-source job, which would require more schema-migration overhead in a relational model. Motor (the async driver) is used from the FastAPI request path so database access never blocks the event loop; Pymongo (synchronous) is used from the worker processes, which run their own process loop rather than FastAPI's.

Media Acquisition and Processing

yt-dlp: Used for downloading source audio/video from YouTube [7], chosen as the de facto standard tool for this purpose, actively maintained against YouTube's frequently-changing internal APIs; the project additionally depends on yt-dlp-ejs specifically for resilience against YouTube's JavaScript-challenge-based access restrictions.

ffmpeg: The actual video compositor — it trims and concatenates the segments the coverage selector chooses, applies transitions between segments from different time ranges or different source videos, and produces the final highlight-reel file. ffmpeg was the only realistic choice for this: it is the de facto standard for programmatic, scriptable video composition, and the project's Docker image build depends on a correctly-configured installation of it.

OpenCV and PySceneDetect: Used for visual keyframe extraction and shot-boundary detection, forming the visual-signal input to the selection pipeline. This signal is currently disabled by default (MERGE_ENABLE_VISUAL_SCORING=0) in favour of a lighter-weight audio-energy scorer for single-speaker, audio-led content such as podcasts and lectures, which is the dominant content type the system was developed and evaluated against; the visual path remains implemented and tested rather than removed, for content types (demonstrations, visually-driven tutorials) where it would matter more.

AI / ML Models

OpenAI Whisper: Used as the transcription fallback [8] when a video has no usable platform-provided captions — the primary path tries YouTube's own caption track first, since it is free and instantaneous, and only falls back to running Whisper locally (the "base" model by default, trading some accuracy for roughly 3.4x real-time throughput on CPU) when captions are unavailable or in an unsupported format.

Sentence-BERT (BAAI/bge-base-en-v1.5): The embedding model behind semantic segmentation, concept-deduplication, redundancy scoring, and the FAISS search index [9]. It was chosen over the originally-used all-MiniLM-L6-v2 specifically for ranking highly on the MTEB benchmark at a still-practical CPU inference cost; the migration from MiniLM to BGE required recalibrating a downstream similarity threshold (concept-deduplication merging) that had been tuned against MiniLM's narrower similarity range, which is documented directly as a lesson learnt in Chapter 9, since an absolute threshold tuned for one encoder is not safely portable to another.

distilBART (sshleifer/distilbart-cnn-12-6): The abstractive summarisation model used for hierarchical compression of selected transcript text [10]. It was chosen over full BART-large specifically for its much smaller footprint (306 MB versus roughly 1.6 GB) at a measured quality cost the project judged acceptable for this use case, since the model only ever has to compress already-selected, already-concept-relevant text rather than summarise an entire raw transcript from scratch.

FAISS: Facebook AI Similarity Search [11] backs the semantic-search and chat-retrieval features, chosen for its maturity and its ability to perform exact or approximate nearest-neighbour search over the dense embedding vectors produced by the SBERT model at a scale well beyond what a naive linear scan could sustain as the number of indexed videos grows.

Ollama (llama3.2:3b): An optionally-present, locally-hosted LLM used for two purposes: concept extraction (asking what a transcript chunk is about, in JSON mode) and chat answer synthesis (producing a natural-language answer from retrieved context) [12]. It is disabled for concept extraction by default (ENABLE_CONCEPT_LLM=false) specifically because it was benchmarked at roughly 180 seconds per transcript chunk on the project's CPU-only development hardware — a genuinely measured hardware constraint, not a design preference — with a deterministic TF-IDF keyphrase extractor used as the default fallback instead; both the LLM path and the fallback are fully implemented and tested, so enabling the LLM path on more capable (GPU) hardware requires only a configuration change.

deep-translator and NLLB-200: Used for translating non-English transcripts along a fast, free Google-Translate-backed path with a circuit breaker (after a small number of consecutive failures, the remaining text is left untranslated rather than retried indefinitely against a provider that has started blocking requests), and a slower, more accurate NLLB-200 fallback reserved for short transcripts specifically because it was measured to be too slow (over 2 seconds per segment on CPU) to apply to a long transcript within a usable request timeout.

edge-tts: A free text-to-speech engine used where an audio narration output is requested, chosen for requiring no API key and no paid quota, consistent with the project's broader preference for dependencies that do not introduce a per-request cost or an external quota the platform does not control.

Authentication and Security

JWT (python-jose) and bcrypt (passlib): JWTs [13] were chosen for stateless session authentication, with the algorithm pinned to HS256 at the configuration-validation level rather than left client-negotiable. bcrypt was chosen for password hashing as the established standard for this purpose; the system additionally carries a documented migration path for accounts whose password was hashed under an older SHA-256 scheme, verifying against that legacy format and transparently upgrading it to bcrypt on the account's next successful login, rather than forcing every existing user through a disruptive password reset.

DevOps

Docker and Docker Compose: The backend API, the dedicated text-output worker, and the dedicated video-output worker are each built from the same Docker image but started with a different process entry point, alongside a MongoDB container, orchestrated with Docker Compose [14]. Compose was chosen over a heavier orchestrator because the project's actual deployment target is a small, fixed set of services on a single host, for which Compose's declarative multi-container file is sufficient.

Testing

pytest and pytest-asyncio: Chosen as the test framework [15] for its fixture system (used extensively to provide a clean, isolated environment for each of the 548 tests without requiring a live MongoDB instance for the majority of them) and native support for testing FastAPI's async request handlers and the workers' async claim/lease logic directly.

Version Control

Git and GitHub: Used for source control and collaboration throughout development.

CHAPTER 7:
SUMMARY AND CONCLUSION

This project set out to build a video-summarisation system whose central claim — that it selects the parts of a source video that actually matter — could be measured rather than merely asserted, and to build the reliability and governance machinery a genuinely multi-user, always-on deployment of such a system would need. Both goals were met in substance. VidFusion takes one to ten YouTube videos and a target duration and produces a condensed highlight video cut from the real source footage, a structured written summary with chapters and takeaways, and, for every job, a checkable set of numbers describing exactly how well that job's selection covered the source's timeline and the concepts it actually discussed.

What worked well was the decision to separate "what is this video about" from "summarise it", and to make selection a budgeted optimisation over an explicit, inspectable scoring formula rather than a sorted list cut at the budget. This directly fixed a measured defect — early development versions of the selector were beginning-heavy, putting up to 100% of a job's output in the source's first half, confirmed on the project's own completed jobs rather than suspected from reading the code — and the fix (a concave, value-gated temporal-coverage term) was verified, not just asserted, by a subsequent held-out evaluation. The decision to make every pipeline stage degrade rather than fail outright (Chapter 6) also paid off directly: the LLM-backed concept-extraction path had to be disabled by default for a genuine, measured hardware reason during development, and the system's behaviour in that case is not "the feature breaks" but "a deterministic, slightly weaker fallback runs instead" — exactly the outcome the fallback was designed to produce.

The project is also honest about what the evaluation did and did not prove. A rigorous, contamination-audited study partitioned videos into development data and several genuinely held-out partitions, and ran a component-ablation study on each — and that same study reported, without softening the result, that one of the selector's five scoring terms (the redundancy penalty) did not demonstrate a measurable benefit on held-out data, and that another (narrative reordering) still awaits human judgement before its benefit can be claimed rather than merely hoped for. Reporting an ablation result that did not favour the system's own design, rather than quietly omitting it, was a deliberate choice, and it is a more honest basis for future work than a one-sided write-up would have been.

Reliability was treated as a design requirement from early in development rather than a late addition. Long-running jobs are tracked through a durable, MongoDB-backed task queue with lease-based claiming, specifically so that a worker crash or a server restart mid-render

recovers the job exactly once rather than losing it or risking a second worker duplicating the work; this was validated directly by automated tests that simulate exactly that failure, not only by code review. The source-rights and publication-gating layer was likewise not a requirement specified up front — it emerged from asking, honestly, what a platform that lets users feed in arbitrary YouTube URLs and produce new, condensed derivative media from them should be responsible for recording about the basis for that reuse.

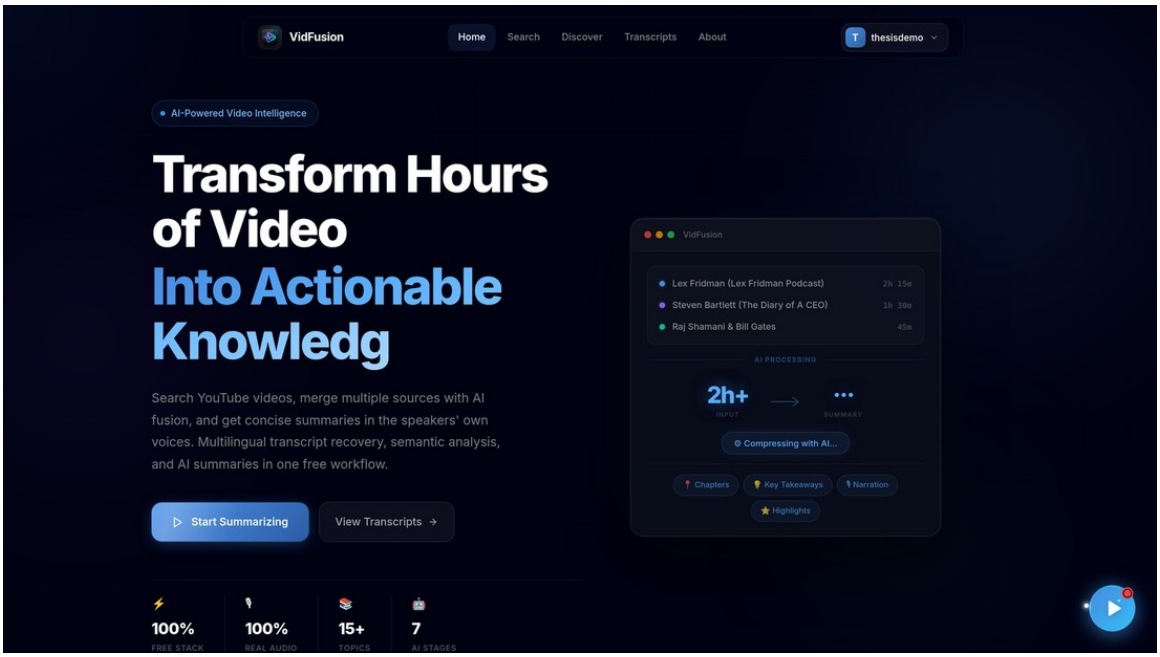
Overall, the project demonstrates a working, non-trivial AI system with genuine technical depth in the area that mattered most for its core promise: a selection pipeline whose claims are backed by a dedicated, held-out, ablation-tested evaluation rather than qualitative impression, and a background-processing architecture built to survive the kind of failure a single-process prototype typically ignores — while being transparent about where the evaluation is still incomplete (human judgement of narrative ordering and overall summary quality), where a component did not earn its place, and where a feature (LLM-backed concept extraction, GPU-class visual scoring) is implemented and tested but deliberately not the default, for reasons documented plainly rather than hidden.

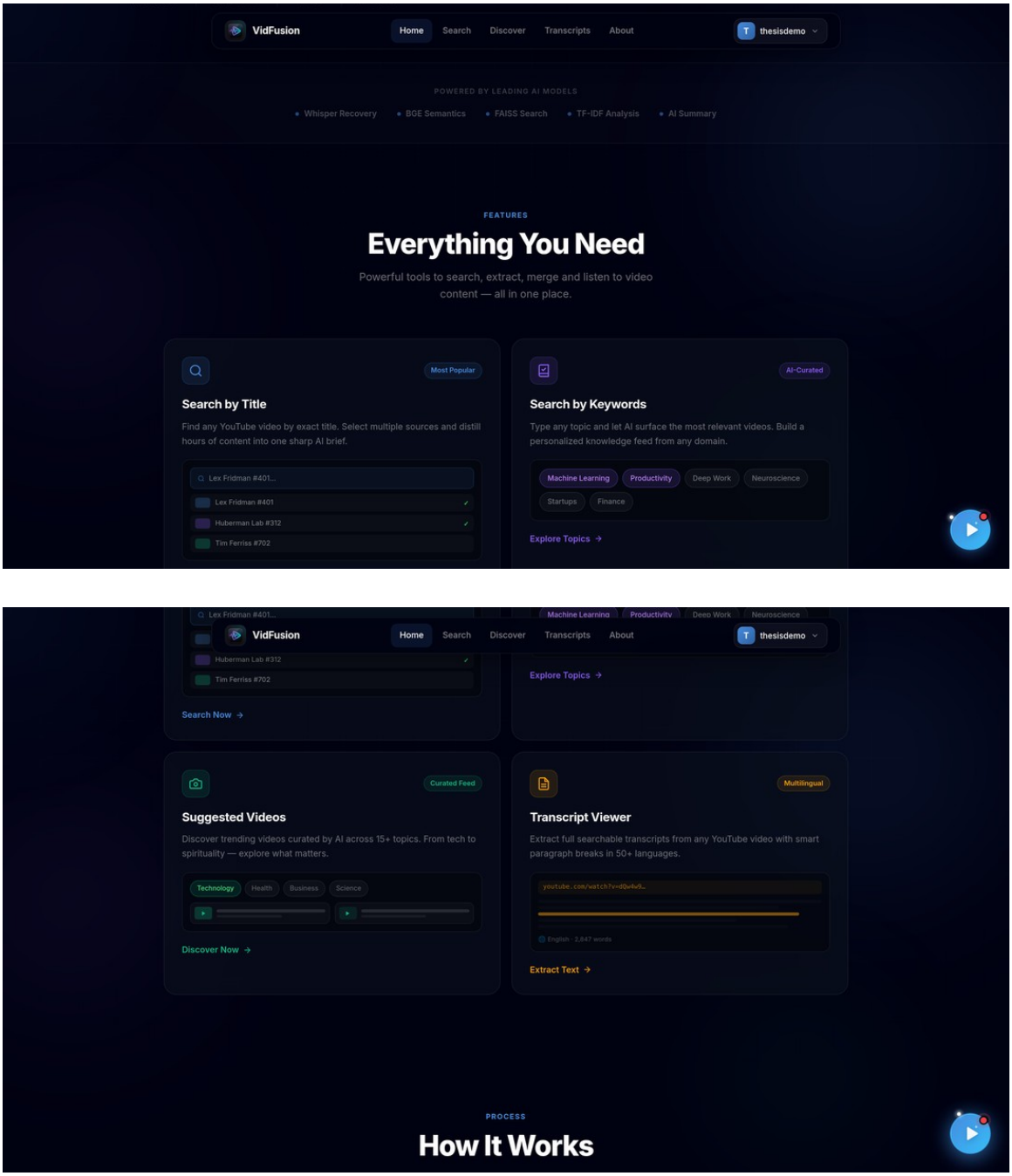
CHAPTER 8:
USER MANUAL

This chapter walks through every feature of VidFusion from the point of view of an end user. All screenshots were captured from a running instance of the system (FastAPI backend serving the React build on a single port, MongoDB, local BART and BGE models) on 20 October 2026, using a demonstration account. The fusion example in Sections 8.6 to 8.8 is a real job: two YouTube videos totalling 22 minutes 25 seconds were fused into a 4 minute 53 second highlight video against a 5-minute target, with about 7 minutes of processing on a CPU-only machine.

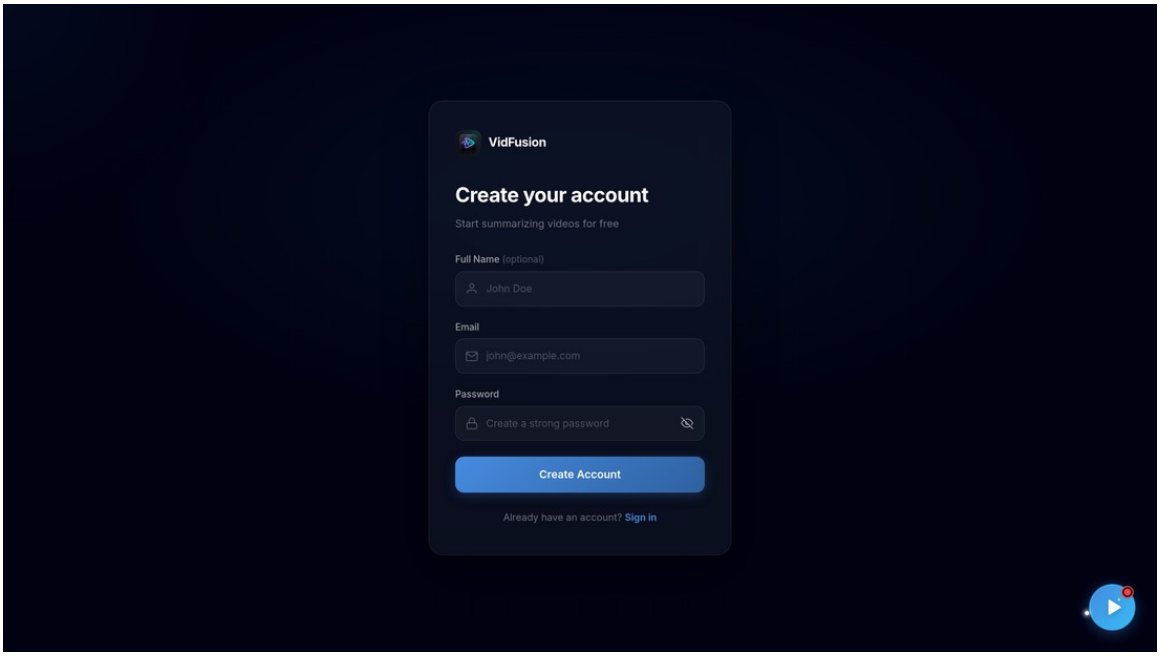
8.1 Sign Up / Login

1. Open the VidFusion landing page. It introduces the platform and its main actions: Start Summarizing and View Transcripts.

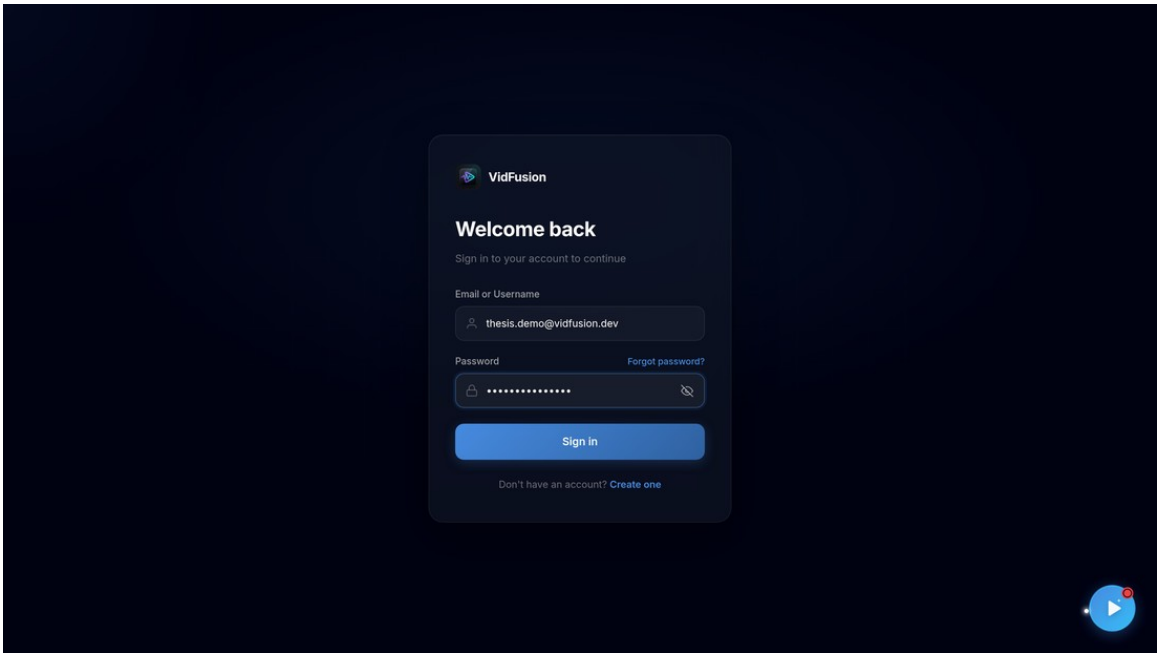




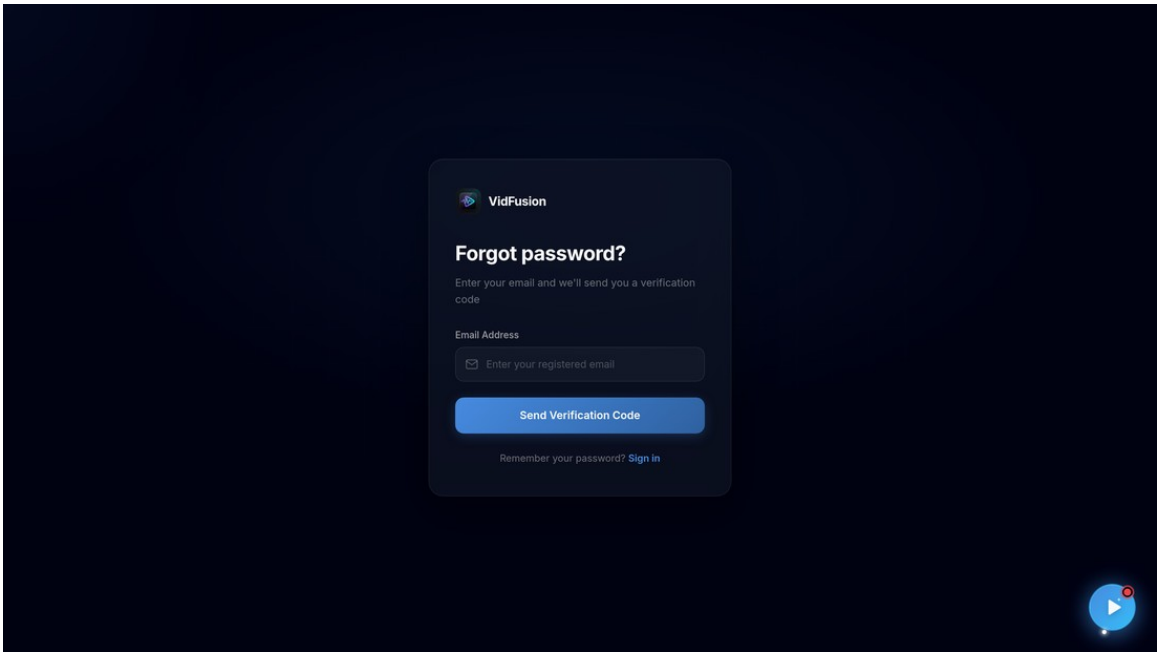
2. Click Sign Up. Enter your full name (optional), email address and a password, then click Create Account.



3. To sign in later, open Log in and enter your email or username and password, then click Sign in.

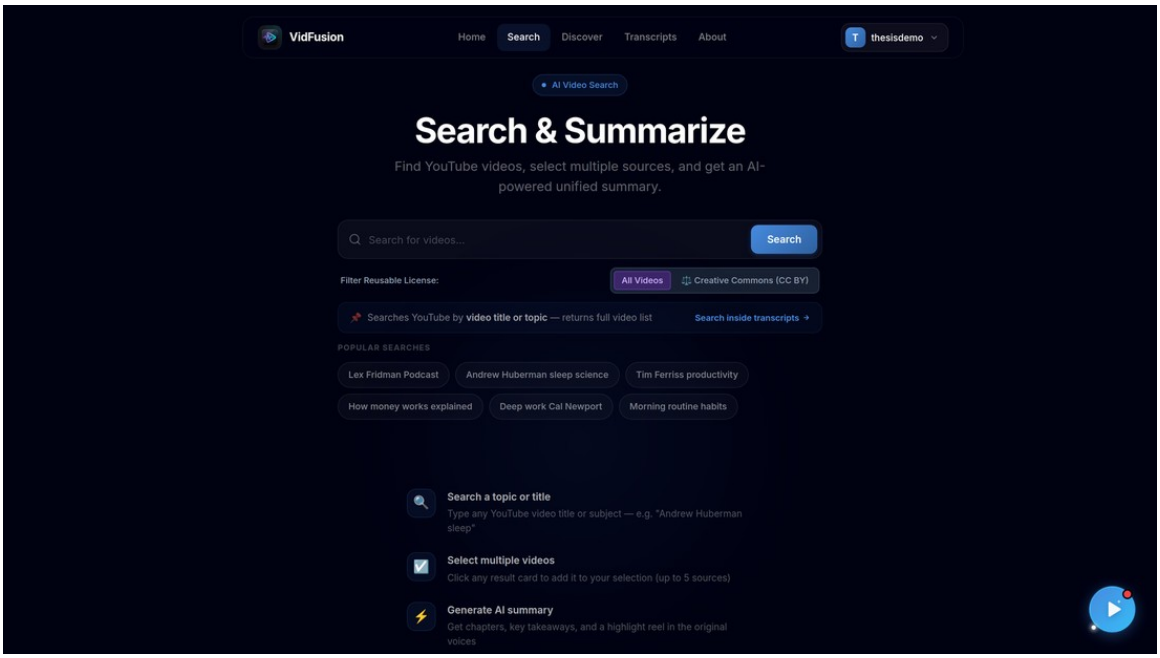


4. If you forget your password, click Forgot password?, enter your registered email and click Send Verification Code. Enter the code you receive to set a new password.

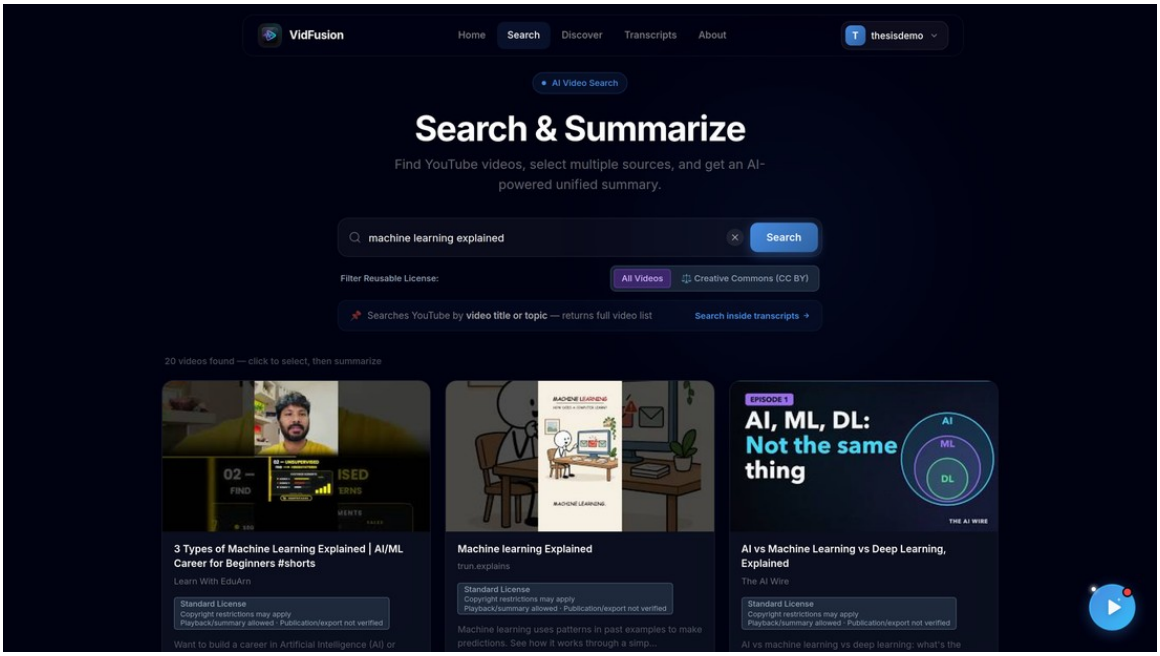


8.2 Search Videos by Title or Topic

1. After signing in you land on Search & Summarize. Type a topic or video title, or click one of the Popular Searches. You can optionally restrict results to Creative Commons (CC BY) videos with the license filter.

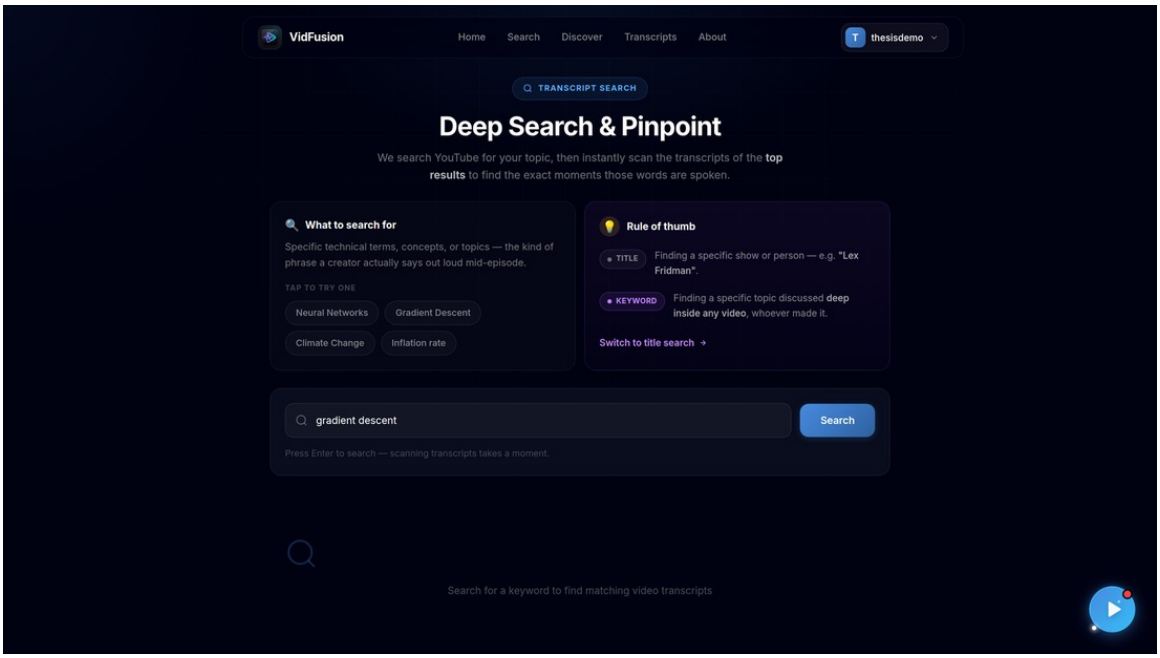


2. Click Search. VidFusion shows matching YouTube videos as cards. Each card shows the title, channel, a license badge and three actions: Add to Merge, Watch and YouTube.

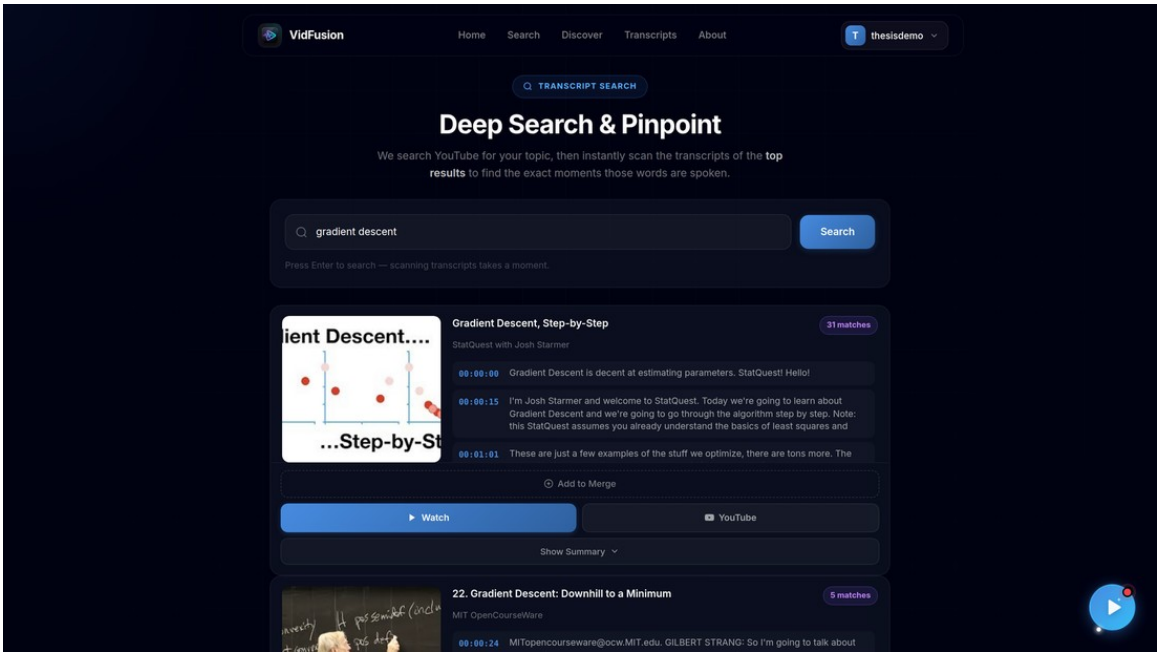


8.3 Search Inside Transcripts (Deep Search)

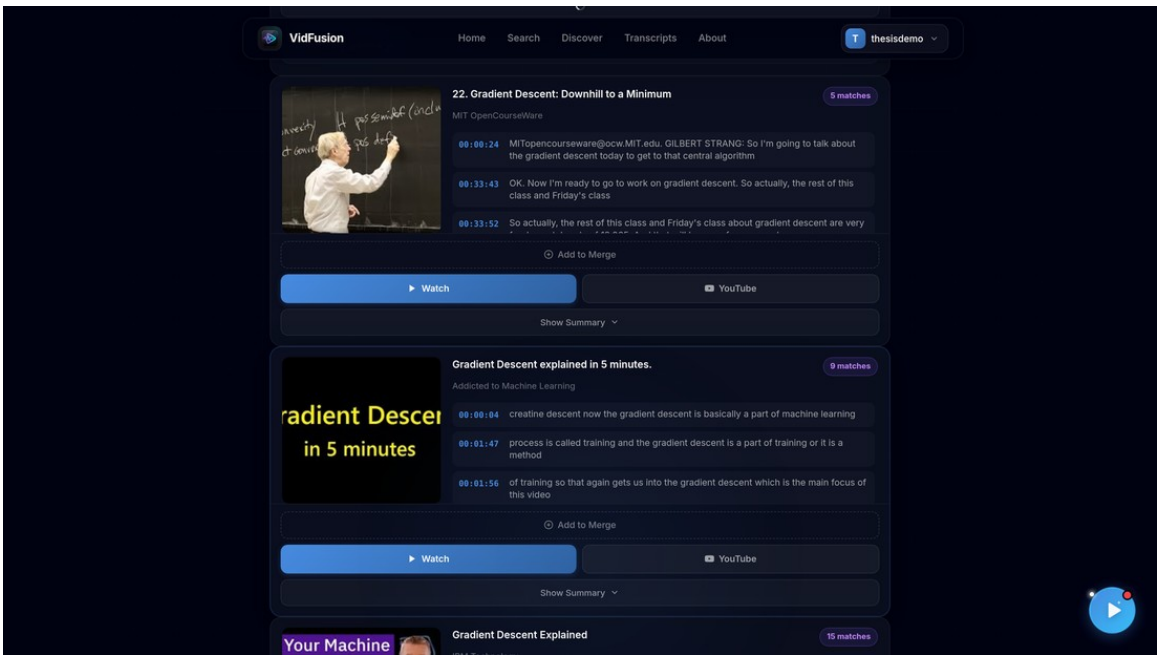
1. Click Search inside transcripts (or open it from the navigation). The page explains when to use a keyword search instead of a title search and offers example terms. Enter a technical term such as "gradient descent" and press Enter.



2. VidFusion searches YouTube for the topic and then scans the transcripts of the top results. Each result card shows the video, its channel, how many times the term is spoken (for example, "31 matches"), and the exact timestamps with the surrounding sentence.

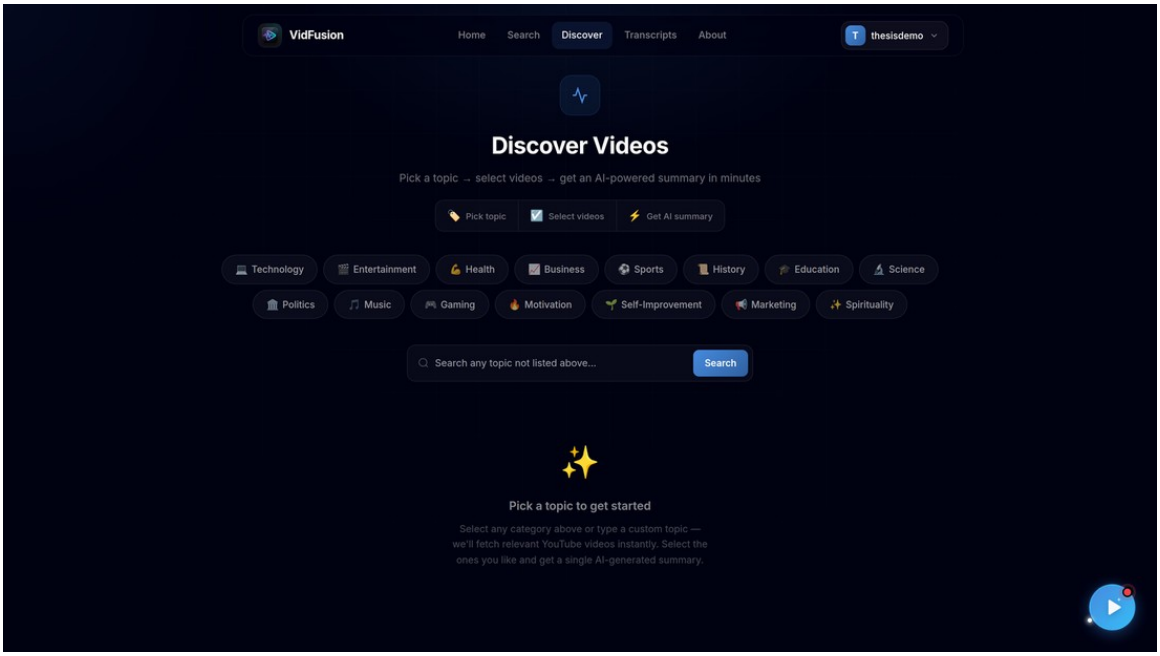


3. Scroll down for more matching videos. Each card can be added to a fusion with Add to Merge, played with Watch, or opened on YouTube.

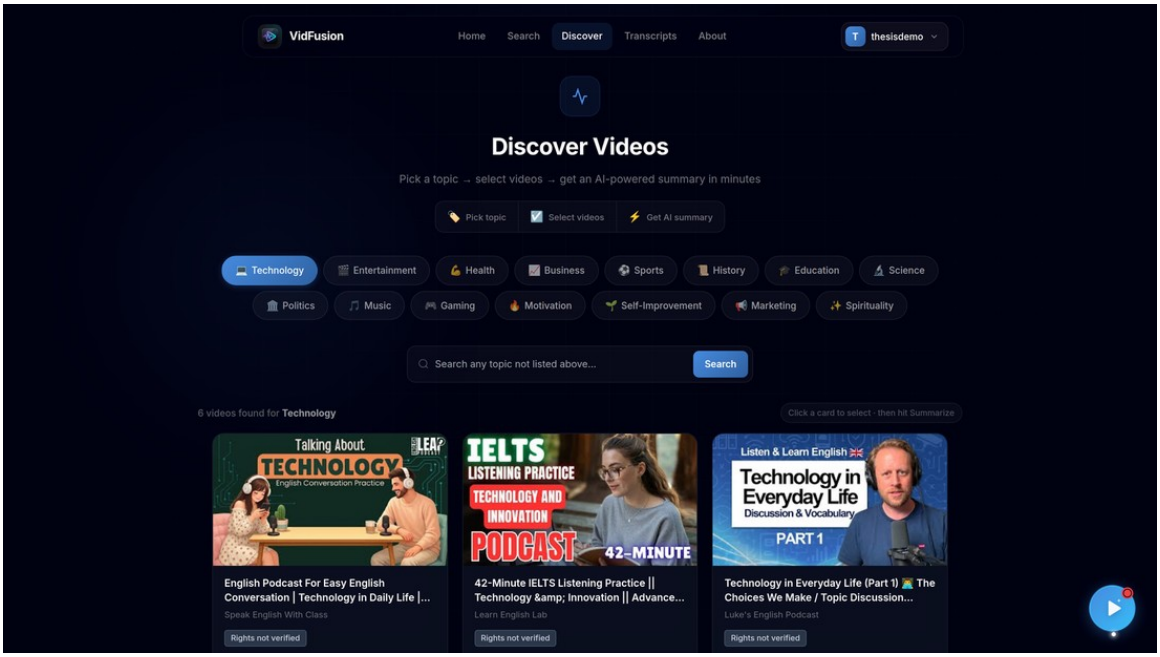


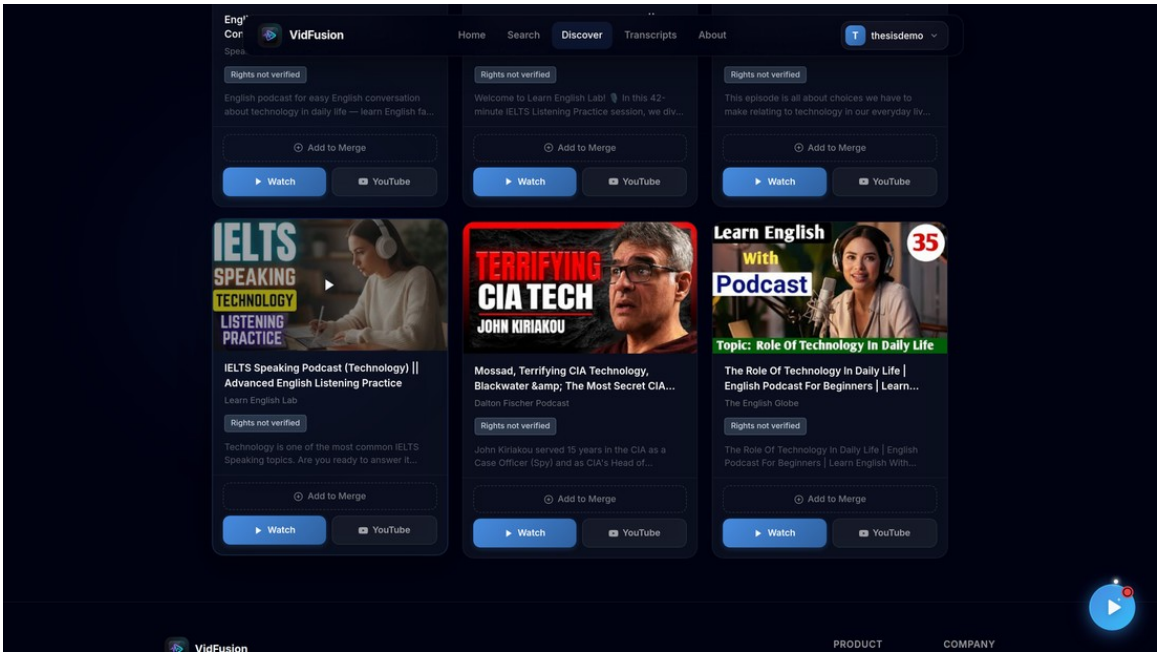
8.4 Discover Videos by Category

1. Open Discover. Pick a category (Technology, Health, Business, Education, Science and so on) or type a custom topic.



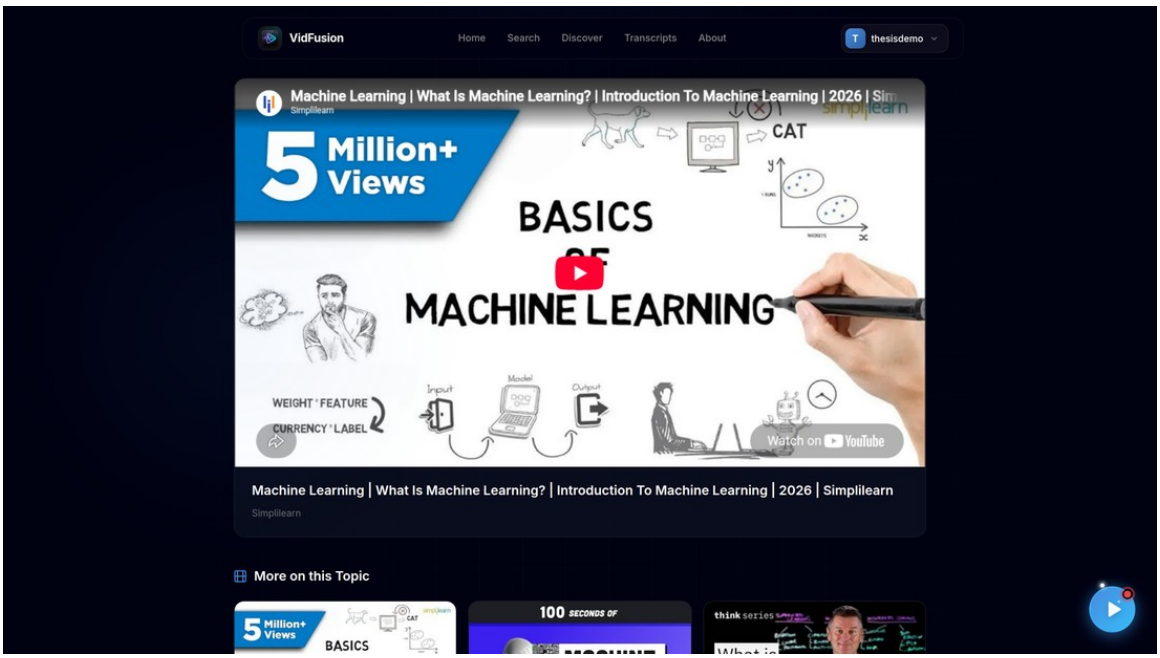
2. VidFusion fetches relevant videos for that category. Select the ones you want, and they can be summarized in the same way as search results.

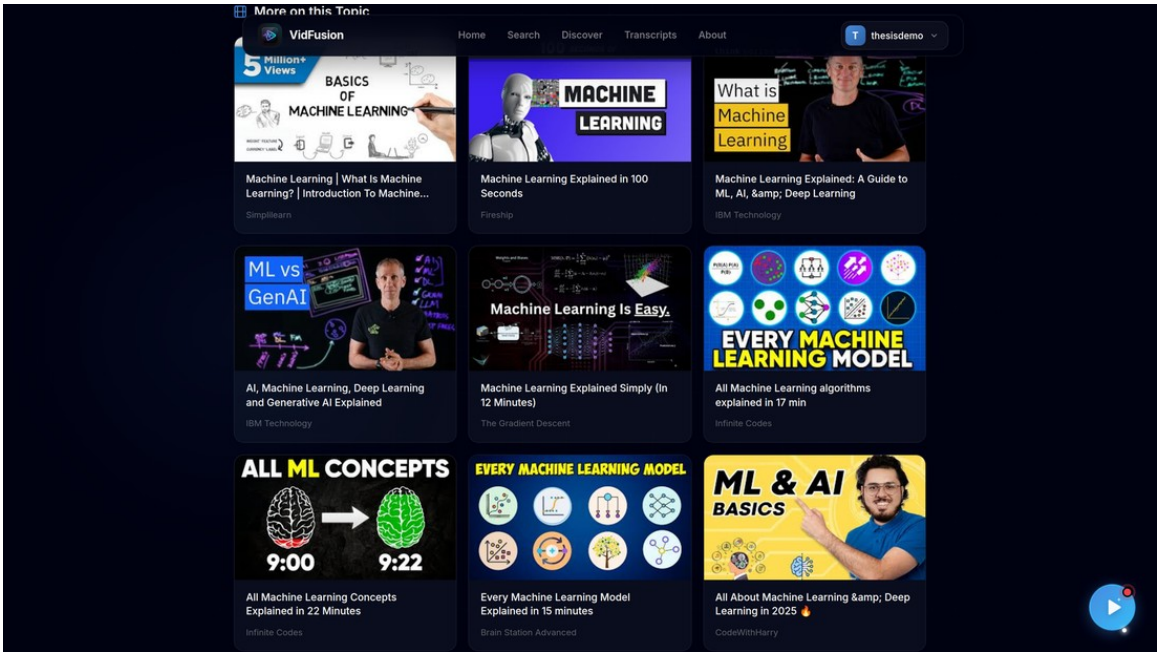




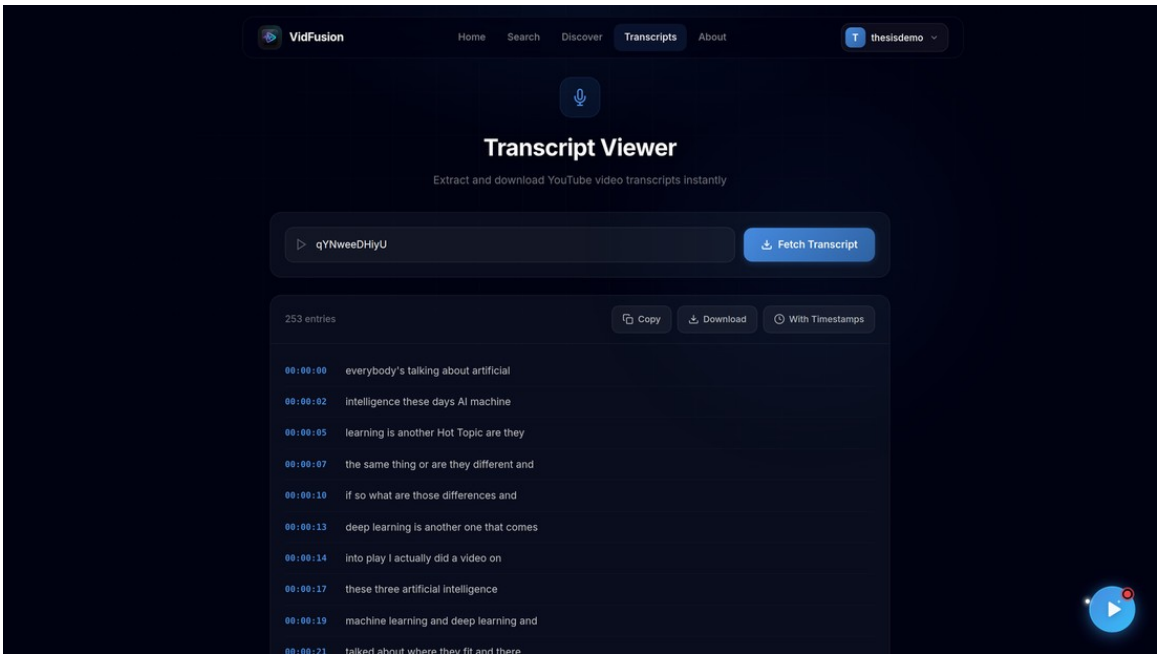
8.5 Watch a Video and View its Transcript

1. Click Watch on any card to play the video inside VidFusion. Related videos appear under More on this Topic.



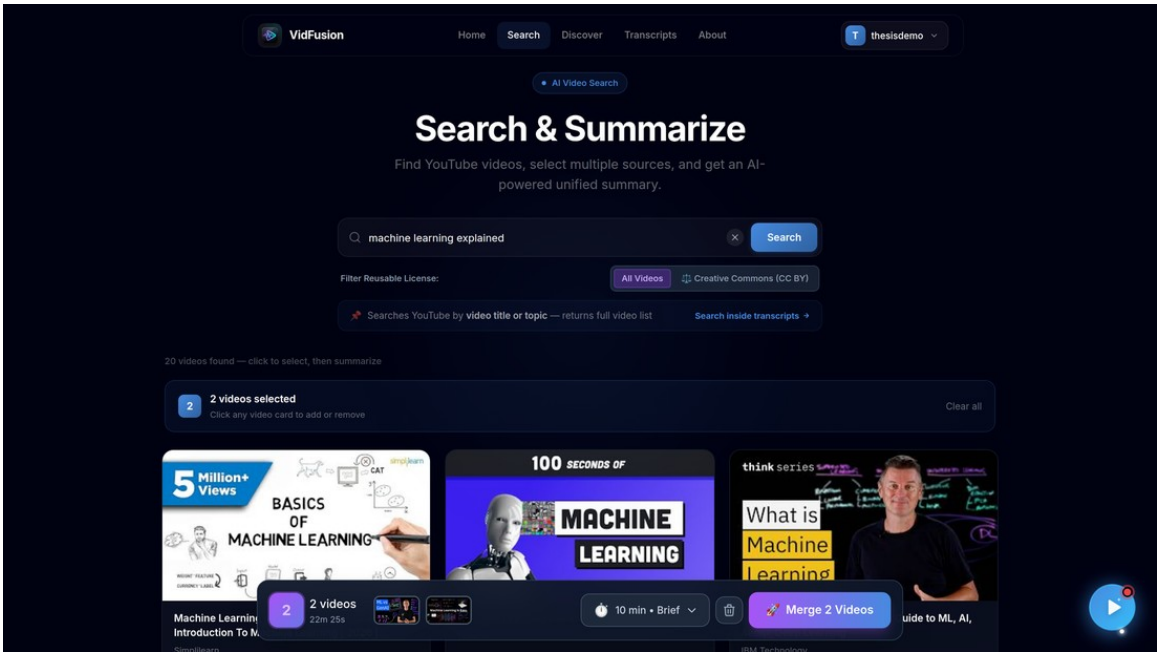


2. Open Transcripts, paste a YouTube video ID or URL, and click Fetch Transcript. The timestamped transcript is shown, and you can Copy it or Download it with or without timestamps.

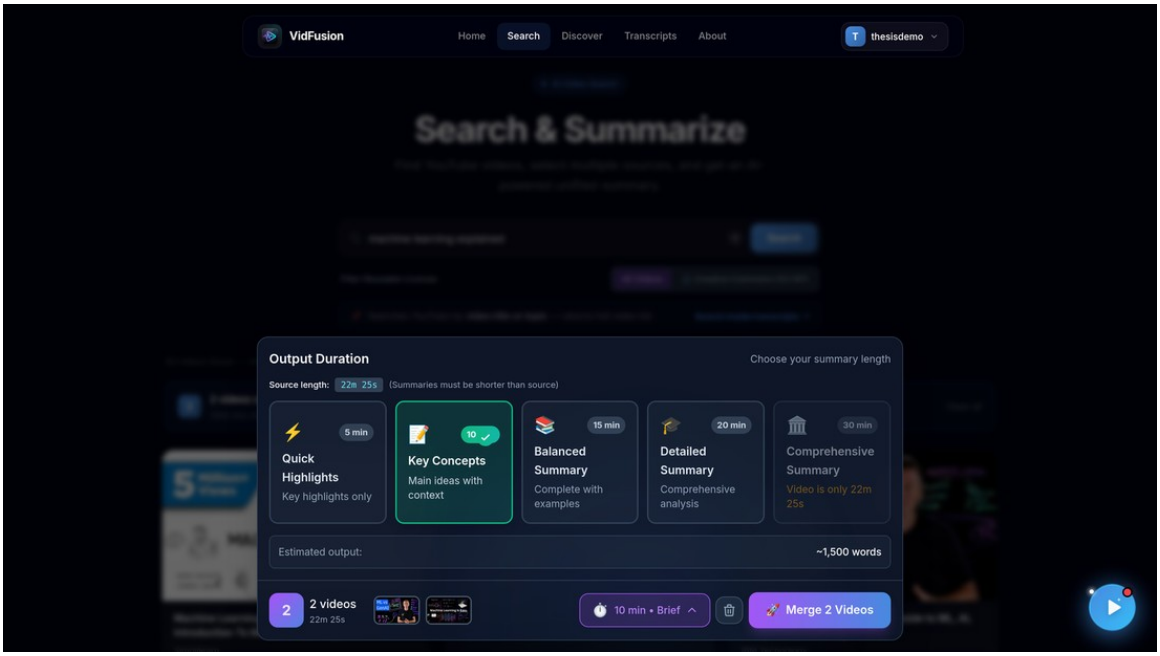


8.6 Create a Video Fusion (Select Videos and Duration)

1. Click Add to Merge on one or more videos (up to five). A selection tray appears at the bottom of the screen showing the number of selected videos, their combined source length and thumbnails.



2. Open the duration menu in the tray and choose an output length: 5 min Quick Highlights, 10 min Key Concepts, 15 min Balanced Summary, 20 min Detailed Summary, or 30 min Comprehensive Summary. Durations longer than the source are disabled, since a summary must be shorter than the source. The estimated output word count is shown below.

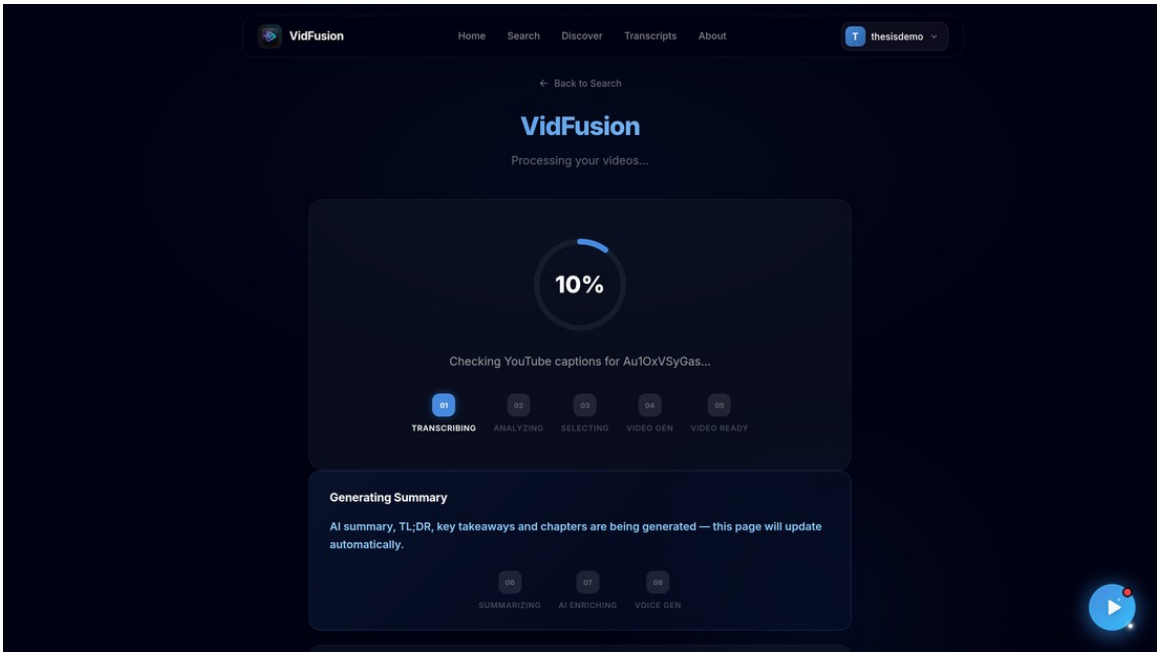


3. Click Merge N Videos. The job is created and you are taken to the processing page.

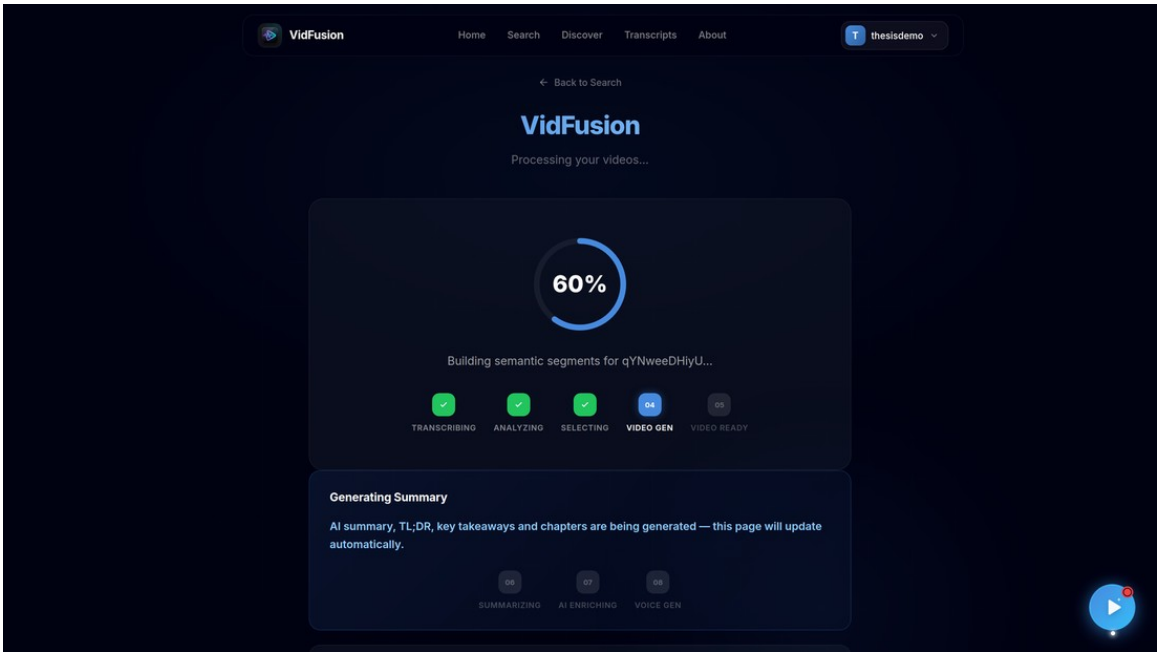
8.7 Monitor Processing Progress

The processing page updates automatically. A progress ring shows the percentage and the current stage message. The pipeline steps are Transcribing, Analyzing, Selecting, Video Gen and Video Ready, followed by Summarizing, AI Enriching and Voice Gen for the text summary.

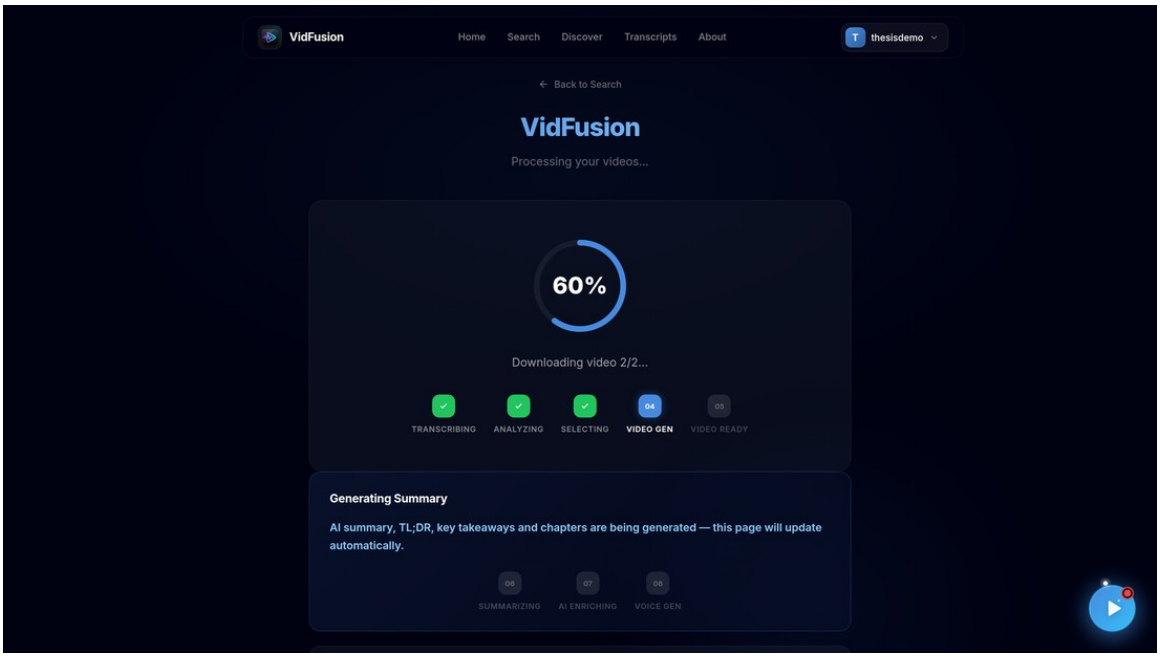
1. Transcribing: VidFusion checks YouTube captions for each source and falls back to Whisper transcription if none are available.



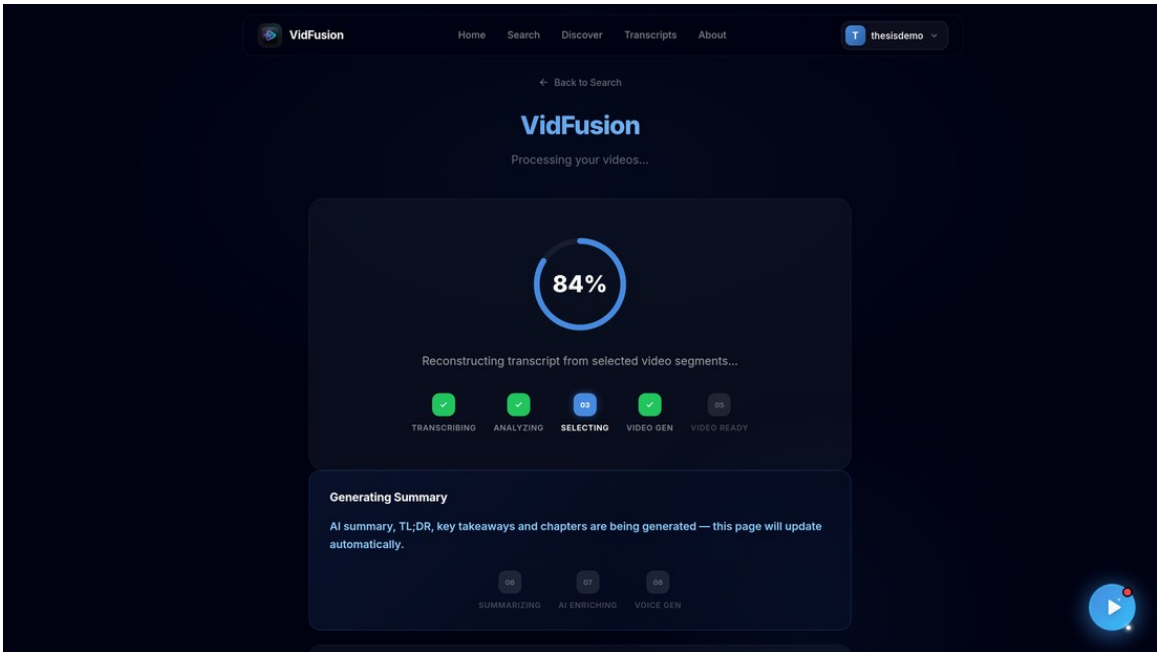
2. Analyzing: transcripts are split into semantic segments (complete thoughts) using sentence-embedding similarity.



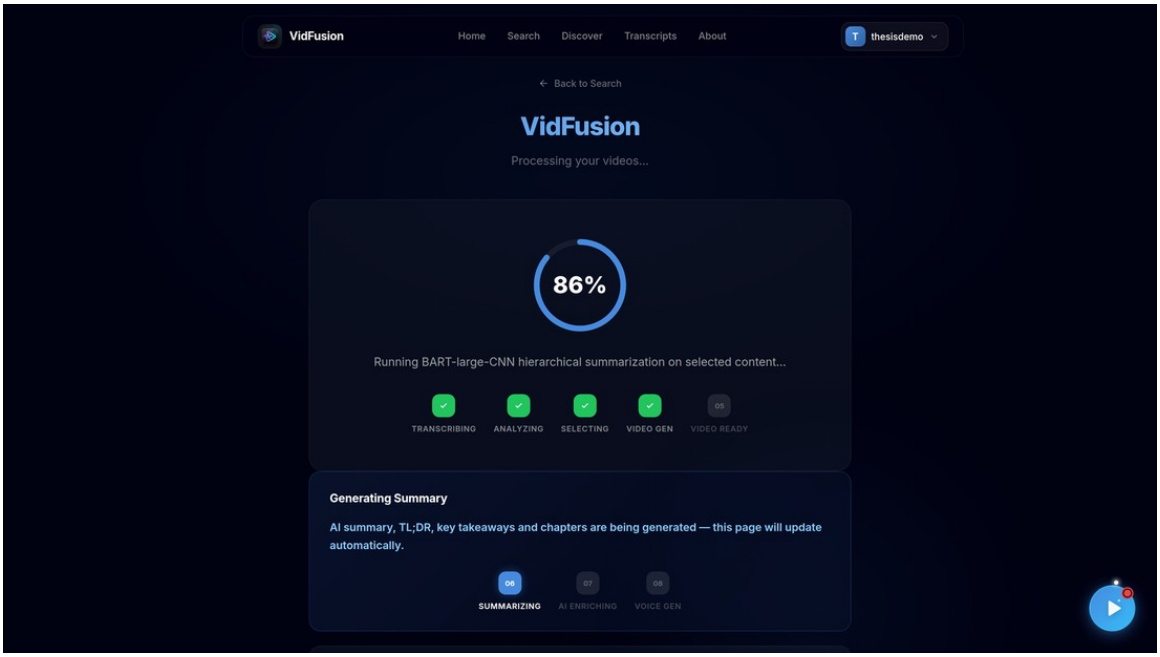
3. Video Gen: the source videos are downloaded so the selected clips can be cut.



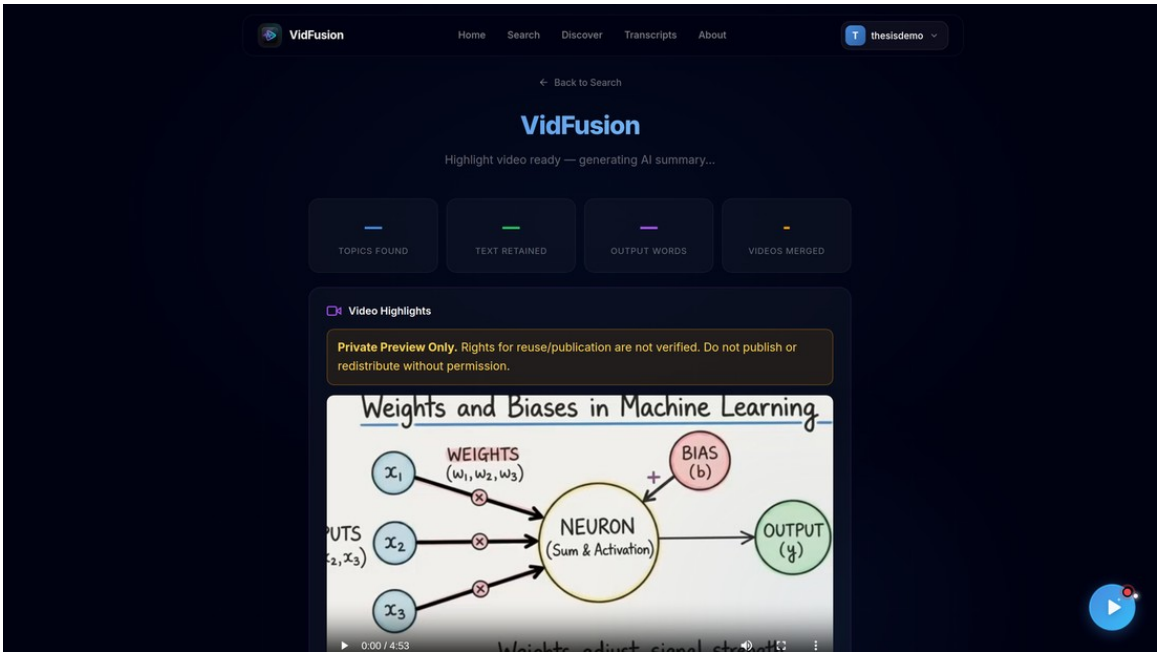
4. Selecting: the transcript of the selected segments is reconstructed for summarization.



5. Summarizing: BART-large-CNN runs hierarchical summarization on the selected content.

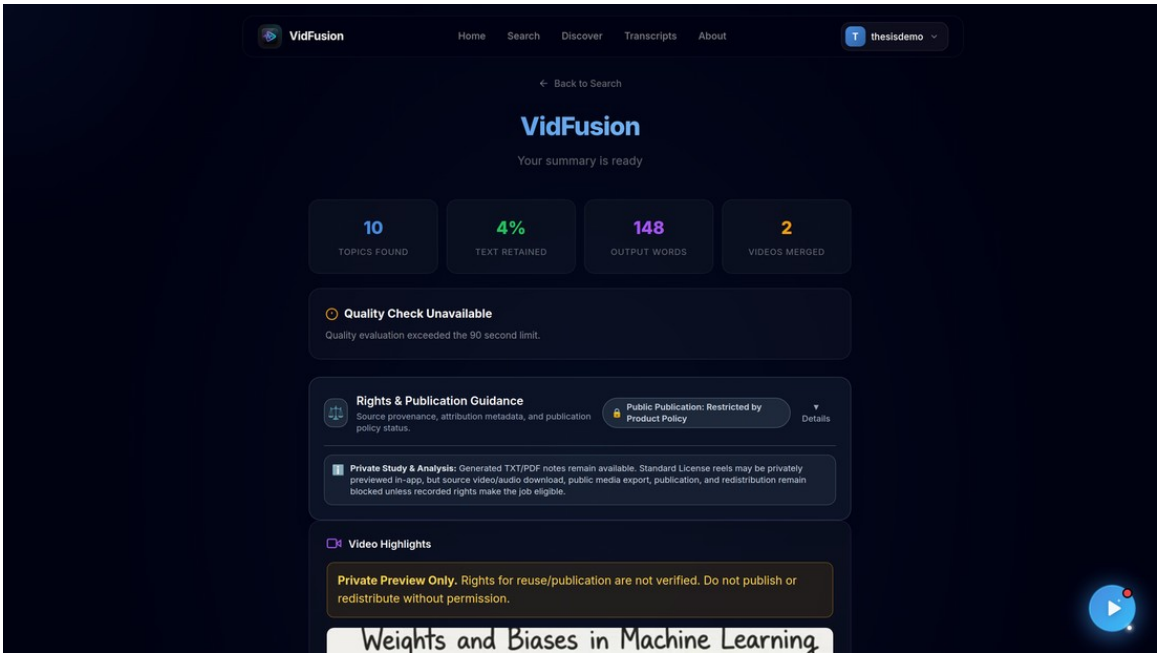


6. When the highlight video is assembled it becomes playable immediately, while the AI summary is still being generated.



8.8 View the Fusion Result

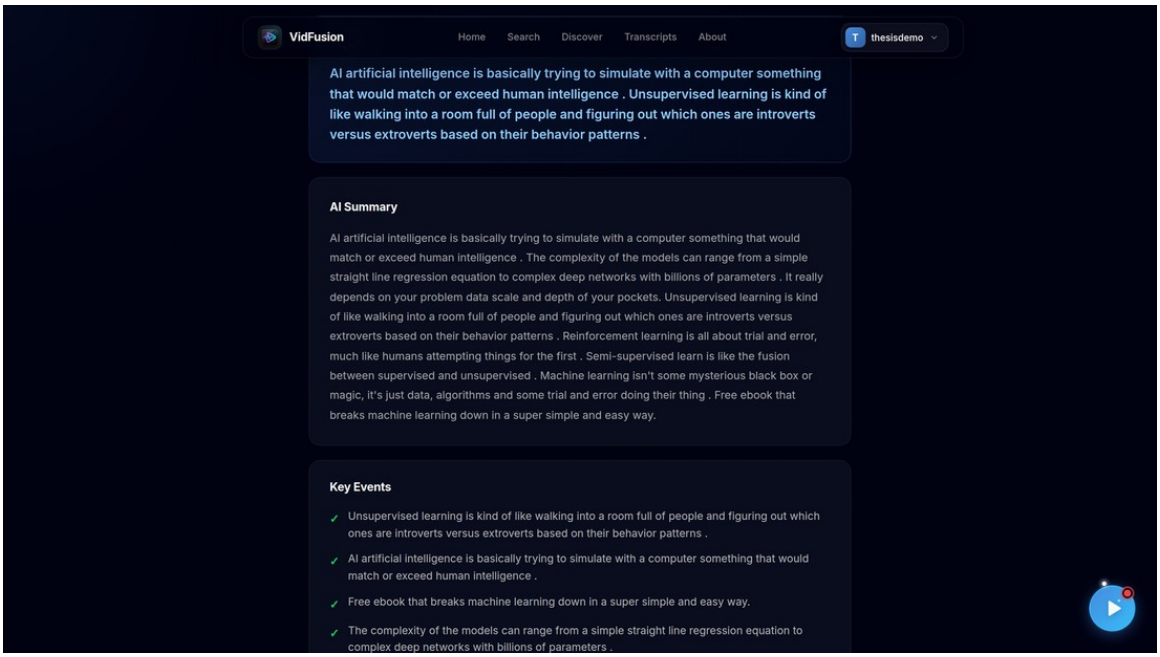
1. When processing finishes, the page reads "Your summary is ready". Summary cards show the number of topics found, the percentage of source text retained, the output word count and the number of videos merged. In this run the optional quality check timed out at its 90-second limit and is reported as unavailable rather than hidden.



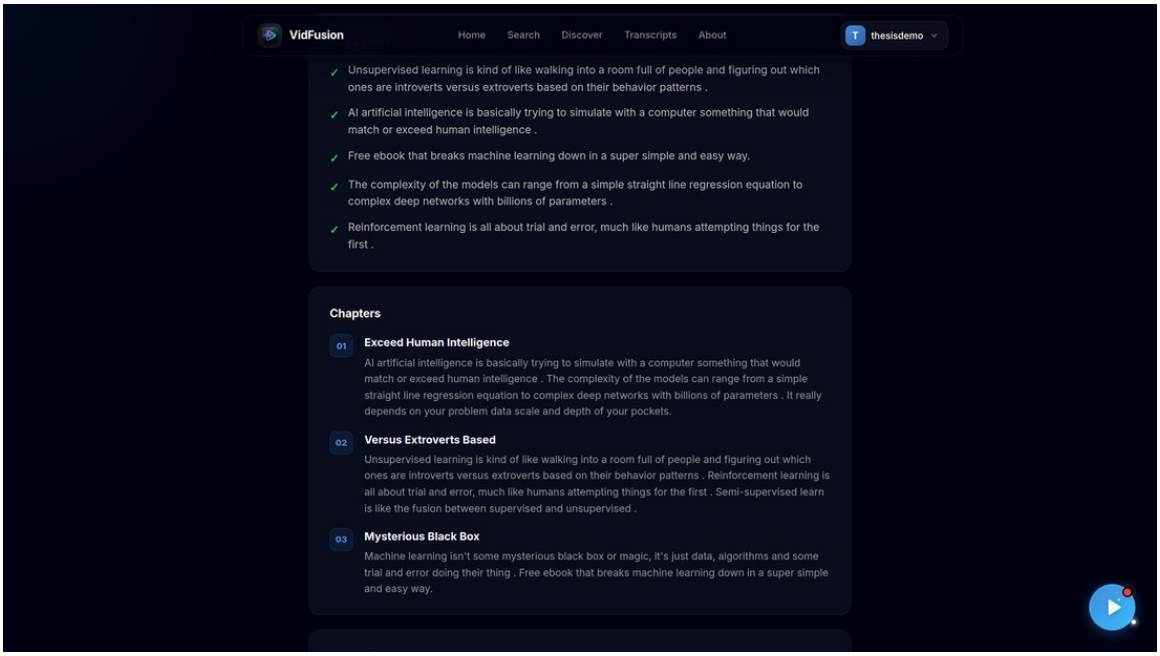
2. Video Highlights plays the fused highlight video, built from clips of both source videos in the original speakers' voices. It is marked "Private Preview Only" because reuse rights for the sources are not verified.



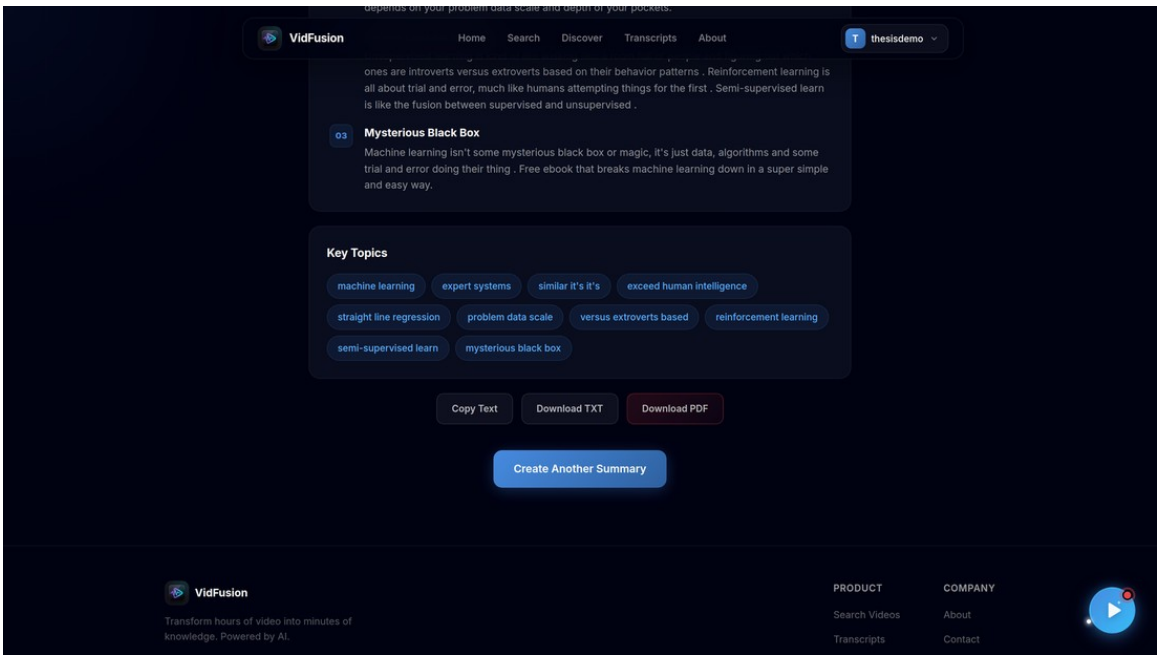
3. Scroll down to read the TL;DR and the full AI Summary.



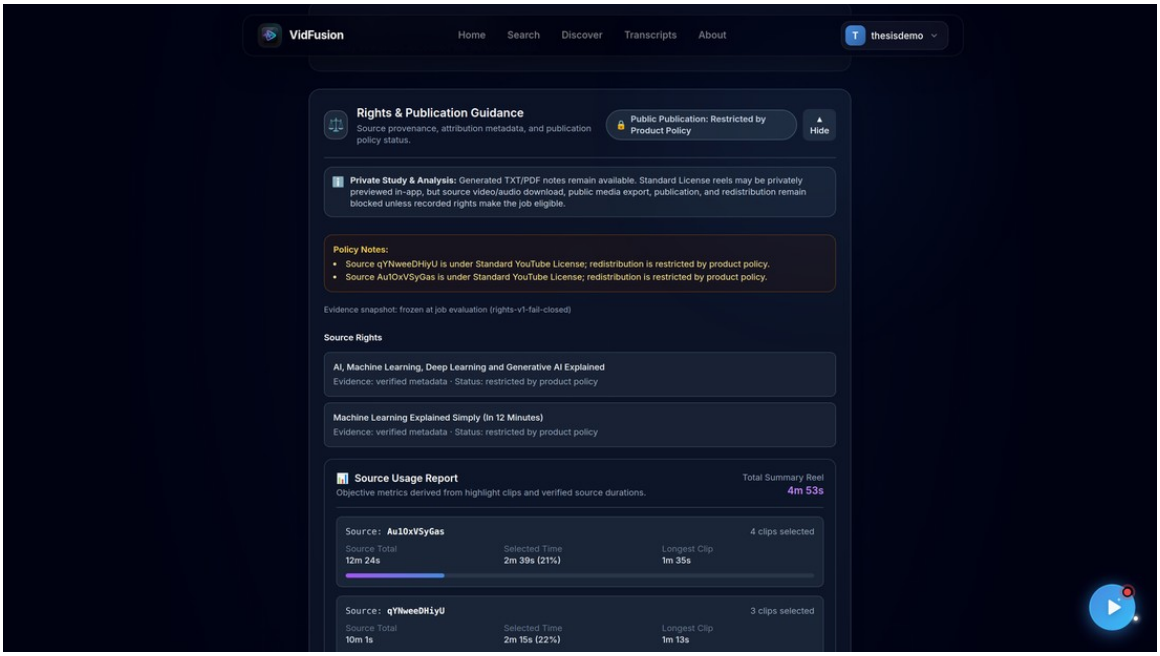
4. Key Events lists the most important points, and Chapters groups the summary into titled sections.



5. Key Topics shows the extracted concepts. Use Copy Text, Download TXT or Download PDF to export the summary, or Create Another Summary to start again.



6. Rights & Publication Guidance explains each source's license status and why public publication is restricted. Click Details to see the per-source policy notes.



7. The downloaded PDF contains the TL;DR, summary, key events and chapters.

AI Video Summary

TL;DR

AI artificial intelligence is basically trying to simulate with a computer something that would match or exceed human intelligence . Unsupervised learning is kind of like walking into a room full of people and figuring out which ones are introverts versus extroverts based on their behavior patterns .

Summary

AI artificial intelligence is basically trying to simulate with a computer something that would match or exceed human intelligence . The complexity of the models can range from a simple straight line regression equation to complex deep networks with billions of parameters . It really depends on your problem data scale and depth of your pockets. Unsupervised learning is kind of like walking into a room full of people and figuring out which ones are introverts versus extroverts based on their behavior patterns . Reinforcement learning is all about trial and error, much like humans attempting things for the first . Semi-supervised learn is like the fusion between supervised and unsupervised . Machine learning isn't some mysterious black box or magic, it's just data, algorithms and some trial and error doing their thing . Free ebook that breaks machine learning down in a super simple and easy way.

Key Events

1. Unsupervised learning is kind of like walking into a room full of people and figuring out which ones are introverts versus extroverts based on their behavior patterns .
2. AI artificial intelligence is basically trying to simulate with a computer something that would match or exceed human intelligence .
3. Free ebook that breaks machine learning down in a super simple and easy way.
4. The complexity of the models can range from a simple straight line regression equation to complex deep networks with billions of parameters .
5. Reinforcement learning is all about trial and error, much like humans attempting things for the first .

Chapters

1. Exceed Human Intelligence

AI artificial intelligence is basically trying to simulate with a computer something that would match or exceed human intelligence . The complexity of the models can range from a simple straight line regression equation to complex deep networks with billions of parameters . It really depends on your problem data scale and depth of your pockets.

2. V

Unsupervised learning is kind of like walking into a room full of people and figuring out which ones are introverts versus extroverts based on their behavior patterns . Reinforcement learning is all about trial and error, much like humans attempting things for the first . Semi-supervised learn is like the fusion between supervised and unsupervised .

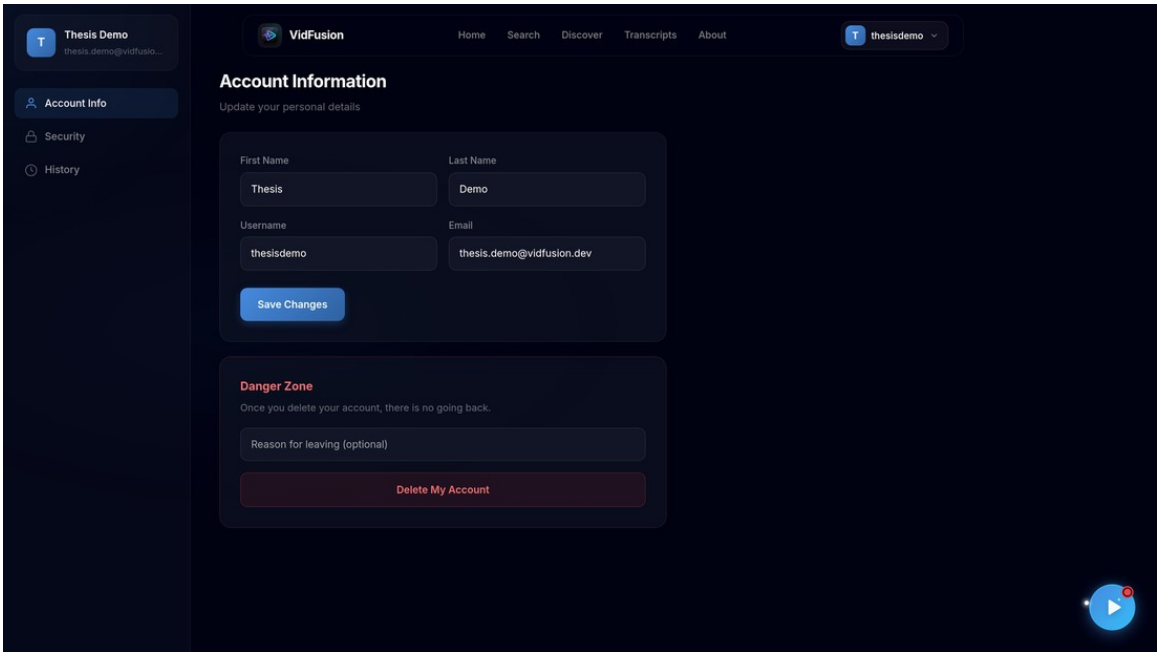
3. M

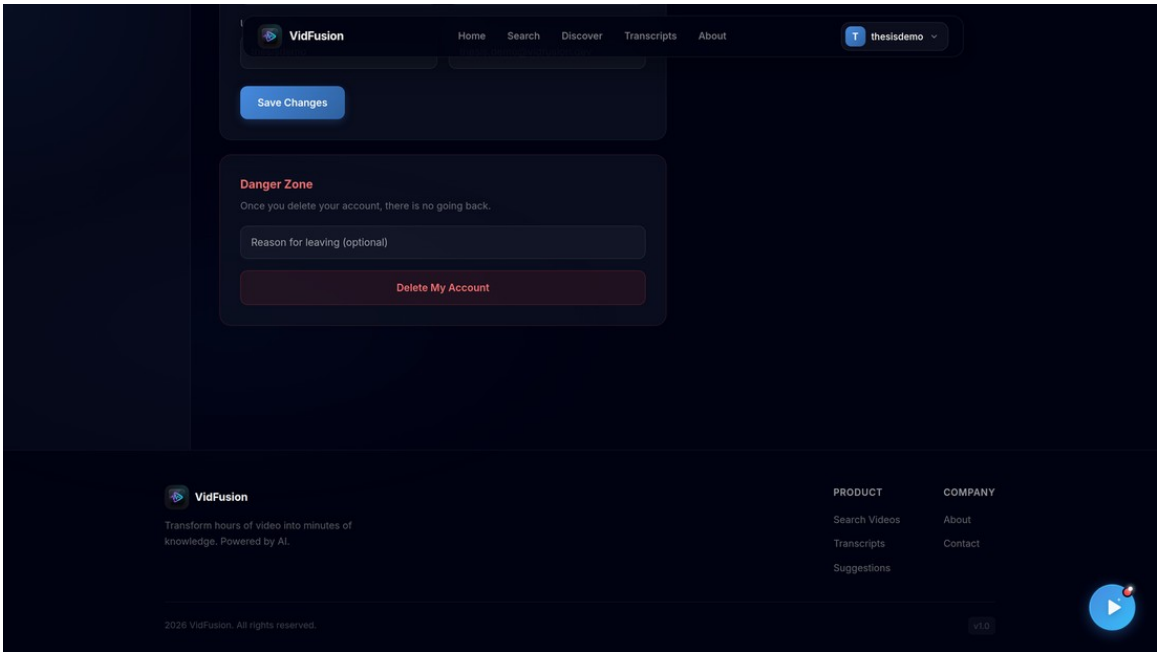
Machine learning isn't some mysterious black box or magic, it's just data, algorithms and some trial and error doing their thing . Free ebook that breaks machine learning down in a super simple and easy way.

Generated by VidFusion

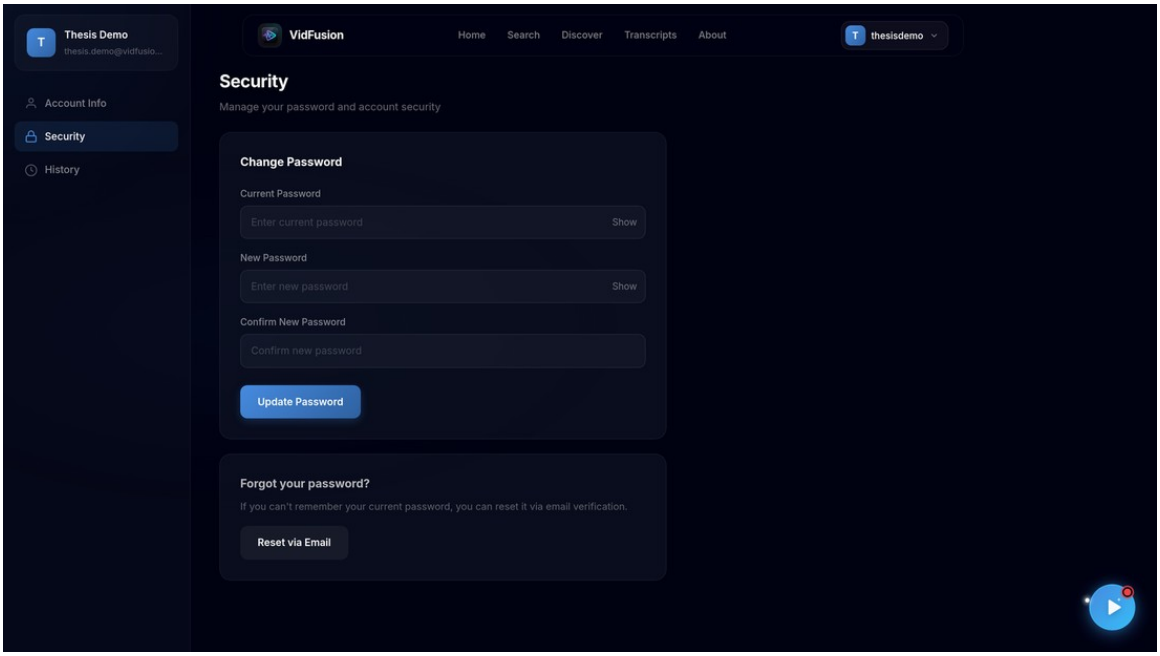
8.9 Manage Your Account

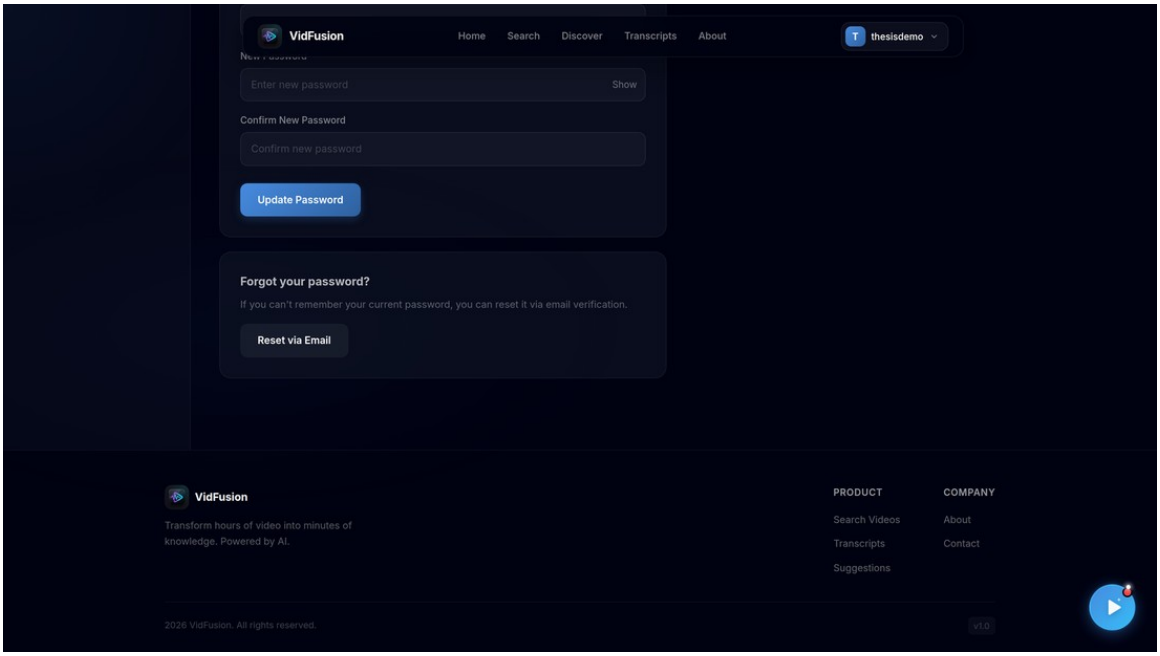
1. Open the profile menu and choose Account Info to edit your first name, last name, username and email. The Danger Zone lets you permanently delete your account.



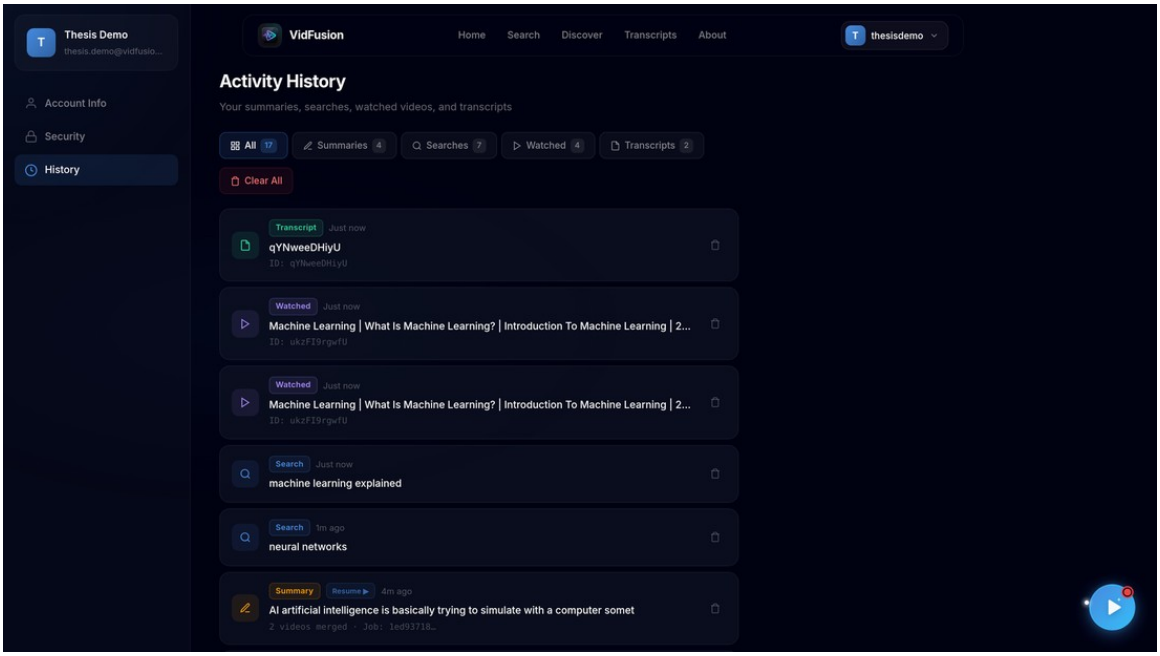


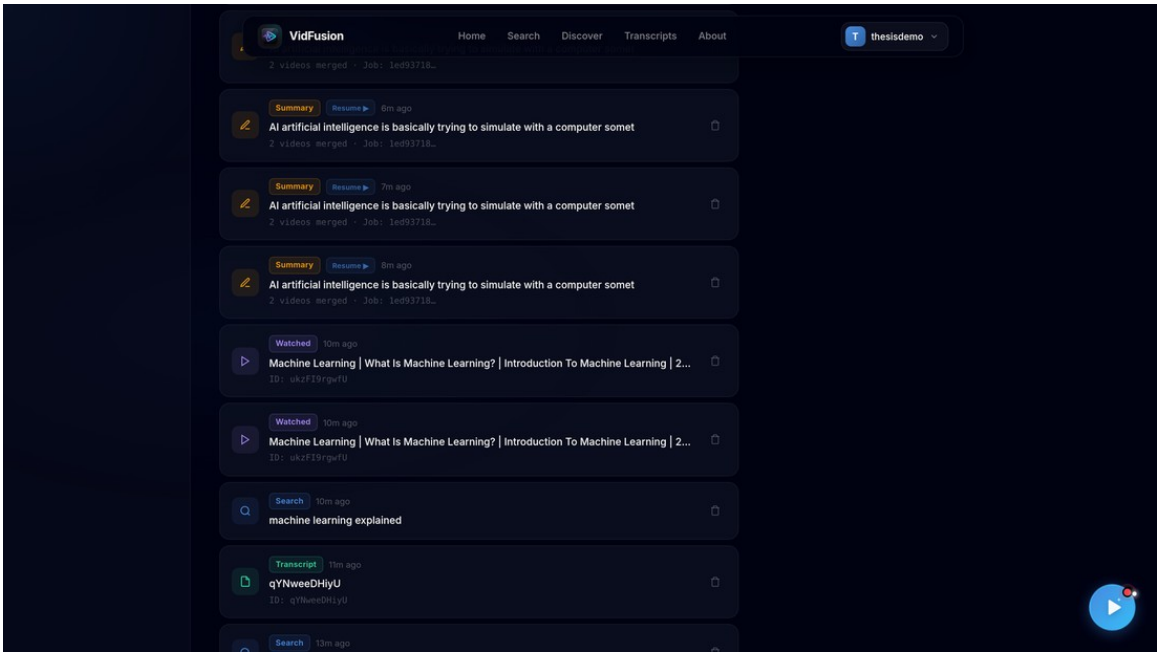
2. Security lets you change your password, or reset it by email if you have forgotten it.





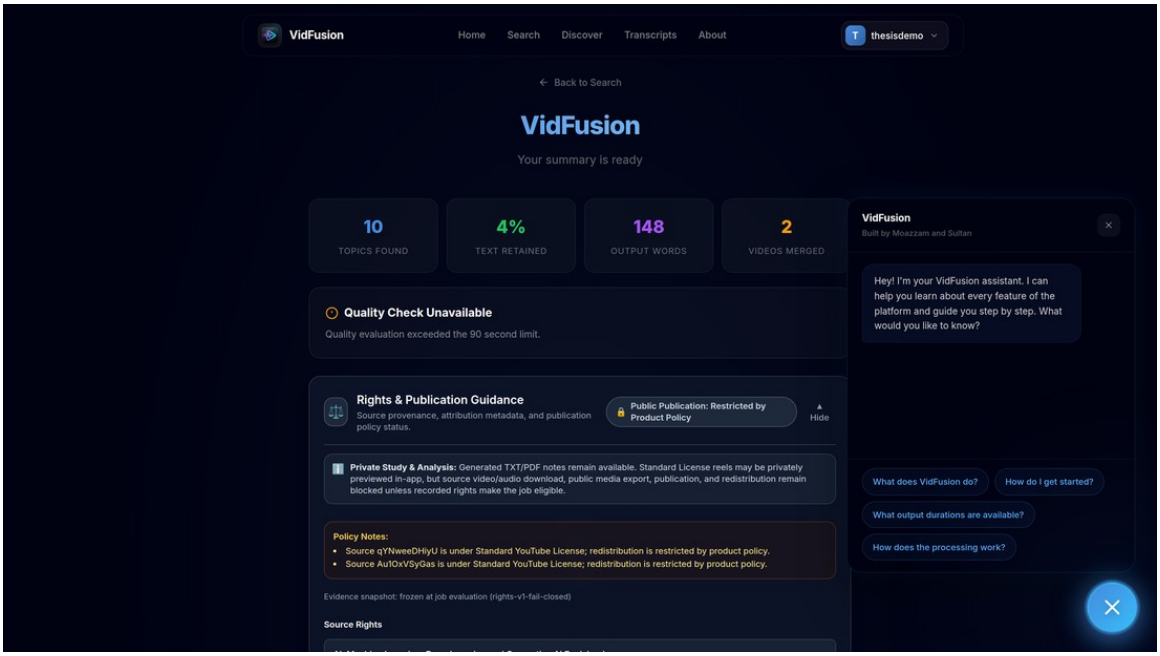
3. History lists your summaries, searches, watched videos and viewed transcripts. Filter by type with the tabs, delete single entries, or use Clear All.



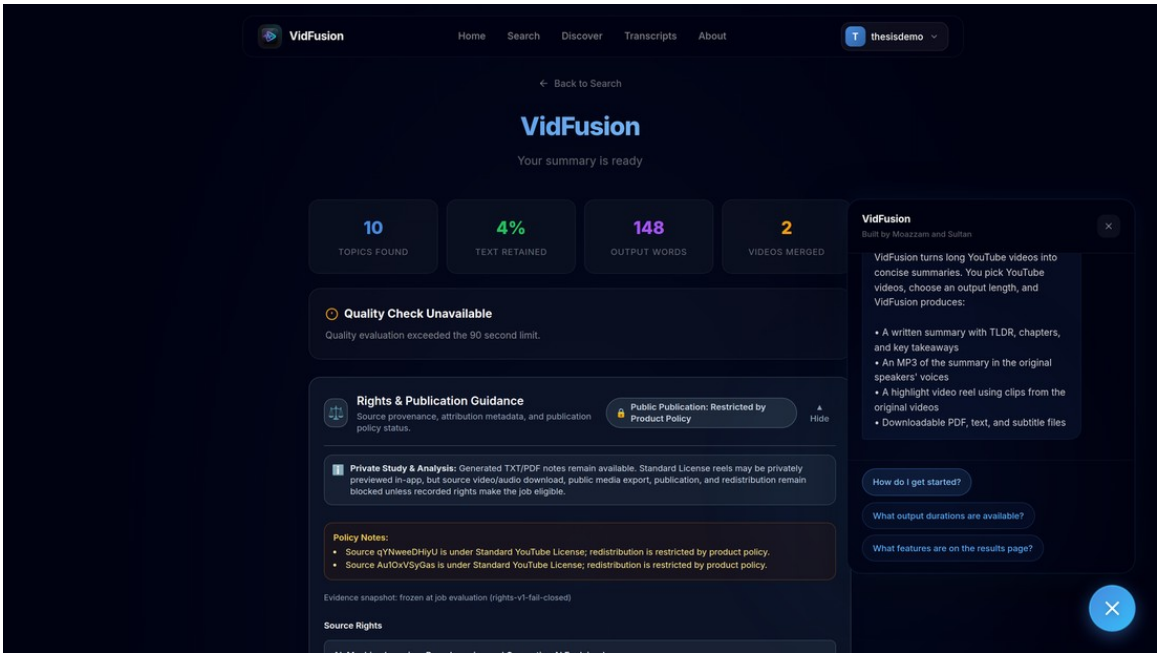


8.10 Use the Help Assistant

1. Click the floating play button in the bottom-right corner on any page to open the VidFusion assistant.

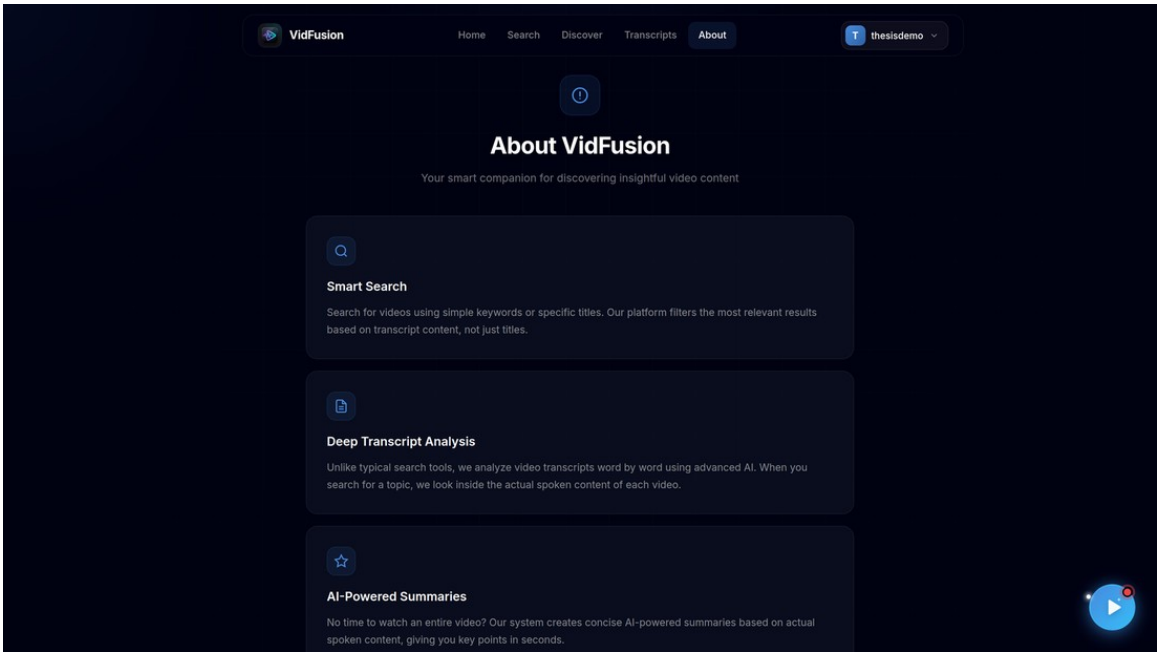


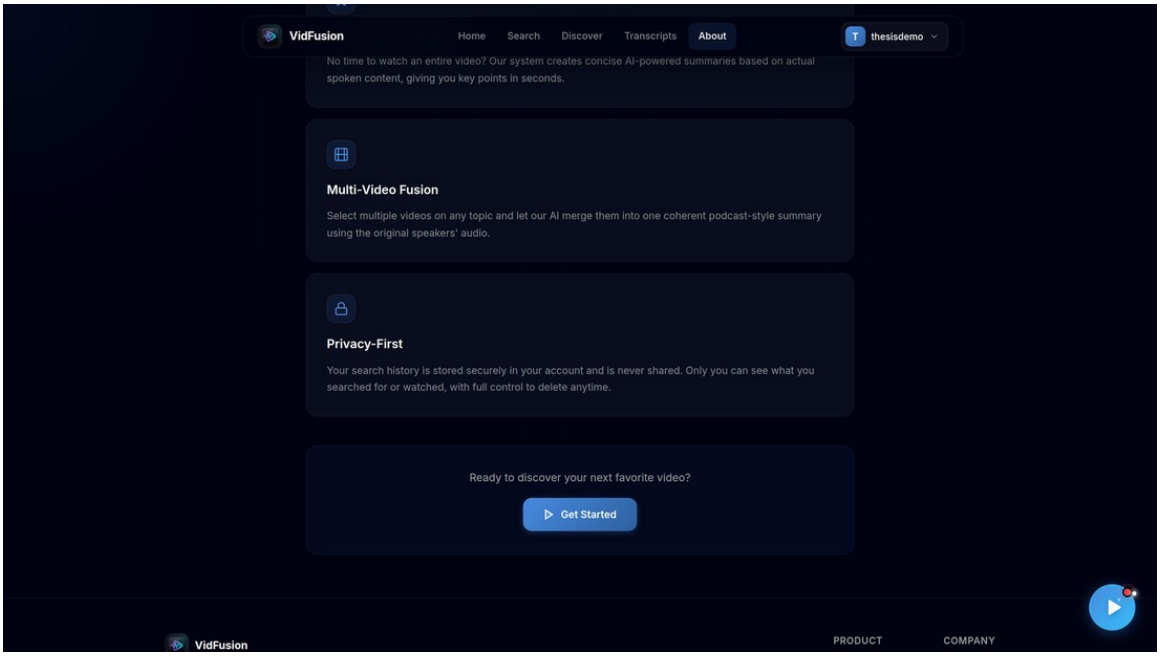
2. Choose a suggested question (for example "What does VidFusion do?") to get an answer with follow-up questions.



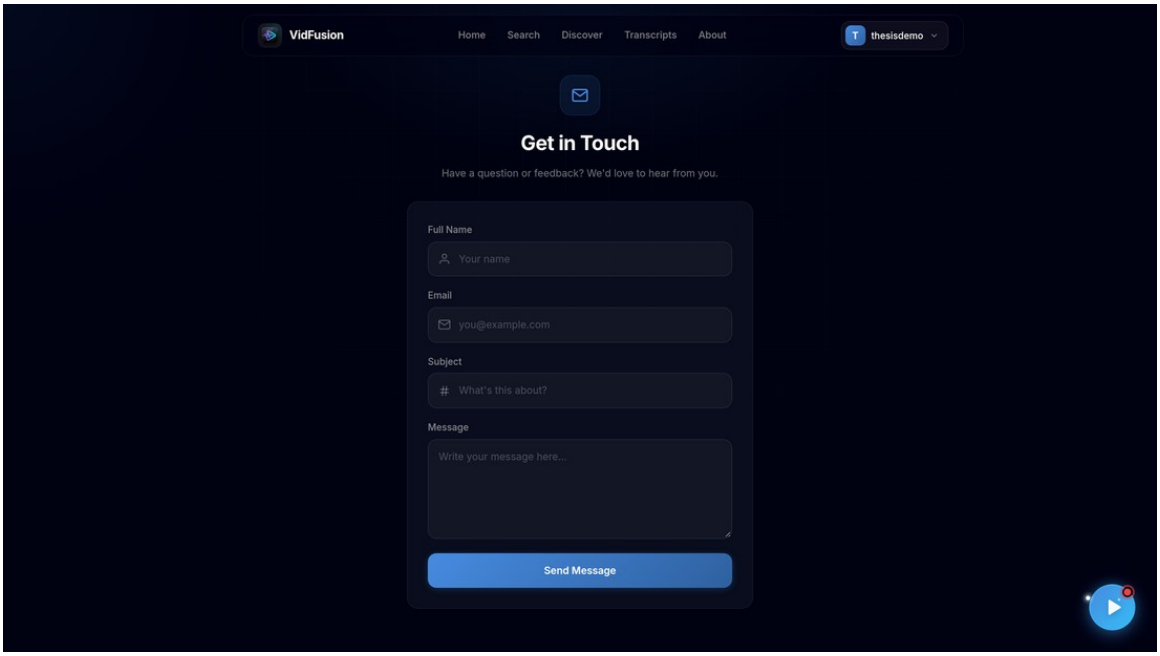
8.11 About and Contact

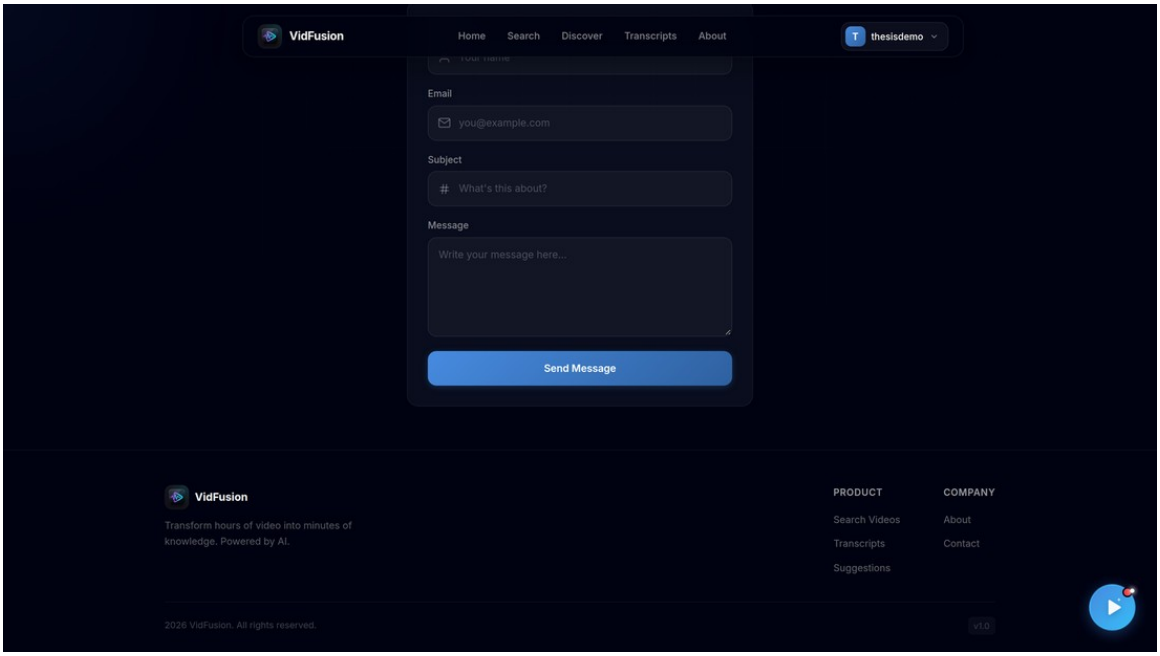
1. About describes the platform's features: Smart Search, Deep Transcript Analysis, AI-Powered Summaries, Multi-Video Fusion and Privacy-First history.





2. Contact provides a form for questions and feedback.

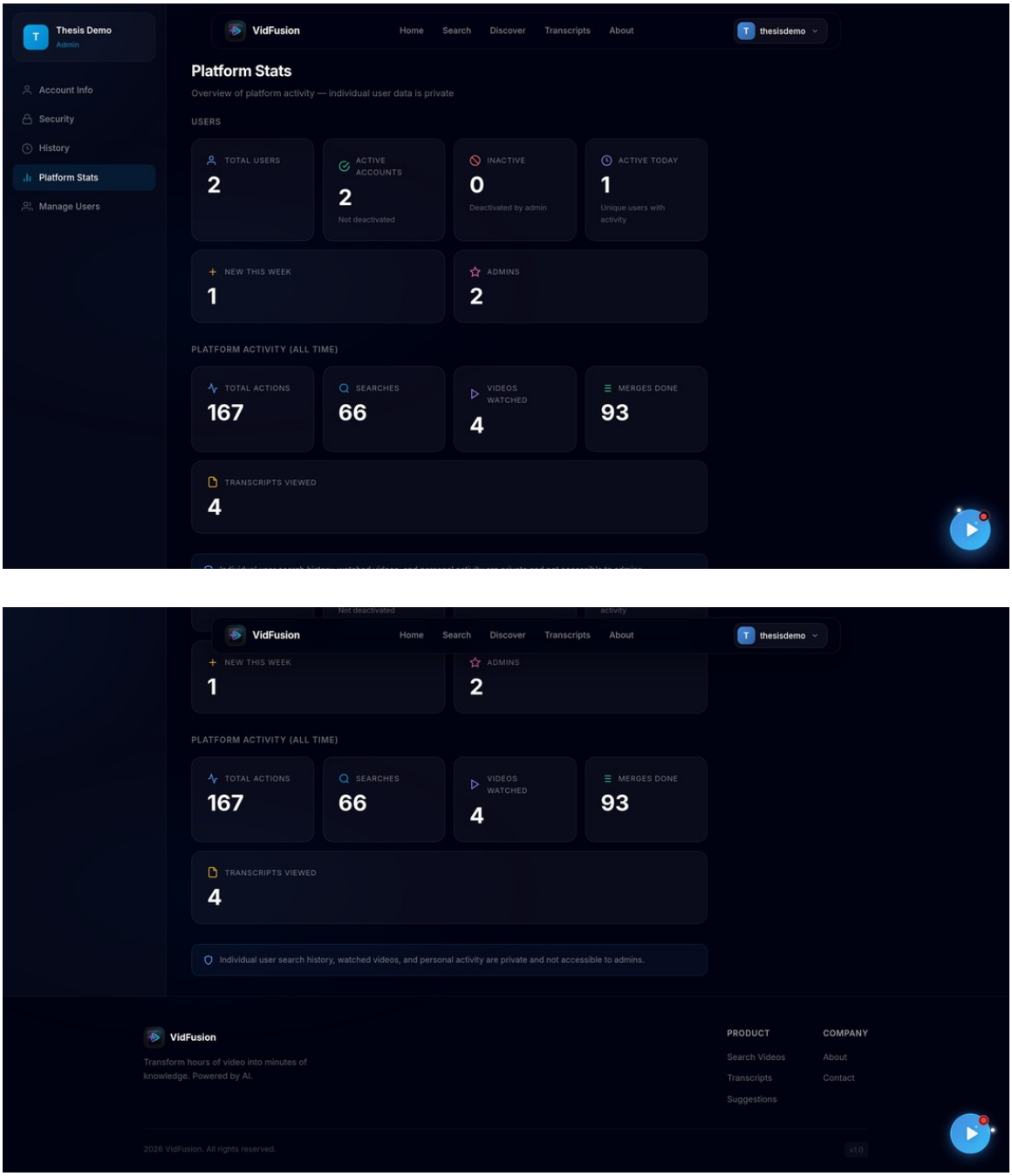




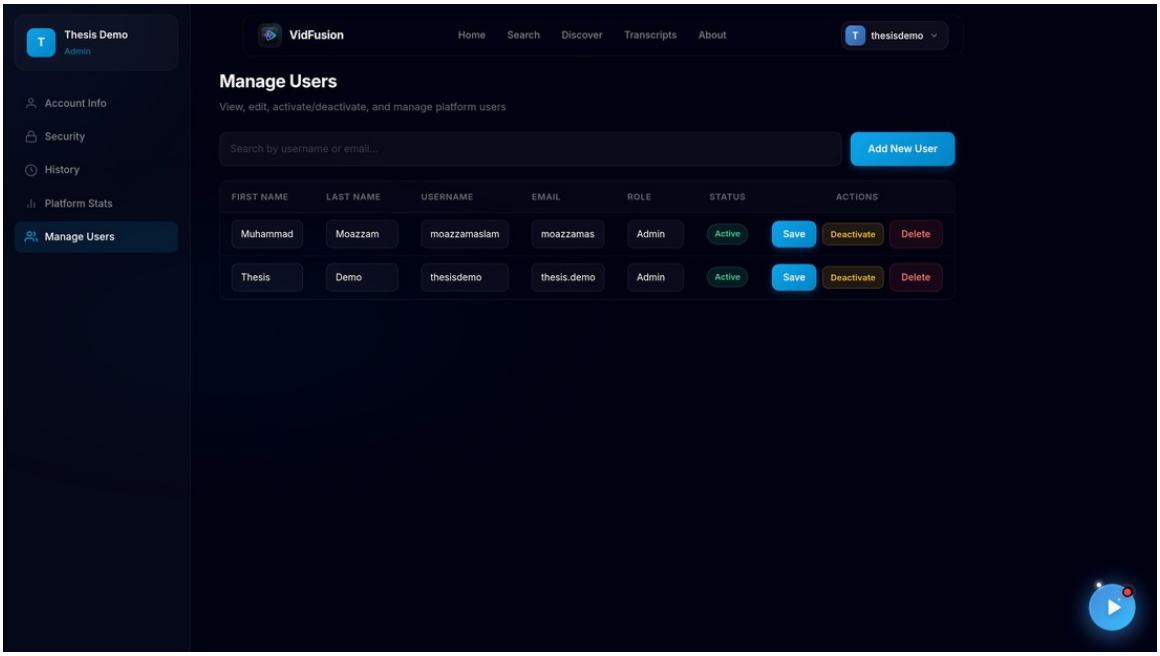
8.12 Administrator Panel

Administrator accounts have extra pages in the profile sidebar.

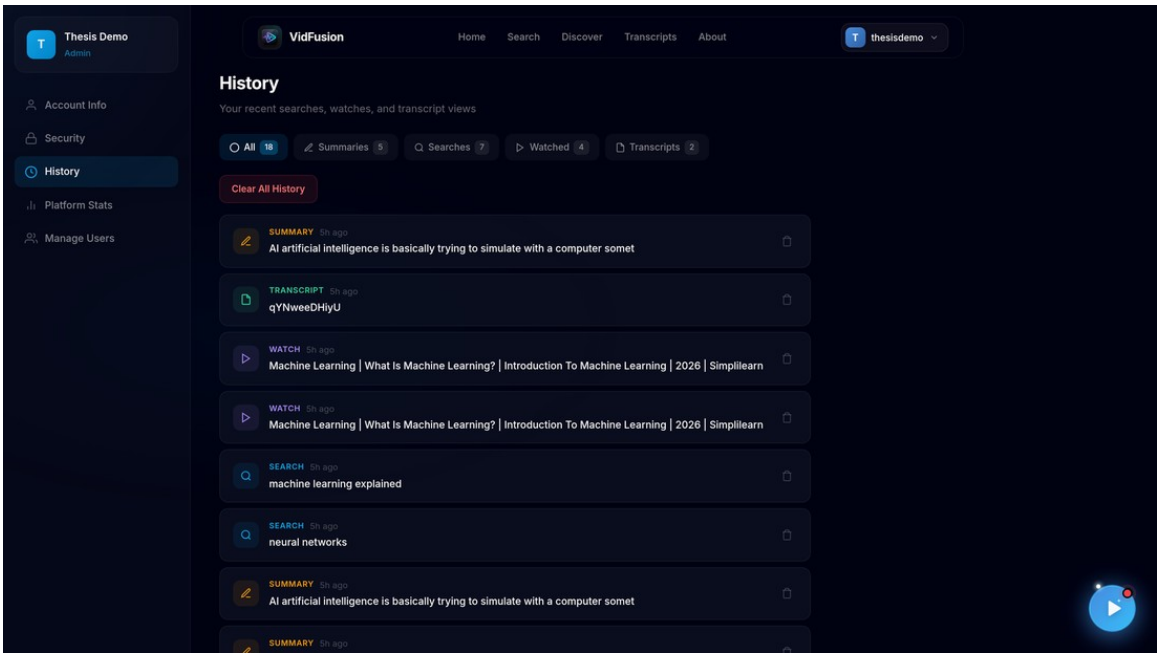
1. Platform Stats shows user counts (total, active, inactive, active today, new this week, admins) and platform activity (total actions, searches, videos watched, merges done, transcripts viewed). Individual users' activity is not visible to admins.

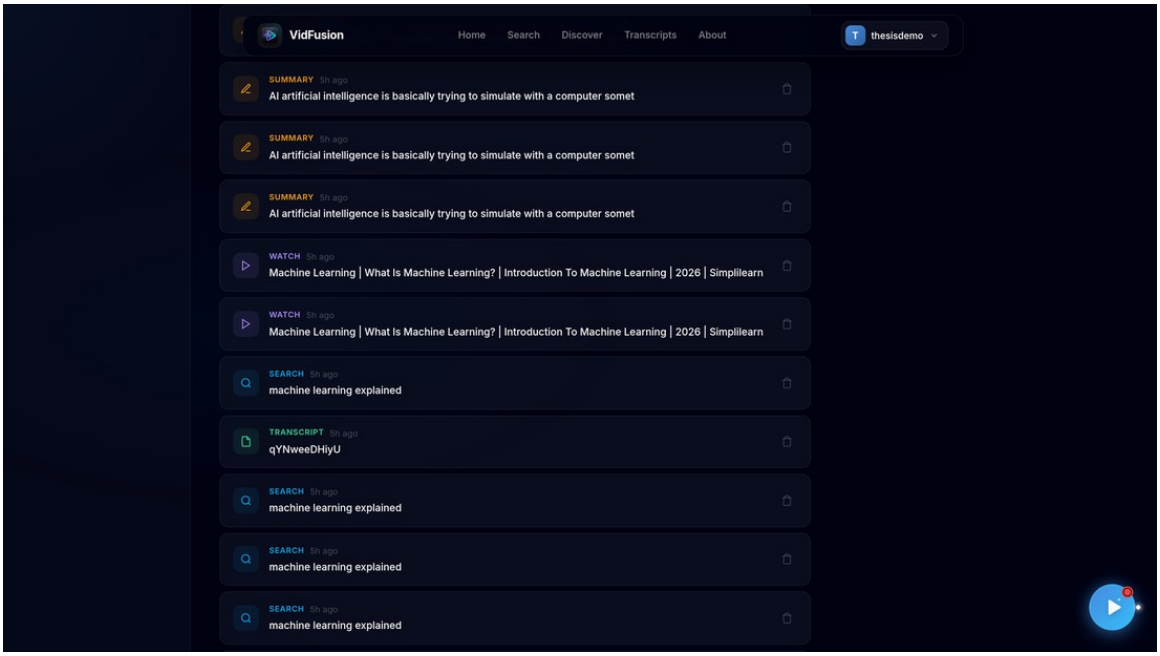


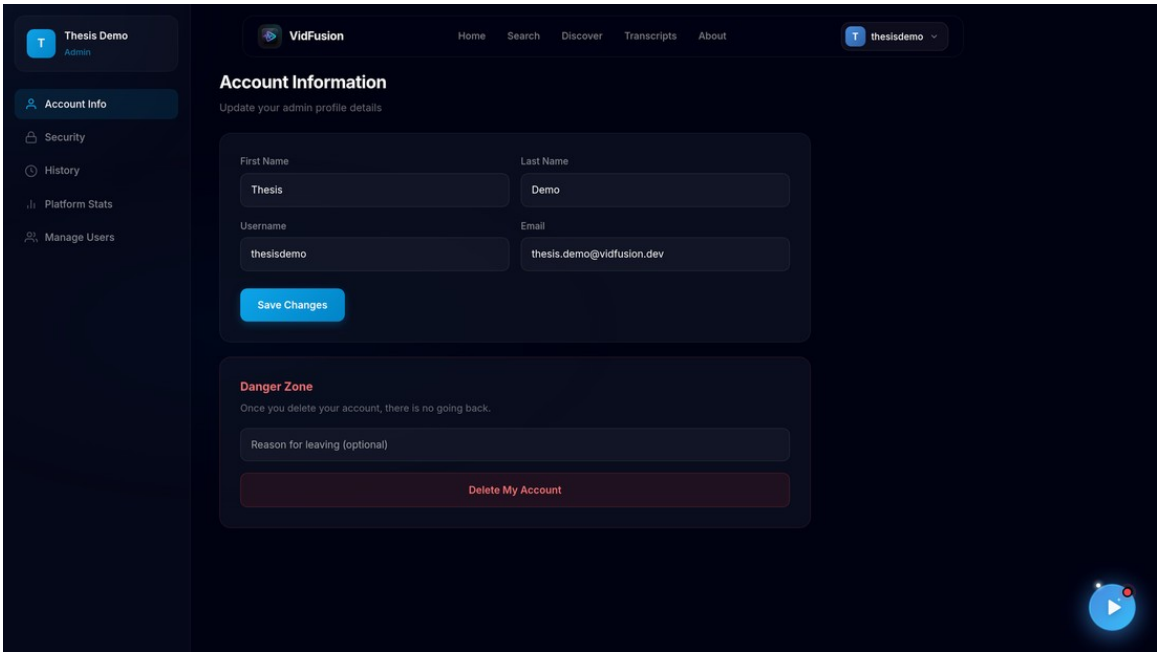
2. Manage Users lets an admin search, edit, deactivate or delete users and add new ones with Add New User.



3. Admins also have their own History and Account Info pages.




VidFusion
Home
Search
Discover
Transcripts
About
thesisdemo
SUMMARY 5h ago
AI artificial intelligence is basically trying to simulate with a computer somet
SUMMARY 5h ago
AI artificial intelligence is basically trying to simulate with a computer somet
SUMMARY 5h ago
AI artificial intelligence is basically trying to simulate with a computer somet
WATCH 5h ago
Machine Learning | What Is Machine Learning? | Introduction To Machine Learning | 2026 | Simplilearn
WATCH 5h ago
Machine Learning | What Is Machine Learning? | Introduction To Machine Learning | 2026 | Simplilearn
SEARCH 5h ago
machine learning explained
TRANSCRIPT 5h ago
qYNweeDHiyU
SEARCH 5h ago
machine learning explained
SEARCH 5h ago
machine learning explained
SEARCH 5h ago
machine learning explained


Thesis Demo
Admin
Account Info
Security
History
Platform Stats
Manage Users
VidFusion
Home
Search
Discover
Transcripts
About
thesisdemo
Account Information
Update your admin profile details
First Name
Thesis
Last Name
Demo
Username
thesisdemo
Email
thesis.demo@vidfusion.dev
Save Changes
Danger Zone
Once you delete your account, there is no going back.
Reason for leaving (optional)
Delete My Account

CHAPTER 9:
LESSONS LEARNT AND FUTURE WORK

9.1 Lessons Learnt

A score with no memory reproduces whatever bias its single input has. The first version of the selection pipeline sorted candidate clips by similarity-to-summary and took them in order until the time budget ran out. Measured directly on this project's own completed jobs rather than suspected from reading the code, this put as much as 80-100% of a job's selected material in the first half of the source video, and in one case the pipeline's own three separate places that each added padding to the running total (the selector, the merge-formatting step, and a later "fit to duration" step) compounded into a budget overshoot of 352% on one job and an undershoot to 8% of the requested budget on another. The fix was not a bigger or better similarity score; it was making the gain from selecting a segment explicitly diminish as similar material was already selected (submodularity), and separately rewarding under-represented regions of the timeline as a concave function rather than a flat bonus. The lesson generalised: a score intended to rank many similar candidates needs a term for "have I already picked something like this", or its ranking will reproduce whatever structural bias its underlying signal has — in this case, that an early segment introducing a topic looks similar to almost any summary of that topic.

A length check is not a quality check. The original summarisation step opened with a single guard: if the selected text was already shorter than the target word count, skip the compression model entirely. This conflated "is this long enough to need compressing" with "is this good enough to ship", and on one real job it meant the text summarisation model (distilBART) ran for one source video and was silently skipped for another, functionally identical, source — purely because of which side of a word-count threshold the two landed on. The version that was skipped carried visible connective-word artifacts ("...Additionally, To reach four and five we need...") stitched in by an earlier narrative-assembly step, landing mid-sentence because the preceding clause had no terminal punctuation on an auto-captioned source. The fix was to separate compression (a length question) from coherence (a text-quality question) into two independent checks, so that a narrative which is short enough to skip compression can still be routed through a lighter coherence-repair pass if it needs one. The general lesson: a cheap proxy check ("is it short enough already") is not a substitute for the property actually being protected ("does it read correctly"), and a pipeline that conflates the two will silently under-serve exactly the inputs that happen to pass the proxy check for the wrong reason.

An absolute similarity threshold is not portable across embedding models. Concept deduplication merges near-duplicate concept names using a cosine-similarity threshold on their sentence embeddings. A threshold tuned against the originally-used all-MiniLM-L6-v2 model, whose embeddings spread unrelated short strings across a comparatively narrow 0.2-0.4 range, was carried over unchanged when the project migrated to BAAI/bge-base-en-v1.5, whose embeddings pack unrelated strings noticeably closer together, above roughly 0.7. Applied unchanged, that threshold merged every concept extracted from a real 25-minute video into a single cluster labelled "work" — a silent, total loss of concept granularity that would not have been obvious without directly inspecting the output. The fix made the operating threshold self-calibrating: the higher of a fixed floor and the 95th percentile of the similarities actually observed in that video's own concept set, rather than one constant assumed to transfer between encoders. The lesson: any threshold tuned against one embedding model's geometry is a property of that model, not of the task, and must be revisited — ideally made self-calibrating — whenever the underlying model changes.

A fixed redundancy baseline measures the wrong thing for a single-topic source. An early version of the redundancy metric scored similarity between two selected passages against a fixed threshold, and reported 59% redundancy for a selection that a direct listen confirmed was in fact more diverse than the source material it was drawn from. The problem was definitional: two passages of one podcast, in one speaker's voice, on one subject, are simply going to be more similar to each other than two passages of unrelated text would be, and how much more similar depends on the embedding model. The fix was to measure redundancy relative to that specific video's own baseline — the mean nearest-neighbour similarity across all of its candidate segments — so the metric answers the question actually being asked ("are these picks more alike than an arbitrary handful from this same video would be") rather than an absolute, context-free one. The same honesty extended to evaluation: a later, more rigorous ablation study found that this redundancy penalty, once measured this way on genuinely held-out data, did not produce a measurable reduction in pairwise similarity relative to not having it at all — a result that was reported plainly rather than omitted, because a metric redesigned to ask the right question can still reveal that the mechanism it is checking does not yet do what it was built to do.

Concurrency limits for a local model server are a correctness fix, not a throughput one. Firing many concurrent requests at a locally-hosted Ollama instance for concept extraction did not speed anything up, because Ollama serialises generation internally; the effect was that most

requests sat in an internal queue burning their own per-request timeout while waiting rather than generating, so they then failed with a timeout error against a server that was, from the outside, perfectly healthy. Bounding concurrency to two in-flight requests fixed this, and a matching design choice — setting the HTTP client's own timeout below the asyncio-level timeout wrapping it — was necessary because asyncio's timeout cannot actually kill a background OS thread already running a blocking call, so the inner timeout has to fire first or the outer one does nothing but abandon a request that keeps running and keeps holding a server slot.

9.2 Future Work

Speaker diarisation as a segmentation signal. The semantic segmenter currently derives topic boundaries purely from sentence-embedding similarity; a speaker change is a strong, currently unused boundary signal, particularly for interview and panel-discussion content where a new speaker very often also means a new sub-topic. Integrating a diarisation model and feeding speaker-change points into the boundary-detection logic alongside the existing similarity signal is the most immediately promising unimplemented improvement to segmentation quality.

Re-enabling visual signal by default for visually-driven content. Visual scoring (via CLIP and the keyframe extractor) is implemented, tested, and currently disabled by default in favour of a lighter audio-energy scorer tuned for single-speaker podcast and lecture content, which is what the project's own evaluation corpus is predominantly composed of. Extending the evaluation to visually-driven content (demonstrations, product reviews, visual tutorials) and re-enabling and re-tuning the visual signal specifically for that content type is a natural next step rather than a rewrite, since the code path already exists.

Human evaluation of narrative ordering and overall summary quality. The project's own held-out ablation study explicitly flagged that its narrative-reordering component's benefit to a human reader, and the system's semantic-efficacy metric more generally, remain marked "pending human annotation" (evaluation/human_annotation_sheet.md and human_rating_protocol.md already define the protocol for this). Running that already-designed human evaluation to completion is the clearest remaining gap between what has been measured and what the system's design is actually trying to achieve.

Enabling LLM-backed concept extraction on appropriate hardware. The Ollama-backed concept-extraction path is complete and covered by the same test suite as the TF-IDF

fallback, but is disabled by default purely because of a measured CPU-latency constraint on the project's development hardware. Deploying the platform on GPU-class hardware (or a smaller/faster local model) and re-enabling this path would very likely improve concept quality (the LLM path extracts named, explained concepts with supporting reasoning text, versus the fallback's bare keyphrases) without requiring new engineering work, only a different deployment environment.

Horizontal scaling of the output workers. The durable, lease-based task-claiming design in output_task_queue.py was built specifically to be correct under multiple concurrent worker instances of the same type, even though the current deployment runs exactly one video worker and one text worker. Actually running multiple replicas of each behind the existing lease mechanism, rather than a single instance of each, would let the platform's rendering throughput scale with deployed hardware rather than being bounded by one worker process per output type.

A CI/CD pipeline. The project's 548-test suite currently runs manually. Wiring pytest into a GitHub Actions workflow on every pull request, and extending it to build and smoke-test the Docker images described in Chapter 4's deployment diagram, would catch a regression before it reaches a deployed environment rather than depending on a developer remembering to run the suite locally.

A native mobile application. The current platform is web-only. Given how much of the user experience already revolves around a job completing in the background while the user is doing something else, a companion mobile app (or an installable PWA as a lighter first step) that can notify a user when a job reaches video_ready or completed is a natural extension of the existing architecture rather than a new one.

REFERENCES

[1] Sommerville, I., "Software Engineering," 10th ed., Pearson Education, 2015.

[2] International Organization for Standardization, "ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," ISO/IEC, Geneva, Switzerland, 2011. Available: https://www.iso.org/standard/35733.html

[3] Jones, M. B., Bradley, J., and Sakimura, N., "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. Available: https://www.rfc-editor.org/rfc/rfc7519

[4] FastAPI, "FastAPI Documentation," 2024. Available: https://fastapi.tiangolo.com/

[5] Meta Platforms, Inc., "React Documentation," 2024. Available: https://react.dev/

[6] MongoDB, Inc., "MongoDB Manual," 2024. Available: https://www.mongodb.com/docs/manual/

[7] yt-dlp contributors, "yt-dlp Documentation," 2024. Available: https://github.com/yt-dlp/yt-dlp

[8] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I., "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv:2212.04356, OpenAI, 2022. Available: https://arxiv.org/abs/2212.04356

[9] Reimers, N., and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP-IJCNLP 2019, Association for Computational Linguistics, 2019. Available: https://arxiv.org/abs/1908.10084

[10] Lewis, M., Liu, Y., Goyal, N., et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," arXiv:1910.13461, 2019. Available: https://arxiv.org/abs/1910.13461

[11] Johnson, J., Douze, M., and Jegou, H., "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, 2019 (FAISS library). Available: https://github.com/facebookresearch/faiss

[12] Ollama, "Ollama Documentation," 2024. Available: https://ollama.com/

[13] python-jose contributors, "python-jose Documentation," 2024. Available: https://python-jose.readthedocs.io/

[14] Docker, Inc., "Docker Compose Documentation," 2024. Available: https://docs.docker.com/compose/

[15] pytest developers, "pytest Documentation," 2024. Available: https://docs.pytest.org/

[16] NLLB Team et al., "No Language Left Behind: Scaling Human-Centered Machine Translation," arXiv:2207.04672, Meta AI, 2022. Available: https://arxiv.org/abs/2207.04672

APPENDIX A:
ENVIRONMENT SETUP AND INSTALLATION GUIDE

A.1 Overview

This appendix documents how to actually run VidFusion from source, for a reader who wants to reproduce the system rather than only read about it. Two supported paths exist: running the backend and frontend directly for day-to-day development, or running the whole stack with Docker Compose as a single reproducible unit, which is also the project's deployment target.

A.2 Prerequisites

Python 3.12 or later (tested on 3.12.3), Node.js v20 or later (tested on v20.20.0) with npm, ffmpeg v6.1 or later installed and reachable on the system PATH, a running MongoDB 7 instance (local, Dockerised, or a cloud Atlas URI), and Git. No GPU is required for any run mode; every model in the pipeline is sized to run on CPU, and the LLM-backed concept-extraction path is disabled by default specifically because it is not practically usable on CPU-only hardware (Chapter 6).

A.3 Option A — Local development (Python + Node directly)

Set up the Python virtual environment and install backend dependencies:

1. python3 -m venv venv

2. source venv/bin/activate (or venv\Scripts\activate on Windows)

3. pip install -r requirements.txt

This also installs yt-dlp into the virtual environment. Install frontend dependencies separately:

4. npm install

Configure the environment by copying .env.example to .env and filling in the required values: MONGODB_URI (defaults to a local instance), MONGODB_DATABASE, and SECRET_KEY, generated with python3 -c "import secrets; print(secrets.token_urlsafe(48))". Set ENVIRONMENT=production only when actually deploying, since the configuration module deliberately refuses to start in that mode with a weak or missing signing key, which is the system failing fast on a misconfiguration rather than booting insecurely. REACT_APP_YOUTUBE_API_KEY is required for the frontend's own keyword-search feature to reach the YouTube Data API; no other API key is strictly required, since Ollama, Gemini-era legacy integrations, and ElevenLabs are all optional or unused by the current default pipeline.

Pre-warm the model cache (optional but recommended before a live demonstration, to avoid a slow first request while HuggingFace downloads the summarisation, embedding, and transcription models):

5. python prewarm_demo.py

Start MongoDB (if not already running elsewhere):

6. docker compose up -d mongo

Run the two halves in separate terminals:

backend: python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 (serves the API on http://localhost:8000, with interactive API docs at /docs) frontend: npm start (the React dev server on http://localhost:3000)

Run the automated test suite:

7. PYTHONPATH=. pytest tests/ backend/tests/ -q

This exercises the semantic segmenter, concept extractor, coverage selector, summarisation policy, the durable output-task queue and worker-recovery logic, and the rights/acquisition services described in Chapter 5, entirely without a reachable MongoDB instance or live model downloads for the large majority of the 548 tests.

A.4 Option B — Docker Compose (whole stack, production topology)

The whole stack — the FastAPI app, a dedicated text-output worker, a dedicated video-output worker, and a MongoDB container — is brought up with:

docker compose up -d --build

The first build installs ffmpeg and the Python dependencies inside the image, so it takes several minutes; subsequent builds are cached. Once running, the API is reachable at http://localhost:8000 and its health check at http://localhost:8000/health. Setting SEPARATE_OUTPUT_WORKERS_ENABLED=true (the default in docker-compose.yml) is what makes the app container stop at evidence-manifest publication and hand off to the two independent worker containers, rather than processing both output components inline within the API process itself.

Named Docker volumes (audio_cache, faiss_index) are shared across all three application containers, so a container rebuild or docker compose down does not discard in-progress

media or force the FAISS search index to be rebuilt from scratch; only docker compose down -v removes those volumes, and is used deliberately rather than by accident.

A.5 Common operational commands

docker compose ps: container status for every service. docker compose logs -f video-worker: follow highlight-reel render progress and ffmpeg output specifically. docker compose logs -f text-worker: follow summarisation-stage progress specifically. docker compose restart app: restart only the API without rebuilding. docker compose down: stop and remove containers while keeping the named volumes (generated media and the search index survive). docker compose down -v: stop and remove containers and volumes together (destructive).

A.6 Known setup caveats

A FAISS dimension-mismatch error at backend startup (seen after changing the configured embedding model, e.g. between all-MiniLM-L6-v2 and BAAI/bge-base-en-v1.5) is resolved by deleting the faiss_index directory and restarting, since the index must be rebuilt rather than mixed between two different embedding dimensions; the backend's own startup lifespan code additionally guards against this automatically for a known set of expected dimensions. A "429 Too Many Requests" error on YouTube search specifically indicates the configured REACT_APP_YOUTUBE_API_KEY is exhausted or invalid, not a backend fault. A video that consistently fails to download is often resolved by upgrading yt-dlp (pip install -U yt-dlp inside the virtual environment), since YouTube's internal APIs change frequently enough that an outdated downloader is a common, easily-fixed cause. Ollama being entirely absent is an expected, supported configuration, not an error state — backend startup logs a warning and the concept-extraction and chat features transparently use their non-LLM fallbacks.

A.7 Key backend API routes referenced in this document

A short index of the routes exercised by the use cases in Chapter 3 and the test cases in Chapter 5: POST /api/signup and /api/login for account creation and authentication; POST /api/v1/merge and GET /api/v1/merge/{job_id} for job creation and status polling, with GET /api/v1/merge/{job_id}/stream for live Server-Sent Events progress; GET /api/v1/merge/{job_id}/result for the completed output; POST /api/v1/merge/{job_id}/retry-{video|text} for component-level retry; GET /api/v1/search for semantic transcript search; POST /api/v1/chat for grounded question-answering; the rights routes under /api/v1/rights for attestation, permission recording, and job rights manifests; and GET /health, used by the Docker healthcheck.